UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS

Detección de productos dañados mediante redes neuronales convolucionales con monitoreo en tiempo real en la línea de producción
Caso: Industria de Alimentos Ltda (Lacteos y Helados)

ASIGNATURA:
- PROYECTO INTEGRADOR INTERMEDIO II

DOCENTE:
- ING. JUAN CARLOS CUELLAR SONCO

ESTUDIANTE:
- JOSE ANGEL MAMANI ESCARZO

La Paz - Bolivia
2026

ÍNDICE

I. INTRODUCCIÓN..... 1
1.2. Antecedentes..... 4
1.2.1. Antecedentes de la Institución..... 4
1.2.2. Antecedentes del problema..... 6
1.2.3. Antecedentes del proyecto de investigación..... 7
1.3. Argumentación da la importancia del objeto de estudio..... 11
1.4. Novedad científica..... 12
II. DESARROLLO..... 13
CAPÍTULO 1: MARCO TEÓRICO..... 14
1.1. Fundamentos de la Industria de Helados y Lácteos..... 15
1.1.1 Concepto de industria alimentaria..... 15
1.2. Fundamentos de la Industria Láctea..... 16
1.2.1. El helado..... 16
1.2.2. Definición del helado..... 16
1.2.3. Características, propiedades del helado..... 17
1.2.4. Clasificación tipos de helados comerciales..... 17
1.2.5. Materias primas, insumos..... 18
1.3. Proceso productivo del helado..... 18
1.3.1. Preparación de la mezcla..... 19
1.3.2. Maduración..... 19
1.3.2.1. Congelación..... 19
1.3.3. Batido, endurecimiento..... 20
1.3.4. Maquinaria..... 20
1.3.4.1. Tecnologías de refrigeración industrial..... 21
1.4. Fundamentos de sistemas..... 21
1.5. Definición de sistema..... 21
1.6. Propiedades de los sistemas..... 22
1.7. Clasificación de sistemas..... 22
1.7.1. Sistemas de información..... 23
1.8. Arquitectura del software..... 23
1.9. Definición de arquitectura del software..... 24
1.10. Modelos arquitectónicos..... 24
1.11. Patrones de arquitectura del software..... 24
1.11.1. MVC..... 24
11.1.1.1. Modelo..... 25
11.1.1.2 Vista..... 25
11.1.1.3. Controlador..... 25
1.12. Tecnologías de desarrollo..... 26

1.13. Lenguajes de programación..... 27
1.13.1. Python.....27
1.13.2. PHP.....27
1.14. Frameworks de desarrollo..... 28
1.14.1. Framework web Laravel.....28
1.14.2. Vue.js..... 28
1.15. Entorno de desarrollo integrado IDE..... 29
1.16. Librerías.....29
1.16.1. OpenCV..... 29
1.16.2. PyTorch..... 30
1.16.3. Ultralytics..... 30
1.16.4. YOLO v8 (You Only Look Once versión 8)..... 30
1.17. Servidores web..... 31
1.17.1. Apache.....31
1.18. Control de versiones.....32
1.18.1. Conceptos de versionamiento..... 32
1.18.2. Repositorios locales.....32
1.18.3. Repositorios remotos.....32
1.18.4. Herramienta Git.....33
1.18.4.1 Plataforma GitLab.....33
1.19. Interfaces API REST.....33
1.19.1. Métodos HTTP.....34
1.20. Gestión de bases de datos..... 34
1.21. Fundamentos de bases de datos..... 34
1.22. Modelos de bases de datos.....35
1.23. Sistemas gestores de bases de datos.....35
1.23.1. PostgreSQL..... 35
1.24. Metodologías de desarrollo..... 36
1.25. Metodología de desarrollo ágil.....36
1.25.1. Scrum.....37
1.25.1.1. Roles.....37
1.25.1.1.1. Product Owner.....37
1.25.1.1.2. Scrum Master.....38
1.25.1.1.3. Equipo de Desarrollo..... 38
1.25.1.2. Eventos.....38
1.25.1.3. Artefactos.....39
1.25.1.3.1. Historias de Usuario.....39
1.25.1.3.2. Producto Backlog.....39
1.25.1.3.3. Sprint Backlog.....40
1.25.2. Metodología CRISP-DM..... 40
1.25.2.1. Comprensión del Negocio.....41

1.25.2.2. Comprensión de los Datos....41
1.25.2.3. Preparación de Datos....42
1.25.2.4. Modelado....42
1.25.2.5. Evaluación....43
1.25.2.6. Despliegue....43
1.26. Lenguaje de Modelado Unificado (UML)....44
1.27. Modelado de sistemas....45
1.27.1. Herramienta StarUML....45
1.28. Diagramas UML....45
1.28.1. Diagrama de casos de uso....46
1.28.2. Diagrama de paquetes....47
1.28.3. Diagrama de clases....48
1.28.4. Diagrama de secuencia....49
1.28.5. Diagrama de colaboración....50
8.4. Diagrama de estados....51
1.29. Inteligencia Artificial....52
1.30. Fundamentos de inteligencia artificial....52
1.31. Técnicas de inteligencia artificial....53
1.32. Modelos y algoritmos....53
1.33. Herramientas de inteligencia artificial....54
1.34. Redes neuronales convolucionales....54
1.34.1. Funcionamiento de las redes neuronales convolucionales....55
1.34.2. Aplicación de las CNN en control de calidad....55
1.35. Métricas de calidad....56
1.36. Integración del modelo inteligente....57
1.37. Normativas....57
1.37.1. Seguridad de la información....58
1.37.2. Seguridad de software....58
1.37.3. Estándares ISO....59
1.37.3.1. ISO IEC 27002....59
1.37.3.2. ISO IEC 25010....59
1.38. Pruebas de software....60
1.38.1. Ejecución de pruebas....60
1.38.1. Prueba de caja negra....61
1.39. Análisis financiero del proyecto....61
1.39.1. Estimación de costos....62
1.39.1.1. Modelo COCOMO II....62
1.39.1.1.1. Punto de función....63
1.39.1.1.2. Cálculo de esfuerzo....64
1.39.1.1.3. Cálculo del tiempo de desarrollo....64
1.39.2. Indicadores de rentabilidad....65

1.39.2.1. VAN Valor Actual Neto.....65
1.39.2.2. TIR Tasa Interna de Retorno.....66
CAPÍTULO 2: JUSTIFICACIÓN DEL PROYECTO.....67
2.1. Justificación Técnica..... 68
2.2. Justificación Económica..... 69
2.3. Justificación Social..... 70
2.4. Justificación Legal..... 71
CAPÍTULO 3: DISEÑO TEÓRICO DE LA INVESTIGACIÓN..... 73
3.1. Problemática..... 74
3.2. Formulación del problema.....74
3.3. Delimitación del contenido..... 75
3.4. Delimitación temporal.....76
3.5. Delimitación espacial.....76
3.6. Objetivos..... 76
3.6.1. Objetivo general.....76
3.6.2. Objetivos específicos..... 77
CAPÍTULO 5: DESCRIPCIÓN DEL PROYECTO.....78
5.1. Introducción a la Metodología de Análisis SCRUM.....79
5.2. Fase 1 – Pre Game..... 79
5.2.1. Análisis de Requerimientos.....79
5.2.1.1. Requerimientos Funcionales..... 80
5.2.2. Historias de Usuario.....81
5.2.3. Producto Backlog..... 83
5.2.4. Sprint Backlog..... 86
5.3. Fase 2 – Game.....88
5.3.1. Análisis y Diseño.....88
5.3.1.1. Diagrama de Paquetes..... 88
5.3.1.2. Diagrama de Casos de Uso.....89
5.3.1.3. Diagrama de Secuencia.....96
5.3.1.4. Diagrama de Colaboración..... 103
5.3.1.5. Diagrama de Actividades.....107
5.3.1.6. Diagrama de Estados.....114
5.3.1.7. Diagrama de Clases de Alto Nivel..... 118
5.3.1.8. Diagrama de Clases de Bajo Nivel..... 119
5.3.1.9. Diagrama Físico.....120
5.3.1.10. Mapa Navegacional del sistema.....121
5.3.1.11. Arquitectura del Software.....125
5.4. Fase 3 – Post Game.....128
5.4.1. Seguridad del Software.....128

5.4.2. Pruebas de software..... 129
5.4.2.1. Prueba de Caja Negra..... 129
5.4.2.2. Prueba de Caja Negra Sprint Nº1 Módulo Sesiones..... 130
5.4.2.3. Prueba de Caja Negra Sprint Nº5 Módulo Inventario..... 132
5.4.2.4. Prueba de Caja Negra Sprint Nº6 Módulo Trazabilidad.....134
5.4.2.4. Prueba de Caja Negra Sprint Nº7 Módulo Reportes..... 136
5.4.3. Métricas de Calidad del Software..... 137
5.4.3.1. Funcionalidad.....137
5.4.3.2. Confiabilidad..... 140
5.4.3.3. Fiabilidad.....141
5.4.3.4. Usabilidad..... 142
5.4.3.5. Capacidad de movimiento..... 143
5.4.3.6. Portabilidad.....144
5.4.4. Presupuesto del Proyecto.....145
5.4.4.1. Cálculo de Puntos de Función Ajustados..... 146
5.4.4.2. Cálculo de Kilo Líneas de Código (KDLC).....146
5.4.4.3. Factores de Escala..... 147
5.4.4.4. Multiplicadores de Esfuerzo.....148
5.4.4.5. Cálculo de Esfuerzo.....148
5.4.4.6. Cálculo del Tiempo de Desarrollo.....148
5.4.4.7. Cálculo de Cantidad de Personas..... 149
5.4.4.8. Costo del Software.....149
5.4.4.9. Costo del Hardware..... 150
5.4.4.10. Costo de la Investigación.....151
5.4.4.11. Costo Total del Proyecto..... 151
III. CONCLUSIONES.....152
IV. RECOMENDACIONES..... 154
V. BIBLIOGRAFÍA.....156
VI. WEBGRAFÍA.....158
VII. ANEXOS..... 160

ÍNDICE DE FIGURAS

Figura 1. Organigrama de la Industria..... 5
Figura 2. Modelo - Vista - Controlador.....26
Figura 3. Roles de SCRUM..... 38
Figura 4. SCRUM procesos.....40
Figura 5. CRISP-DM procesos..... 44
Figura 6. Ejemplo de casos de uso..... 47
Figura 7. Ejemplo de diagrama de paquetes.....48
Figura 8. Ejemplo de diagrama de clases..... 49
Figura 9. Ejemplo de diagrama de secuencia..... 50
Figura 10. Ejemplo de diagrama de colaboración..... 51
Figura 11. Ejemplo de diagrama de estado.....52
Figura 12. Arquitectura de las Redes Neuronales Convolucionales..... 56
Figura 13. Diagrama de Paquetes..... 88
Figura 14. Diagrama de Casos de Uso - Sesiones..... 89
Figura 15. Diagrama de Casos de Uso - Detección Automática..... 90
Figura 16. Diagrama de Casos de Uso - Monitoreo en tiempo real..... 91
Figura 17. Diagrama de Casos de Uso - Alertas automáticas.....92
Figura 18. Diagrama de Casos de Uso - Inventario de Productos..... 93
Figura 19. Diagrama de Casos de Uso - Trazabilidad.....94
Figura 20. Diagrama de Casos de Uso - Reportes.....95
Figura 21. Diagrama de Secuencia - Sesiones..... 96
Figura 22. Diagrama de Secuencia - Detección Automática..... 97
Figura 23. Diagrama de Secuencia - Monitoreo en tiempo real..... 98
Figura 24. Diagrama de Secuencia - Alertas Automáticas..... 99
Figura 25. Diagrama de Secuencia - Inventario de Productos..... 100
Figura 26. Diagrama de Secuencia - Trazabilidad.....101
Figura 27. Diagrama de Secuencia - Reportes..... 102
Figura 28. Diagrama de Colaboración - Sesiones.....103
Figura 29. Diagrama de Colaboración - Detección Automática.....103
Figura 30. Diagrama de Colaboración - Monitoreo en tiempo real.....104
Figura 31. Diagrama de Colaboración - Alertas Automáticas.....104
Figura 32. Diagrama de Colaboración - Inventario de Productos.....105
Figura 33. Diagrama de Colaboración - Trazabilidad..... 105
Figura 34. Diagrama de Colaboración - Reportes..... 106
Figura 35. Diagrama de Actividades - Sesiones.....107
Figura 36. Diagrama de Actividades - Detección Automática..... 108
Figura 37. Diagrama de Actividades - Monitoreo en Tiempo Real..... 109
Figura 38. Diagrama de Actividades - Monitoreo en Tiempo Real..... 110

Figura 39. Diagrama de Actividades - Inventario de Productos..... 111
Figura 40. Diagrama de Actividades - Trazabilidad..... 112
Figura 41. Diagrama de Actividades - Reportes..... 113
Figura 42. Diagrama de Estado - Sesiones..... 114
Figura 43. Diagrama de Estado - Detección Automática..... 114
Figura 44. Diagrama de Estado - Monitoreo en tiempo real..... 115
Figura 45. Diagrama de Estado - Alertas Automáticas..... 115
Figura 46. Diagrama de Estado - Inventario Productos..... 116
Figura 47. Diagrama de Estado - Trazabilidad..... 116
Figura 48. Diagrama de Estado - Reportes..... 117
Figura 49. Diagrama de Clases de Alto Nivel..... 118
Figura 50. Diagrama de Clases de Bajo Nivel..... 119
Figura 51. Diagrama Físico..... 120
Figura 52. Mapa Navegacional - Administrador..... 121
Figura 53. Mapa Navegacional - Supervisor..... 122
Figura 54. Mapa Navegacional - Operador..... 123
Figura 55. Mapa Navegacional - Distribuidor..... 124
Figura 56. Arquitectura del Software..... 127
Figura 57. Cronograma de actividades..... 161
Figura 58. Árbol de problemas..... 173
Figura 59. Árbol de Objetivos..... 174

ÍNDICE DE TABLAS

Tabla 1. Implementación de una red neuronal para la detección de anomalías en bandejas..... 7
Tabla 2. Detección de defectos mediante análisis estadístico.....8
Tabla 3. Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales..... 9
Tabla 4. Diseño e implementación de una red neuronal para la detección de anomalías.. 10
Tabla 5. Requisitos Funcionales..... 81
Tabla 6. Historias de Usuario..... 83
Tabla 7. Producto Backlog..... 86
Tabla 8. Sprint Backlog..... 87
Tabla 9. Tabla de Casos de Uso - Sesiones..... 89
Tabla 10. Tabla de Casos de Uso - Detección Automática.....90
Tabla 11. Tabla de Casos de Uso - Monitoreo en tiempo real..... 91
Tabla 12. Tabla de Casos de Uso - Monitoreo en tiempo real..... 92
Tabla 13. Tabla de Casos de Uso - Inventario de Productos.....93
Tabla 14. Tabla de Casos de Uso - Trazabilidad..... 94
Tabla 15. Tabla de Casos de Uso - Reportes..... 95
Tabla 16. Seguridad del Software.....129
Tabla 17. Prueba de Caja Negra Sprint Nº1 Módulo Sesiones..... 131
Tabla 18. Prueba de Caja Negra Sprint Nº5 Módulo Inventario..... 133
Tabla 19. Prueba de Caja Negra Sprint Nº6 Módulo Trazabilidad.....135
Tabla 20. Prueba de Caja Negra Sprint Nº7 Módulo Reportes.....136
Tabla 21. Dominio de información de punto de función.....138
Tabla 21. Entradas para el cálculo de funcionalidad..... 139
Tabla 22. Cálculo de puntos de cuenta total.....139
Tabla 23. Resultados de facilidad de uso del sistema.....142
Tabla 24. Factor Técnico..... 145
Tabla 25. Resumen de métricas de calidad del software - ISO/IEC 25010.....145
Tabla 26. Cálculo de kilo de líneas de código.....147
Tabla 27. Factores de escala - COCOMO II.....147
Tabla 28. Costo total del software.....150
Tabla 29. Costo total del hardware..... 151
Tabla 30. Costo de la Investigación.....151
Tabla 31. Resumen del presupuesto total del proyecto.....152
Tabla 31. Tabla - Camaras.....162
Tabla 32. Tabla - destinos.....162
Tabla 34. Tabla - detecciones..... 163

Tabla 35. Tabla - distribuciones..... 163
Tabla 36. Tabla - ingreso _detalle..... 164
Tabla 37. Tabla - ingreso_revisión..... 165
Tabla 38. Tabla - inventario_productos..... 165
Tabla 39. Tabla - lineas_produccion..... 166
Tabla 40. Tabla - lotes..... 167
Tabla 41. Tabla - modelos_deteccion..... 167
Tabla 42. Tabla - producto_resetas..... 168
Tabla 43. Tabla - productos..... 168
Tabla 44. Tabla - reportes..... 169
Tabla 45. Tabla - roles..... 169
Tabla 46. Tabla - salida_detalle..... 170
Tabla 47. Tabla - salida_revision..... 171
Tabla 48. Tabla - tipo_defecto..... 171
Tabla 49. Tabla - users..... 172

I. INTRODUCCIÓN

La industria alimentaria representa uno de los sectores productivos de mayor relevancia económica en Bolivia, siendo un pilar fundamental para el abastecimiento de productos de consumo masivo y la generación de empleo en distintas regiones del país. Desde sus inicios, la empresa logró posicionarse como referente del sector lácteo nacional, ofreciendo productos funcionales y nutricionales de calidad para sus clientes en Bolivia bajo la marca Panda.

A lo largo de su trayectoria, la Industria de Alimentos Ltda. (INAL). Desarrolló una amplia cartera de productos para satisfacer al consumidor boliviano. Entre sus productos destacan helados en cono, paletas heladas, helados en vasito y tortas heladas, productos que han acompañado a generaciones de consumidores en todo el país, también tenemos que tomar en cuenta el incremento en la demanda representa un desafío directo para los procesos internos de la empresa, respectivamente al control de calidad dentro de sus líneas de producción, donde la velocidad y el volumen de producción exigen una inspección exigente y continua.

Los procesos de control de calidad de la Industria de Alimentos Ltda. (INAL), se han mantenido bajo modalidades tradicionales de inspección visual manual, donde los operarios deben revisar los productos de forma continua durante toda la jornada productiva. Esta situación genera limitaciones como la fatiga visual, como consecuencia, productos con defectos visuales como empaques rotos, sellados incorrectos o deformaciones físicas pueden continuar avanzando dentro del proceso productivo sin ser detectados oportunamente.

El avance de la tecnología ha abierto nuevas posibilidades para la automatización de procesos industriales en el sector alimentario, permitiendo incorporar herramientas de inteligencia artificial y visión computacional que superan ampliamente las capacidades de la inspección manual. Modelos de detección de objetos basados en redes neuronales

convolucionales, han demostrado resultados sobresalientes en entornos industriales, siendo capaces de analizar imágenes en tiempo real e identificar defectos visuales con alta precisión y en fracciones de segundo.

En respuesta a esta necesidad, el presente proyecto propone el desarrollo de un sistema inteligente de control de calidad basado en redes neuronales convolucionales para la identificación automática de productos dañados en la línea de producción de Industria de Alimentos Ltda. (INAL), integrando monitoreo en tiempo real, gestión de alertas automáticas, registro de evidencia fotográfica, módulo de inventario y trazabilidad de productos. La implementación de este sistema representa un paso significativo hacia la modernización tecnológica de los procesos productivos de la empresa.

1.1. Presentación del problema de investigación.

El control de calidad en los procesos de producción industrial es un factor fundamental para garantizar que los productos cumplan con los estándares requeridos antes de ser distribuidos al mercado. En esta actualidad muchas industrias continúan realizando la inspección de productos de manera manual, lo que implica la participación directa de operarios encargados de detectar defectos visuales durante la línea de producción.

En el contexto de la empresa Industria de Alimentos Ltda. (INAL), dedicada a la producción de alimentos, este proceso presenta diversas limitaciones, tales como la posibilidad de algunas fallas humanas, fatiga visual y dificultades para mantener un monitoreo constante en tiempo real.

El problema se intensifica en escenarios de alta producción, donde la velocidad de la línea dificulta una inspección adecuada, reprocesos y posibles afectaciones a la imagen de la empresa.

Esta situación genera ineficiencia en el control de calidad, incrementa el margen de error y dificulta la toma de decisiones basada en datos. Ante este escenario, se evidencia la necesidad de desarrollar un sistema inteligente que permita la identificación automática de productos dañados en línea de producción mediante el uso de redes neuronales convolucionales, integrando monitoreo en tiempo real, generación de alertas y almacenamiento de evidencia visual.

1.2. Antecedentes

1.2.1. Antecedentes de la Institución.

Industria de Alimentos Ltda. (INAL), es una empresa boliviana dedicada a la producción y comercialización de productos alimenticios, principalmente en el sector lácteo y de helado. Es una empresa creada en mayo de 1988 con el fin de producir helados. Comenzó con un capital de 400.000 dólares. Inició sus actividades conformado por tres socios, 8 trabajadores y 1 vehículo de distribución. Sus primeros productos fueron helados. La empresa ha logrado posicionarse dentro del sector alimentario, contribuyendo al abastecimiento de productos esenciales en distintas regiones del país.

Cuenta con sucursales en El Alto, La Paz y en otras ciudades, pero la ubicación principal se encuentra en El Alto, La Paz específicamente en la carretera a Viacha Km 8, Villa Concepción 605, en El Alto, La Paz.

La empresa cuenta con procesos de producción organizados en líneas, donde se desarrollan diferentes etapas como la elaboración, procesamiento, empaquetado y distribución de

los productos. Estos procesos requieren un control constante de calidad para garantizar que los productos cumplan con los estándares establecidos antes de su comercialización.

La empresa ha experimentado un crecimiento sostenido impulsado por la demanda constante de productos alimenticios. Este crecimiento ha permitido el desarrollo de procesos productivos organizados en líneas de producción, donde se llevan a cabo diferentes etapas como la elaboración, empaquetado y distribución de los productos.

- **Organigrama de Industria de Alimentos Ltda. (INAL).**

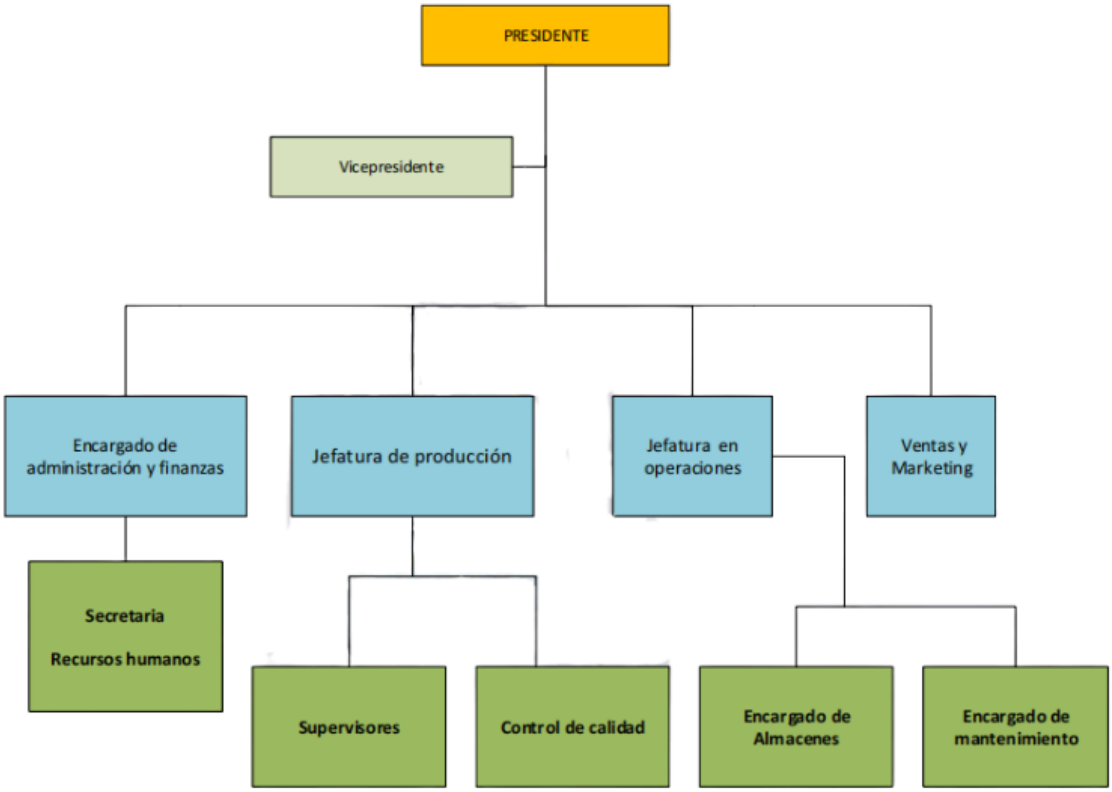


Figura 1. Organigrama de la Industria

Fuente: *Repositorio UMSA*

1.2.2. Antecedentes del problema.

La Industria de Alimentos Ltda. (INAL), ya que es una empresa consolidada en el sector alimentario a nivel nacional, cuenta con procesos productivos organizados que le permiten abastecer la demanda del mercado. Sin embargo, a pesar de su crecimiento y posicionamiento, muchos de sus procesos relacionados al control de calidad aún no incorpora una tecnología como tal, que permita optimizar la inspección de productos dentro de la línea de producción.

En este contexto, el control de calidad se realiza principalmente mediante métodos tradicionales, osea donde una persona tiene que estar revisando a cada rato, lo que dificulta mantener un seguimiento constante y preciso en cada etapa del proceso productivo. Esta situación genera limitaciones al momento de identificar productos que presentan daños y defectos, especialmente en entornos donde el volumen de producción es elevado y la velocidad de las líneas de trabajo es constante.

Estas limitaciones pueden ocasionar que productos en condiciones no óptimas continúen dentro del proceso o lleguen a etapas finales de distribución, afectando la eficiencia operativa y la calidad del producto. Se hace necesario investigar e implementar un sistema que permita optimizar el control de calidad mediante el uso de tecnologías modernas, facilitando la identificación de productos dañados y mejorando la gestión de la información dentro del entorno productivo.

1.2.3. Antecedentes del proyecto de investigación.

Autor:	Sánchez Santalices, Julián Moya de la Torre
Institución:	Universidade da coruña
Título:	Implementación de una red neuronal para la detección de anomalías en bandejas
Año de publicación:	2023
Resumen del autor:	El objetivo de este trabajo es la detección de diversos tipos de defectos que aparecen en las bandejas biodegradables durante su fabricación, más concretamente a la entrada y salida del proceso de laminado. Para ello, se emplea un sistema de visión artificial, que se va a encargar de realizar un análisis de las bandejas que vayan pasando por la línea de producción y expulsando las que considere defectuosas. Dicho sistema de visión está compuesto por tres cámaras, dos situadas a la entrada del proceso y otra a la salida, un PLC que comanda las cámaras, un sistema neumático que expulsa las bandejas defectuosas, y un ordenador exclusivamente destinado al análisis de cada imagen tomada mediante el lanzamiento de una red neuronal. El modelo desarrollado permite detectar anomalías de las bandejas durante la producción con una precisión, velocidad y efectividad muy elevada, produciendo un aumento muy significativo de la calidad de la producción.

Tabla 1. Implementación de una red neuronal para la detección de anomalías en bandejas
Fuente: *Universidade da coruña*

Aunque el trabajo de Sánchez Santalices y Moya de la Torre (2023) aborda la detección de anomalías mediante redes neuronales en un entorno industrial, su enfoque se limita principalmente a la identificación automática de defectos en tiempo real dentro de la línea de producción, utilizando un sistema de visión artificial conectado a cámaras, un PLC y mecanismos físicos de expulsión. Sin embargo, este proyecto no contempla un sistema integral de gestión y monitoreo como el que se quiere hacer en este proyecto.

Autor:	Aguirre Fernando Estévez
Institución:	MAESA-Madrid
Título:	Detección de defectos mediante análisis estadístico no-paramétrico de imágenes digitales y redes neuronales convolucionales
Año de publicación:	2024
Resumen del autor:	El sistema parte de un dron autónomo que captura imágenes, que tienen que ser analizadas y procesadas, para detectar defectos. Como las técnicas de comparación de imágenes, o partes de ellas, con patrones predefinidos, píxel a píxel, supone un coste computacional considerable, en este trabajo se ha desarrollado una tecnología para análisis y procesado de imágenes, en dos pasos. Primeramente, en lugar de emplear imágenes, se utiliza una función estadística representativa de dicha imagen digital, como es el histograma, para, mediante técnicas de análisis no-paramétricas definir un estadístico de contraste, basado en el test Z de Kolmogorov-Smirnov (KS), que, aplicado a una serie de histogramas, permite definir un parámetro de medida, tal que, mientras este parámetro se encuentre dentro de márgenes, las imágenes pueden considerarse similares y no hay defecto, siendo posible caracterizar diferentes grupos de tipología de defectos. Posteriormente, aplicando redes neuronales convolucionales, se pueden confirmar los defectos obtenidos con KS, con una cierta probabilidad.

Tabla 2. Detección de defectos mediante análisis estadístico
Fuente: *google scholar*

Un proyecto que propone una metodología innovadora para la detección de defectos mediante el uso de histogramas y redes neuronales convolucionales, su enfoque se centra principalmente en la optimización del procesamiento de imágenes y reducción del costo computacional. Este proyecto está orientado al análisis técnico de imágenes capturadas por un dron, sin contemplar un entorno industrial completo ni la integración con procesos productivos en tiempo real.

Autor:	Haro Armero Abel
Institución:	Universitat Politecnica de Valencia
Título:	Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales con optimizaciones específicas para hardware NVIDIA
Año de publicación:	2025
Resumen del autor:	Este trabajo de fin de grado aborda el desafío de la detección automática de defectos en objetos que se desplazan en entornos industriales, como líneas de producción. Se presenta el diseño y la implementación de un sistema completo de visión artificial basado en técnicas de aprendizaje profundo. El núcleo del sistema son los modelos de Redes Neuronales Convolucionales (CNN), específicamente unas variantes de la arquitectura YOLO, entrenadas para identificar y localizar con precisión defectos en los objetos en tiempo real. Para hacer frente a las restricciones computacionales y energéticas propias de los sistemas embebidos, los modelos se han implementado sobre aceleradores hardware de bajo consumo de la serie NVIDIA Jetson. Se han aplicado técnicas avanzadas de optimización, incluyendo la conversión a TensorRT y la exploración de diferentes precisiones numéricas, para maximizar la velocidad de inferencia (FPS) y minimizar el consumo energético (Watts) sin comprometer significativamente la precisión de la detección (mAP). El trabajo incluye un análisis exhaustivo del rendimiento del sistema bajo diversas configuraciones de hardware y software, demostrando la viabilidad de aplicar soluciones de inteligencia artificial de alto rendimiento en escenarios industriales con recursos limitados. Palabras clave: Redes Neuronales Convolucionales, Detección de defectos, Visión por computador, NVIDIA Jetson, Tiempo real, Optimización energética

Tabla 3. Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales

Fuente: *Universitat Politécnica de Valencia*

Este proyecto desarrolla un sistema avanzado de detección de defectos en tiempo real utilizando redes neuronales convolucionales optimizadas para hardware NVIDIA, su enfoque se

centra principalmente en el rendimiento técnico del modelo y la eficiencia computacional en entornos embebidos. Este proyecto prioriza la velocidad de inferencia, el consumo energético y la precisión del modelo, pero no contempla la integración de un sistema completo de gestión dentro de un entorno industrial.

Autor:	López Ruiz Javier
Institución:	Universitat Politecnica de Valencia
Título:	Diseño e implementación de una red neuronal para la detección de anomalías en piezas sobrantes de un proceso industrial
Año de publicación:	2025
Resumen del autor:	El proyecto consiste en el diseño e implementación de una red neuronal especializada en identificar defectos en piezas sobrantes de un proceso de corte industrial, determinando su viabilidad para la reutilización. Para ello, se utilizará una estructura diseñada para la captura de imágenes de las piezas, un software avanzado de inteligencia artificial para el análisis de estas imágenes, clasificando las piezas en aptas o no para su reutilización basándose en la detección de defectos o contaminaciones. Este enfoque innovador no solo apunta a reducir el desperdicio de material en la industria, sino también a optimizar el ahorro de costos y promover un proceso de producción más sostenible y respetuoso con el medio ambiente.

Tabla 4. Diseño e implementación de una red neuronal para la detección de anomalías
Fuente: *Universitat Politécnica de Valencia*

Este proyecto propone el diseño de una red neuronal para identificar defectos en piezas sobrantes y determinar su reutilización, su enfoque se limita principalmente a la clasificación de piezas mediante análisis de imágenes. Este proyecto está orientado a la optimización del uso de materiales y la sostenibilidad, pero no contempla la implementación de un sistema integral dentro de un entorno industrial.

1.3. Argumentación da la importancia del objeto de estudio.

Este proyecto requiere crear un sistema que permita a la empresa Industria de Alimentos Ltda. (INAL), mejorar el control de calidad dentro de su proceso de producción, especialmente en la identificación de productos dañados. Este sistema busca ser una solución tecnológica que permita ayudar a optimizar la supervisión de los productos en tiempo real, permitiendo detectar fallas de manera más rápida y eficiente durante la línea de producción.

Este proyecto incorpora el uso de redes neuronales convolucionales, las cuales permiten que el sistema aprenda a reconocer defectos en los productos mediante el análisis de imágenes. De esta manera el sistema toma decisiones basadas en el aprendizaje obtenido durante su entrenamiento, esto con un monitoreo continuo mediante el uso de cámaras, logrando identificar productos defectuosos de forma automática, hacer que los productos en mal estado continúen en el proceso y mejorar la calidad final antes de su distribución. Se podrán registrar las detecciones realizadas, lo que facilitará llevar un mejor control y seguimiento de los productos.

También hay que considerar las limitaciones de iluminación en sistemas de visión artificial, se evidencia la importancia de ubicar correctamente las cámaras dentro de la industria. Una mala iluminación puede generar sombras, reflejos o imágenes poco claras, afectando directamente la precisión de la red neuronal convolucional. En sí hay que hacer un reposicionamiento de las cámaras en zonas donde exista una iluminación adecuada o controlada, permitiendo obtener imágenes más claras y consistentes, claro si este ya cuenta con la cámara en una zona con iluminación no será necesario moverla.

1.4. Novedad científica

La novedad científica de este proyecto se basa en la incorporación de la visión artificial aplicada al control de calidad dentro del proceso de producción. Con los métodos tradicionales, donde la supervisión depende principalmente de la observación humana, este enfoque permite que el sistema analice imágenes capturadas en tiempo real para identificar características visuales de los productos, como deformaciones, fallas en el empaque o mal sellado. De esta manera, se introduce un enfoque más innovador y objetivo, capaz de procesar grandes cantidades de información visual de forma continua.

El uso de visión artificial no solo se limita a la detección, sino que también aporta en la generación de evidencia visual y en la trazabilidad de los productos, permitiendo registrar y analizar cada producto detectado. Este aporte busca que el proyecto integre el análisis visual automatizado con un sistema de gestión que facilita la toma de decisiones y mejora el control de calidad en la industria.

II. DESARROLLO.

CAPÍTULO 1:

MARCO TEÓRICO

1.1. Fundamentos de la Industria de Helados y Lácteos

La industria alimentaria transforma materias primas de origen agrícola, pecuario y otros insumos en alimentos aptos para el consumo. Su actividad integra abastecimiento, procesamiento, conservación, envasado, distribución y control de calidad. En este sector, la inocuidad, la continuidad de la cadena de frío y la uniformidad del producto son condiciones esenciales para proteger al consumidor y mantener la competitividad.

El rendimiento de esta actividad se mide a través de controles de recepción, higiene, temperatura, formulación y conservación. Cada tarea debe documentarse en procedimientos de trabajo para reducir la variabilidad entre lotes, prevenir pérdidas de insumos y sostener una calidad homogénea hasta la entrega final.

1.1.1 Concepto de industria alimentaria

Una industria alimentaria es una organización productiva que aplica procesos técnicos, controles sanitarios y recursos humanos para convertir insumos en alimentos elaborados. Sus operaciones se orientan a asegurar volumen, calidad, trazabilidad y disponibilidad. La gestión del proceso debe controlar variables como tiempo, temperatura, higiene, formulación y condiciones de almacenamiento.

La eficacia de esta labor se valora mediante verificaciones de recepción, limpieza, temperatura, dosificación y almacenamiento. Cada actividad requiere estar descrita en procedimientos operativos para reducir diferencias entre lotes, evitar desperdicio de materia prima y conservar una calidad constante hasta que el producto llega al cliente.

1.2. Fundamentos de la Industria Láctea

La industria láctea complementa a la industria heladera al aportar conocimientos técnicos sobre el manejo de leche, crema y sus derivados. Su actividad también integra abastecimiento, procesamiento, conservación y control de calidad de los insumos lácteos, garantizando la inocuidad y la continuidad de la cadena de frío desde la recepción hasta la entrega del producto final.

El resultado se comprueba con controles de ingreso de insumos, higiene, temperatura, formulación y conservación. Cada paso del proceso debe registrarse en procedimientos de trabajo, de manera que se reduzcan las variaciones entre lotes, se eviten pérdidas de materia prima y se conserve una calidad pareja hasta la entrega.

1.2.1. El helado

El helado es un alimento congelado obtenido a partir de una mezcla líquida o semilíquida que incorpora agua, sólidos lácteos o vegetales, azúcares, grasa, estabilizantes y saborizantes según su formulación.

El nivel de cumplimiento de esta actividad se mide con controles de recepción, saneamiento, temperatura, formulación y conservación. Cada operación se documenta en procedimientos de trabajo para limitar diferencias de un lote a otro, prevenir la pérdida de insumos y mantener uniforme la calidad hasta que el producto es entregado.

1.2.2. Definición del helado

El helado se define por su estructura compleja, que combina una fase congelada de agua, aire incorporado, glóbulos de grasa y componentes disueltos o dispersos.

Se evalúa a partir de controles de ingreso, limpieza, temperatura, formulación y almacenamiento. Cada operación debe estar definida en procedimientos operativos, con el propósito de reducir la variación entre lotes, evitar pérdidas de insumos y sostener la uniformidad de la calidad hasta la entrega.

1.2.3. Características, propiedades del helado

La textura cremosa del helado depende del tamaño de los cristales de hielo, del contenido de grasa, de la estabilidad de la emulsión y del aireado. Su calidad sensorial se aprecia por el sabor, color, cuerpo, derretimiento uniforme y ausencia de cristales gruesos.

El desempeño de esta labor productiva se verifica mediante controles de recepción, higiene, temperatura, dosificación y conservación. Cada operación requiere estar plasmada en un procedimiento de trabajo, con el fin de disminuir variaciones entre lotes, prevenir pérdidas de materia prima y preservar una calidad uniforme hasta el momento de la entrega.

1.2.4. Clasificación tipos de helados comerciales

Los helados comerciales pueden clasificarse por su base de elaboración: helados de crema o lácteos, helados de leche, helados de agua, sorbetes y productos vegetales o sin lactosa. También se distinguen por su presentación, como paletas, conos, vasos, barras, sándwiches y envases familiares. Cada tipo exige parámetros específicos de formulación, congelación, empaque y almacenamiento.

La consistencia de esta actividad se comprueba con controles de recepción, limpieza, temperatura, formulación y conservación de insumos. Cada operación debe quedar establecida en procedimientos de trabajo para reducir las diferencias entre lotes, evitar pérdidas de materia prima y garantizar una calidad uniforme hasta que el producto llega al consumidor.

1.2.5. Materias primas, insumos

Las materias primas más frecuentes son leche, crema, leche en polvo, azúcar, agua, frutas, cacao y otros ingredientes que definen el sabor y valor nutricional. Los insumos incluyen envases, etiquetas y materiales de embalaje. Los estabilizantes, emulsificantes, colorantes y saborizantes se emplean en cantidades controladas para mejorar textura, estabilidad, apariencia y vida útil; su empleo debe responder a la normativa aplicable y a buenas prácticas de manufactura.

Se mide por medio de controles de recepción, higiene, temperatura, formulación y conservación. Cada tarea del proceso debe describirse en procedimientos de trabajo para acotar variaciones entre lotes, prevenir pérdidas de insumos y mantener homogénea la calidad hasta la entrega final del producto.

1.3. Proceso productivo del helado

El proceso productivo del helado reúne operaciones consecutivas para obtener una mezcla segura, homogénea y congelada. Empieza con la recepción y verificación de insumos, continúa con la preparación de la mezcla y termina con el envasado, endurecimiento, almacenamiento y distribución. El control de cada etapa evita defectos de calidad y variaciones entre lotes.

Se determina mediante revisiones de recepción, limpieza, temperatura, formulación y conservación. Cada operación debe estar documentada en procedimientos de trabajo para disminuir la variabilidad entre lotes, evitar pérdidas de insumos y mantener constante la calidad hasta la entrega del producto.

1.3.1. Preparación de la mezcla

Durante el mezclado se incorporan los ingredientes en proporciones definidas hasta obtener una dispersión uniforme. La pasteurización aplica calor controlado para reducir microorganismos y mejorar la seguridad de la mezcla. La homogeneización reduce el tamaño de los glóbulos de grasa y favorece una emulsión estable, lo que contribuye a una textura más fina y uniforme.

La calidad se evalúa por medio de controles de recepción, saneamiento, temperatura, formulación y conservación. Cada operación se define dentro de procedimientos de trabajo para reducir diferencias entre lotes, prevenir pérdidas de materia prima y conservar una calidad uniforme hasta que el producto sea entregado.

1.3.2. Maduración

La maduración mantiene la mezcla refrigerada durante un tiempo determinado para hidratar proteínas y estabilizantes y permitir la cristalización parcial de la grasa.

El desarrollo correcto se realiza con controles de recepción, higiene, temperatura, formulación y conservación de insumos. Cada operación debe fijarse en procedimientos de trabajo para minimizar variaciones entre lotes, evitar pérdidas de materia prima y preservar la uniformidad de la calidad hasta la entrega.

1.3.2.1. Congelación

En la congelación dinámica se extrae calor de la mezcla mientras se incorpora aire mediante batido, lo que da inicio a la formación de la estructura final del helado.

El buen funcionamiento se comprueba mediante controles de recepción, limpieza, temperatura, formulación y conservación. Cada operación debe encontrarse definida en

procedimientos de trabajo, reduciendo así las variaciones entre lotes, evitando pérdidas de materia prima y manteniendo una calidad uniforme hasta la entrega del producto final.

1.3.3. Batido, endurecimiento

El batido incorpora aire a la mezcla durante la congelación dinámica, contribuyendo a la textura final del producto. El endurecimiento completa la congelación en condiciones de baja temperatura, fija la estructura del helado y reduce el crecimiento de cristales de hielo.

El control de esta actividad se realiza a través de revisiones de recepción, higiene, temperatura, formulación y conservación. Cada operación debe registrarse en procedimientos de trabajo para reducir las variaciones entre lotes, evitar la pérdida de materia prima y sostener una calidad uniforme hasta la entrega.

1.3.4. Maquinaria

La producción utiliza tanques de mezcla, pasteurizadores, homogeneizadores, maduradores, congeladores continuos o por lotes, dosificadores, envasadoras, túneles de endurecimiento y cámaras frigoríficas.

La verificación se lleva a cabo mediante controles de recepción, limpieza, temperatura, formulación y conservación. Cada operación se plasma en procedimientos de trabajo con el objetivo de reducir variaciones entre lotes, evitar pérdida de materia prima y mantener una calidad uniforme hasta la entrega final.

1.3.4.1. Tecnologías de refrigeración industrial

Los equipos de refrigeración controlan temperatura y humedad para conservar la calidad. La instrumentación, los sensores y los sistemas de monitoreo apoyan el control de variables críticas y la detección temprana de desviaciones.

1.4. Fundamentos de sistemas

Un sistema es un conjunto de elementos relacionados que trabajan de manera coordinada para alcanzar un propósito. Sus componentes reciben entradas, ejecutan procesos, producen salidas y emplean retroalimentación para corregir o mejorar su desempeño. Esta perspectiva permite estudiar una solución tecnológica como un todo y no solo como partes aisladas.

El enfoque sistémico ayuda a identificar las dependencias que existen entre personas, datos, equipos y procedimientos. Si un componente falla, el impacto puede extenderse hacia otras actividades del sistema. Por ello, los requisitos deben precisar entradas válidas, resultados esperados, responsables y mecanismos de control para cada proceso relevante.

1.5. Definición de sistema

En un sistema de información, las entradas pueden ser datos, imágenes, instrucciones o eventos; los procesos los transforman mediante reglas y algoritmos; y las salidas se presentan como registros, alertas, reportes o decisiones de apoyo. La retroalimentación permite comparar el resultado con el objetivo y ajustar parámetros. Un sistema de inspección integra estos elementos para recibir imágenes, analizarlas, clasificar hallazgos y registrar evidencia.

Analizar el sistema en su conjunto permite detectar cómo se relacionan personas, datos, equipos y procedimientos. Una falla en un componente puede propagarse hacia otras actividades

relacionadas. Por esa razón, es necesario que los requisitos establezcan entradas válidas, resultados esperados, responsables y mecanismos de control en cada proceso relevante.

1.6. Propiedades de los sistemas

La totalidad indica que el comportamiento del conjunto no se explica únicamente por cada componente. La sinergia expresa que la coordinación puede generar resultados superiores a los de partes independientes. La homeostasis se relaciona con la capacidad de conservar condiciones de operación mediante control y retroalimentación. Los límites definen qué pertenece al sistema y qué corresponde a su entorno, mientras que la jerarquía organiza subsistemas y niveles de responsabilidad.

Ver al sistema como un todo facilita reconocer las dependencias entre las personas, los datos, los equipos y los procedimientos involucrados. Cuando falla un componente, el problema puede trasladarse a otras actividades. Por ello, cada proceso importante requiere requisitos que definan entradas válidas, resultados esperados, responsables y mecanismos de control.

1.7. Clasificación de sistemas

Los sistemas pueden ser abiertos, cuando intercambian información o recursos con su entorno, o relativamente cerrados cuando su interacción es limitada. También pueden ser naturales o artificiales, físicos o abstractos, determinísticos o probabilísticos. Un sistema informático de inspección es artificial, abierto y socio-técnico: depende de equipos, software, operadores, datos y condiciones reales de producción.

El análisis del sistema como conjunto permite identificar cómo dependen entre sí las personas, los datos, los equipos y los procedimientos. La falla de un componente puede afectar

otras actividades del proceso. En consecuencia, los requisitos deben especificar entradas válidas, resultados esperados, responsables y mecanismos de control para cada proceso clave.

1.7.1. Sistemas de información

Un sistema de información recopila, procesa, almacena y distribuye datos para apoyar las operaciones y la toma de decisiones. Puede ser administrativo, web, móvil, experto, transaccional o de monitoreo. En un entorno productivo, el sistema debe convertir eventos de la línea en información útil, accesible y trazable para supervisores y operadores.

Considerar el sistema de forma integral permite descubrir las dependencias existentes entre personas, datos, equipos y procedimientos. Cuando un componente se ve afectado, el efecto puede repercutir en otras actividades. Por tal motivo, los requisitos deben establecer entradas válidas, resultados esperados, responsables y mecanismos de control para cada proceso importante.

1.8. Arquitectura del software

La arquitectura del software define la estructura de alto nivel de una aplicación, los componentes que la forman, sus responsabilidades y la manera en que se comunican. Una arquitectura bien definida facilita la evolución, el mantenimiento, la seguridad y la integración de módulos con objetivos distintos.

La separación de responsabilidades evita concentrar funciones distintas en un mismo componente. Esta organización facilita localizar errores, realizar pruebas y modificar una parte del sistema sin afectar de forma innecesaria a las demás. La comunicación entre componentes debe estar definida mediante interfaces y reglas claras.

1.9. Definición de arquitectura del software

La arquitectura establece decisiones técnicas de largo alcance, tales como la separación entre interfaz, lógica de negocio, datos y servicios de inteligencia artificial. No se limita a un diagrama: guía la organización del código, la comunicación entre módulos y los atributos de calidad esperados, como rendimiento, escalabilidad y mantenibilidad.

1.10. Modelos arquitectónicos

Entre los modelos comunes se encuentran cliente-servidor, arquitectura por capas, monolítica, orientada a servicios y basada en la nube. En cliente-servidor, el cliente solicita funciones y el servidor procesa la lógica y los datos. La arquitectura por capas separa presentación, aplicación, dominio y persistencia, reduciendo el acoplamiento y facilitando pruebas y mantenimiento.

1.11. Patrones de arquitectura del software

Un patrón arquitectónico es una solución reutilizable para problemas recurrentes de organización de software. Ofrece una forma probada de distribuir responsabilidades, sin imponer una implementación única. Su elección debe responder a las necesidades del sistema, al tamaño del equipo y a la evolución esperada.

1.11.1. MVC

El patrón de diseño Modelo-Vista-Controlador (MVC) es una arquitectura ampliamente utilizada en el desarrollo de aplicaciones interactivas, especialmente en entornos web. Su objetivo principal es separar la lógica de negocio de la presentación y del control de entrada, lo que facilita la mantenibilidad, escalabilidad y reutilización del código.

En el proyecto, el patrón MVC es implementado a través del framework Laravel, que estructura el sistema de control de calidad de INAL en tres capas bien diferenciadas: el Modelo, la Vista y el Controlador. Esta separación permite que los módulos del sistema, como la Detección Automática, el Monitoreo en Tiempo Real, las Alertas y el módulo de Reportes, puedan desarrollarse, mantenerse y escalar, de forma independiente sin afectar los demás componentes del sistema.

11.1.1.1. Modelo

El Modelo representa la lógica de negocio, los datos y las reglas que gobiernan el funcionamiento de la aplicación. Es responsable de acceder, almacenar y procesar datos desde la base de datos, manteniéndose independiente de la interfaz de usuario, lo que permite que los datos puedan ser utilizados por múltiples vistas

11.1.1.2 Vista

La vista constituye la interfaz visual de la aplicación, con la que interactúa el usuario. Su función principal es presentar los datos del modelo de manera adecuada, mediante pantallas, formularios, tablas o gráficos. Un mismo modelo puede tener múltiples vistas, según el rol del usuario o el contexto de uso

11.1.1.3. Controlador

El controlador actúa como intermediario entre la vista y el modelo, recibiendo las entradas del usuario (como clics o envíos de formularios), procesando y actualizando el modelo o la vista según corresponda. Define la lógica del flujo de la aplicación y coordina las peticiones del usuario.

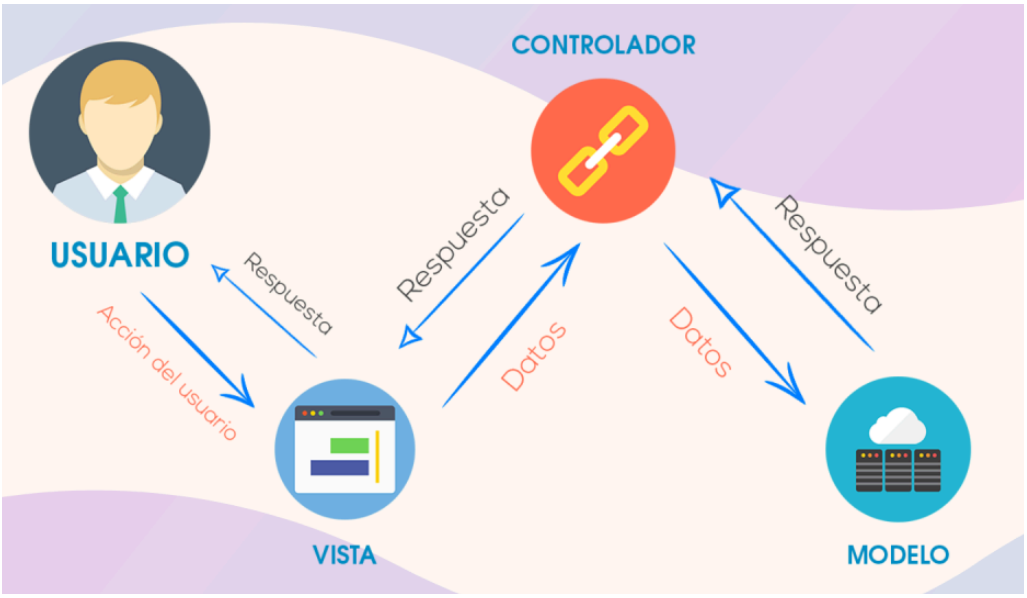


Figura 2. Modelo - Vista - Controlador

Fuente: *Tomado de Linked in https://es.linkedin.com/pulse/modelo-vista-controlador-precognis*

1.12. Tecnologías de desarrollo

Las tecnologías de desarrollo son herramientas, lenguajes, plataformas y bibliotecas que permiten construir, ejecutar y mantener una aplicación. Su selección debe considerar compatibilidad, comunidad, documentación, seguridad, rendimiento y adecuación a los requisitos del proyecto.

El uso de esta tecnología requiere una configuración documentada, control de versiones de sus dependencias y pruebas de compatibilidad. Las herramientas no resuelven por sí solas los requisitos del proyecto; deben integrarse con criterios de seguridad, rendimiento y mantenimiento para que el sistema sea sostenible durante su operación.

1.13. Lenguajes de programación

Un lenguaje de programación es un medio formal para expresar instrucciones que una computadora puede ejecutar. Existen lenguajes de bajo y alto nivel, compilados e interpretados, con distintos propósitos. La elección debe equilibrar productividad, legibilidad, ecosistema de bibliotecas y necesidades de integración.

Este concepto aporta una base para definir decisiones técnicas y operativas dentro del proyecto. Su aplicación requiere identificar el objetivo, los responsables, las condiciones de uso y la evidencia que permitirá verificar el resultado. De este modo, la documentación se mantiene vinculada con necesidades reales y con criterios que pueden ser evaluados.

1.13.1. Python

Python es un lenguaje de alto nivel caracterizado por una sintaxis legible y un ecosistema amplio para ciencia de datos, automatización y aprendizaje automático. Permite desarrollar prototipos y servicios de análisis de imágenes con rapidez. En visión artificial, se utiliza para preparar datos, entrenar modelos, ejecutar inferencias y procesar resultados.

1.13.2. PHP

PHP es un lenguaje orientado principalmente al desarrollo web del lado del servidor. Permite procesar solicitudes, aplicar reglas de negocio, acceder a bases de datos y generar respuestas para el navegador o una API. Su integración con frameworks facilita una organización estructurada del backend.

1.14. Frameworks de desarrollo

Un framework proporciona una estructura de trabajo, convenciones y componentes reutilizables para acelerar el desarrollo. A diferencia de una librería, que es llamada por el programa cuando se necesita, un framework suele controlar el flujo principal de la aplicación e invocar el código desarrollado dentro de su estructura.

1.14.1. Framework web Laravel

Laravel es un framework de PHP orientado al desarrollo de aplicaciones web. Integra enrutamiento, controladores, plantillas, acceso a datos, validación, autenticación y herramientas para crear APIs. Su organización basada en MVC ayuda a separar responsabilidades y a mantener un código más ordenado.

Para este proyecto, el Framework de Laravel se usa por su amplia variedad de estilo y con un contexto claro, asi para que el proyecto llegue a ser escalable.

1.14.2. Vue.js

Vue.js es un framework progresivo de JavaScript orientado al desarrollo de interfaces de usuario y aplicaciones web de una sola página (SPA, Single Page Application). Fue creado por Evan You en 2014 y se caracteriza por su arquitectura basada en componentes reactivos, su sistema de enlace de datos bidireccional y su integración sencilla con proyectos existentes (You, 2021).

En este proyecto, Vue.js se emplea en combinación con el motor de plantillas Blade de Laravel para construir el panel de monitoreo en tiempo real y las interfaces interactivas del sistema. La reactividad de Vue.js permite actualizar la visualización del estado de la línea de

producción, las detecciones activas y los indicadores de calidad sin necesidad de recargar la página, lo cual resulta fundamental para el módulo de monitoreo en tiempo real.

1.15. Entorno de desarrollo integrado IDE

Un entorno de desarrollo integrado reúne editor de código, herramientas de depuración, gestión de archivos, extensiones y conexión con control de versiones. Su función es concentrar actividades frecuentes de programación en un mismo ambiente. Un IDE o editor extensible mejora la productividad al ofrecer resaltado de sintaxis, autocompletado y revisión de errores.

1.16. Librerías

Una librería es un conjunto de funciones y componentes que puede incorporarse a un programa para resolver tareas específicas. Aporta capacidades concretas sin definir toda la estructura de la aplicación. Su uso reduce el trabajo repetitivo, pero exige controlar versiones, licencias, actualizaciones y compatibilidad.

1.16.1. OpenCV

OpenCV (Open Source Computer Vision Library) es una biblioteca de código abierto orientada a la visión por computadora y el aprendizaje automático. Desarrollada originalmente por Intel y actualmente mantenida por la comunidad OpenCV, esta biblioteca tiene más de 2500 algoritmos optimizados que abarcan desde operaciones básicas de procesamiento de imágenes hasta técnicas avanzadas de aprendizaje profundo.

En el presente proyecto, OpenCV cumple la función de gestionar la captura continua de imágenes desde las cámaras instaladas en la línea de producción de INAL. La biblioteca permite establecer conexiones con cámaras IP y USB, gestionar flujos de video en tiempo real, realizar

operaciones de preprocesamiento como redimensionamiento, normalización y ajuste de contraste, y transmitir los fotogramas procesados al modelo de detección YOLOv8 para su análisis.

1.16.2. PyTorch

PyTorch es un framework de aprendizaje profundo de código abierto desarrollado por Meta AI, ampliamente utilizado para la investigación y el desarrollo de modelos de inteligencia artificial, proporciona un sistema de diferenciación automática flexible que facilita la definición, entrenamiento y evaluación de redes neuronales complejas, incluyendo las redes neuronales convolucionales utilizadas en tareas de visión por computadora.

1.16.3. Ultralytics

Ultralytics es la empresa responsable del desarrollo y mantenimiento de la implementación oficial de YOLO. La librería Ultralytics proporciona una interfaz simplificada para entrenar, validar y desplegar modelos YOLO, con soporte integrado para múltiples formatos de exportación y plataformas de hardware. En este proyecto, PyTorch y Ultralytics se emplean conjuntamente para el entrenamiento del modelo de detección y su posterior integración con el sistema web desarrollado en Laravel.

1.16.4. YOLO v8 (You Only Look Once versión 8)

YOLOv8 es la octava versión del algoritmo YOLO (You Only Look Once), desarrollado por Ultralytics. Es un modelo de detección de objetos en tiempo real que se basa en redes neuronales convolucionales para identificar y localizar objetos dentro de imágenes con alta precisión y velocidad. A diferencia de los modelos de detección en dos etapas, YOLOv8 procesa

toda la imagen en una sola pasada por la red neuronal, esto nos permite alcanzar velocidades de inferencia adecuadas para aplicaciones industriales.

Introduce mejoras significativas respecto a versiones anteriores, incluyendo una arquitectura de red más eficiente, un nuevo sistema de asignación de etiquetas durante el entrenamiento y soporte nativo para tareas de detección, segmentación y clasificación. En el contexto del presente proyecto, se utiliza como núcleo del módulo de detección automática, entrenar con imágenes de los productos de INAL para identificar con precisión los distintos tipos de defectos visuales que pueden presentarse durante el proceso productivo.

1.17. Servidores web

Un servidor web recibe solicitudes mediante protocolos como HTTP o HTTPS y entrega recursos o respuestas de aplicación. Puede servir contenido estático, ejecutar lógica del backend mediante un entorno configurado y aplicar controles de seguridad. Es un componente central para publicar una aplicación y sus servicios.

Este concepto aporta una base para definir decisiones técnicas y operativas dentro del proyecto. Su aplicación requiere identificar el objetivo, los responsables, las condiciones de uso y la evidencia que permitirá verificar el resultado. De este modo, la documentación se mantiene vinculada con necesidades reales y con criterios que pueden ser evaluados.

1.17.1. Apache

Apache HTTP Server es un servidor web de código abierto que permite atender solicitudes, configurar sitios, aplicar módulos y gestionar reglas de acceso. Puede trabajar junto con PHP y actuar como servidor frontal de una aplicación. Su configuración debe considerar certificados, permisos, registros y protección frente a solicitudes no autorizadas.

1.18. Control de versiones

El control de versiones registra cambios realizados sobre archivos a lo largo del tiempo. Permite recuperar estados anteriores, comparar modificaciones, trabajar en paralelo y documentar la evolución del software. Es una práctica necesaria para reducir riesgos en proyectos individuales y colaborativos.

El historial debe conservar cambios pequeños, verificables y descritos con claridad. Antes de integrar una modificación conviene revisar el código, ejecutar pruebas y registrar el propósito del cambio. Esta disciplina reduce pérdidas de trabajo y facilita que el proyecto pueda mantenerse aun cuando cambien sus responsables.

1.18.1. Conceptos de versionamiento

Versionar implica guardar confirmaciones o commits que describen cambios coherentes. Las ramas permiten desarrollar funcionalidades o correcciones de forma aislada y luego integrarlas mediante fusiones. Un historial claro facilita identificar el origen de un cambio, revisar código y recuperar una versión estable cuando es necesario.

1.18.2. Repositorios locales

Un repositorio local es la copia del historial de versiones almacenada en el equipo de desarrollo. Permite crear commits, ramas y etiquetas sin depender de conexión permanente. Es el espacio de trabajo inmediato donde se preparan y verifican los cambios antes de compartirlos.

1.18.3. Repositorios remotos

Un repositorio remoto almacena una copia compartida del proyecto en un servidor o plataforma en línea. Facilita la colaboración, el respaldo y la revisión de cambios. La

sincronización se realiza mediante operaciones de envío y obtención de actualizaciones, manteniendo reglas claras para resolver conflictos.

1.18.4. Herramienta Git

Git es un sistema distribuido de control de versiones que registra el historial del código y de otros archivos. Operaciones como init, status, add, commit, branch, merge, pull y push forman parte de un flujo básico. Su uso responsable requiere mensajes descriptivos y revisión antes de integrar cambios.

1.18.4.1 Plataforma GitLab

GitLab es una plataforma que hospeda repositorios Git y agrega herramientas de colaboración, revisión, incidencias, automatización y documentación. Puede centralizar el trabajo del proyecto y conservar una copia remota del código. Los permisos y las ramas protegidas ayudan a cuidar la integridad del repositorio.

En el proyecto se utiliza para guardar el código del sistema, lo nuevo, las versiones para así tener un mejor control del código y que se pueda compartir y hacer actualizaciones.

1.19. Interfaces API REST

Una API REST es una interfaz que permite la comunicación entre sistemas mediante recursos identificados por direcciones web y operaciones HTTP. Sus respuestas suelen emplear formatos como JSON. Una API bien diseñada separa clientes y servicios, valida datos, autentica solicitudes y comunica errores de forma consistente.

1.19.1. Métodos HTTP

GET obtiene información, POST crea recursos o inicia procesos, PUT o PATCH actualiza información y DELETE elimina recursos según las reglas del sistema. Los códigos de estado comunican el resultado de una solicitud, por ejemplo éxito, error de validación, falta de autorización o recurso inexistente. La elección correcta del método mejora la claridad y previsibilidad de la API.

1.20. Gestión de bases de datos

La gestión de bases de datos reúne métodos y herramientas para almacenar, consultar, proteger y mantener datos de forma organizada. En un sistema de inspección, los registros deben conservar fecha, resultado, usuario, producto, evidencia y otros atributos necesarios para la trazabilidad.

Este concepto aporta una base para definir decisiones técnicas y operativas dentro del proyecto. Su aplicación requiere identificar el objetivo, los responsables, las condiciones de uso y la evidencia que permitirá verificar el resultado. De este modo, la documentación se mantiene vinculada con necesidades reales y con criterios que pueden ser evaluados.

1.21. Fundamentos de bases de datos

Una base de datos es una colección estructurada de datos relacionados. Su diseño identifica entidades, atributos, relaciones, claves y restricciones. La normalización reduce redundancia innecesaria y mejora la consistencia, mientras que los índices optimizan consultas frecuentes cuando se usan de manera justificada.

1.22. Modelos de bases de datos

Los modelos de datos describen cómo se representan y relacionan los datos. El modelo relacional organiza la información en tablas conectadas mediante claves; el modelo documental almacena documentos con estructura flexible; y otros modelos responden a necesidades específicas. La elección depende de las consultas, integridad requerida, volumen y patrón de crecimiento.

1.23. Sistemas gestores de bases de datos

Un sistema gestor de bases de datos permite crear, administrar y consultar bases de datos, además de controlar usuarios, transacciones, copias de seguridad y recuperación. Debe preservar la integridad de la información incluso cuando existen accesos concurrentes o fallos. Las políticas de respaldo y restauración son parte indispensable de su operación.

El análisis permite reconocer dependencias entre personas, datos, equipos y procedimientos. Cuando un componente presenta una falla, el efecto puede llegar a las demás actividades. Por esta razón, los requisitos deben definir entradas válidas, resultados esperados, responsables y mecanismos de control para cada proceso importante.

1.23.1. PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto. Ofrece consultas SQL, transacciones, restricciones, índices, tipos de datos y mecanismos de seguridad. Su robustez y capacidad de crecimiento lo hacen apropiado para aplicaciones que necesitan conservar registros estructurados y realizar consultas confiables.

La información debe almacenarse con reglas que eviten registros incompletos, duplicados o inconsistentes. Las claves, restricciones y relaciones permiten conservar la coherencia de los

datos. El diseño también debe prever consultas frecuentes, respaldo periódico y recuperación ante fallas para mantener la trazabilidad de las operaciones.

1.24. Metodologías de desarrollo

Las metodologías de desarrollo organizan actividades, responsabilidades y entregables para construir software de manera controlada. Ayudan a planificar, priorizar requisitos, revisar resultados y aprender de cada iteración. La metodología seleccionada debe ser coherente con el tipo de producto y la disponibilidad del equipo.

No hay una metodología universalmente correcta; depende del tamaño del equipo, de qué tan claros estén los requisitos desde el inicio y de cuánto puede cambiar el alcance sobre la marcha. Un proyecto con pocos integrantes y comunicación constante puede funcionar bien con procesos ligeros, mientras que uno con varias áreas involucradas suele necesitar más documentación y puntos de control formales.

1.25. Metodología de desarrollo ágil

El desarrollo ágil busca entregar valor de forma incremental, con ciclos cortos de trabajo y retroalimentación frecuente. Promueve la comunicación continua, la adaptación a cambios y la entrega de funcionalidades verificables. No elimina la planificación; la realiza de manera progresiva y basada en evidencia.

Adoptar agilidad no significa trabajar sin plan, sino aceptar que el plan se ajusta con lo que se va aprendiendo en cada entrega. Esto exige que el equipo revise sus resultados con cierta frecuencia y esté dispuesto a cambiar prioridades cuando la evidencia lo pida, en lugar de seguir un cronograma fijo solo porque ya estaba escrito.

1.25.1. Scrum

SCRUM es un marco de trabajo ágil para el desarrollo de software que organiza el trabajo en ciclos cortos denominados sprints, generalmente de dos a cuatro semanas. SCRUM se fundamenta en tres pilares: transparencia, inspección y adaptación, los cuales permiten al equipo de desarrollo responder con agilidad ante los cambios en los requisitos y entregar valor de manera incremental.

En el presente proyecto, SCRUM fue aplicado en la fase administrativa para organizar el desarrollo de los módulos del sistema web, permitiendo planificar y ejecutar las funcionalidades de manera incremental a través de sprints definidos. Su aplicación facilitó la gestión ordenada del trabajo individual, la identificación temprana de problemas y la entrega progresiva de componentes funcionales del sistema.

1.25.1.1. Roles

Scrum define tres roles principales: el Product Owner, el Scrum Master y el Equipo de Desarrollo. Estos tres roles colaboran estrechamente, con responsabilidades claras pero complementarias, para asegurar el correcto funcionamiento de Scrum en un proyecto.

1.25.1.1.1. Product Owner

El Product Owner es el responsable de gestionar el Product Backlog, definir las prioridades y asegurar que el equipo cumpla con los objetivos del negocio.

1.25.1.1.2. Scrum Master

El Scrum Master es un facilitador que asegura que Scrum se entienda y se aplique correctamente, ayudando al equipo a remover impedimentos y promoviendo los valores y prácticas de Scrum.

1.25.1.1.3. Equipo de Desarrollo

El Equipo de Desarrollo es un grupo multifuncional y auto-organizado que se compromete a entregar un incremento de producto terminado al final de cada Sprint.

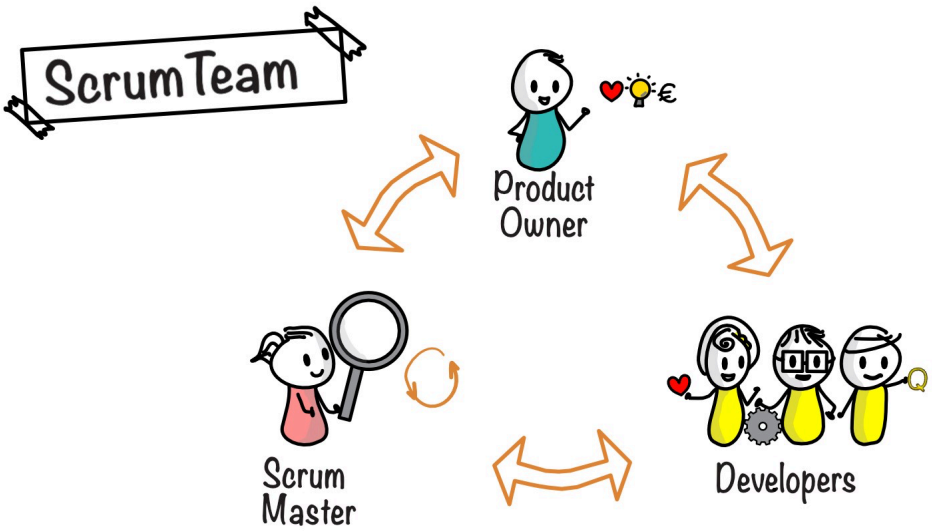


Figura 3. Roles de SCRUM

Fuente: *Tomado de ITDO*
https://www.itdo.com/blog/framework-scrum-roles-y-responsabilidades/

1.25.1.2. Eventos

Los eventos de Scrum incluyen el Sprint, la planificación del Sprint, el Daily Scrum, la revisión y la retrospectiva. Estos espacios permiten definir objetivos, sincronizar el trabajo,

mostrar resultados y mejorar el proceso. Su duración y enfoque deben ser acotados para proteger el tiempo de construcción.

Estos eventos solo aportan valor si se respetan sus tiempos; un Daily Scrum que se alarga termina convirtiéndose en una reunión de estado más, y una retrospectiva sin acciones concretas termina siendo un espacio para quejarse sin que nada cambie después.

1.25.1.3. Artefactos

Los artefactos son elementos tangibles que se utilizan para facilitar la transparencia y la comunicación en el desarrollo de un producto. Estos artefactos representan información clave sobre el trabajo que se está realizando y son utilizados por el Equipo Scrum y otras partes interesadas para comprender el estado del proyecto. Los principales artefactos en Scrum son:

1.25.1.3.1. Historias de Usuario

Las Historias de Usuario son descripciones breves y simples de una funcionalidad del sistema narradas desde la perspectiva del usuario que la requiere.

1.25.1.3.2. Producto Backlog

El Product Backlog es una lista priorizada de los requisitos, funcionalidades y cambios que se deben abordar en un producto. Es gestionado y mantenido por el Product Owner.

1.25.1.3.3. Sprint Backlog

El Sprint Backlog es el conjunto de elementos del Product Backlog seleccionados para ser abordados en un Sprint específico. Es una previsión del equipo sobre lo que se compromete a realizar durante el Sprint.

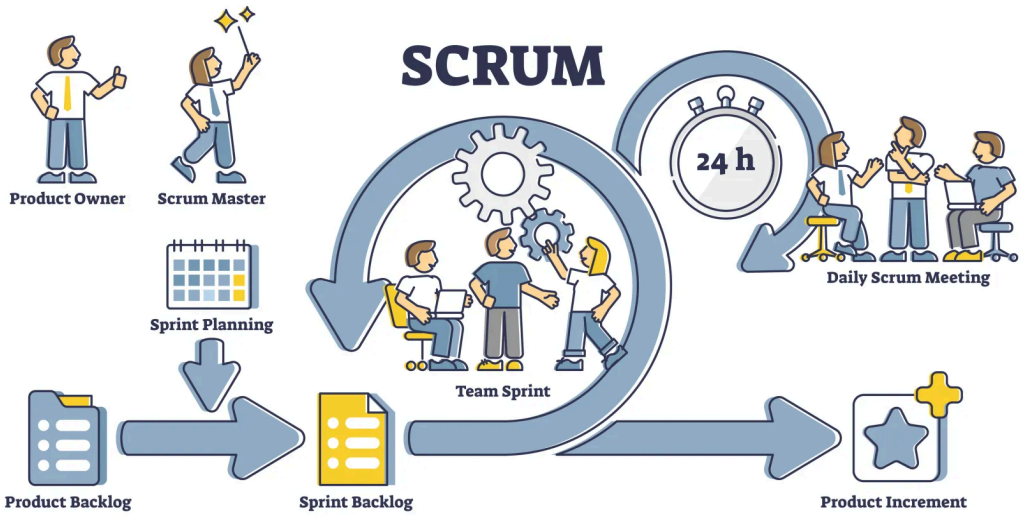


Figura 4. SCRUM procesos

Fuente: *Tomado de ILUNION https://www.ilunion.com/es/blog-puntoilunion/scrum*

1.25.2. Metodología CRISP-DM

La metodología CRISP-DM (Cross-Industry Standard Process for Data Mining) es un proceso estándar y ampliamente adoptado para el desarrollo de proyectos de minería de datos e inteligencia artificial. CRISP-DM proporciona una descripción completa del ciclo de vida de un proyecto de ciencia de datos, estructurado en seis fases iterativas que guían al equipo desde la comprensión del negocio hasta el despliegue del modelo en producción.

1.25.2.1. Comprensión del Negocio

La primera fase de CRISP-DM consiste en comprender los objetivos del proyecto desde la perspectiva del negocio y traducirlos en objetivos técnicos de ciencia de datos, esta etapa implica identificar el problema que se desea resolver, determinar los criterios de éxito del proyecto y establecer un plan de acción detallado.

En el presente proyecto, la fase de comprensión del negocio implicó analizar el proceso de control de calidad de INAL, identificar los tipos de defectos visuales más frecuentes en sus productos (empaques rotos, deformaciones físicas y sellados incorrectos) y definir los requisitos de precisión y velocidad de detección necesarios para que el modelo sea operacionalmente útil en la línea de producción.

1.25.2.2. Comprensión de los Datos

La segunda fase abarca la recopilación inicial de los datos, la exploración de sus características y la identificación de posibles problemas de calidad, la comprensión profunda de los datos disponibles es fundamental para seleccionar las técnicas de modelado más apropiadas y anticipar los desafíos del proceso de entrenamiento.

Para el proyecto, esta fase involucra la captura de imágenes de los productos de INAL en la línea de producción bajo distintas condiciones de iluminación y ángulos de captura, el análisis de la distribución de clases (productos conformes y no conformes por tipo de defecto) y la identificación de posibles problemas como desbalance de clases o imágenes de baja calidad que puedan afectar el rendimiento del modelo.

1.25.2.3. Preparación de Datos

La preparación de los datos comprende todas las actividades necesarias para construir el conjunto de datos final que será utilizado para entrenar el modelo, esta fase incluye la selección, limpieza, construcción y formateo de los datos, y típicamente consume la mayor parte del tiempo en un proyecto de inteligencia artificial.

En el proyecto, la preparación de datos abarca el etiquetado de imágenes mediante herramientas de anotación, el aumento de datos mediante técnicas de rotación, cambio de brillo y volteo horizontal para incrementar la diversidad del conjunto de entrenamiento, y la división del dataset en conjuntos de entrenamiento, validación y prueba siguiendo proporciones estándar.

1.25.2.4. Modelado

La fase de modelado corresponde a la selección y aplicación de las técnicas de aprendizaje automático sobre los datos preparados, esta etapa implica la selección del algoritmo más apropiado, la configuración de sus hiper parámetros y la ejecución del proceso de entrenamiento, ajustando iterativamente los parámetros para maximizar el rendimiento del modelo.

En el proyecto, esta fase contempla el entrenamiento del modelo YOLOv8 con el dataset de imágenes de productos de INAL mediante el framework PyTorch y la librería Ultralytics, la configuración de hiper parámetros como la tasa de aprendizaje, el tamaño del batch y el número de épocas de entrenamiento, y la evaluación del modelo durante el entrenamiento.

1.25.2.5. Evaluación

La fase de evaluación tiene como objetivo determinar si el modelo desarrollado cumple con los criterios de éxito definidos en la fase de comprensión del negocio, en esta etapa se revisan los resultados del modelado en función de los objetivos del negocio, se identifican posibles problemas no contemplados previamente y se decide si el modelo está listo para su despliegue o requiere iteraciones adicionales.

Para el proyecto, la evaluación del modelo YOLOv8 se realizará utilizando métricas estándar, los resultados obtenidos determinarán si el modelo alcanza el nivel de precisión requerido para su integración del sistema de INAL.

1.25.2.6. Despliegue

La fase de despliegue corresponde a la integración del modelo entrenado en el entorno de producción, haciendo que sus resultados estén disponibles para los usuarios finales, el despliegue no marca necesariamente el fin del proyecto, sino el inicio de un ciclo de monitoreo y mejora continua del modelo en condiciones reales de operación.

En el proyecto, el despliegue implica su exportación al formato optimizado compatible con el sistema Laravel, el desarrollo de la interfaz de comunicación entre el servicio Python de detección y el backend del sistema.

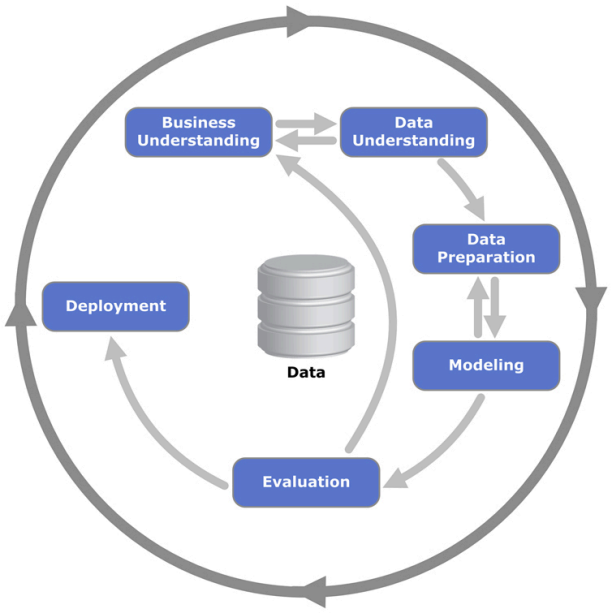


Figura 5. CRISP-DM procesos

Fuente: *Tomado de AIHealth*
https://www.ai-health.es/la-metodologia-crisp-dm-para-proyectos-de-ia/

1.26. Lenguaje de Modelado Unificado (UML)

UML es un lenguaje visual estándar utilizado para especificar, visualizar y documentar sistemas de software. Sus diagramas permiten comunicar estructuras, comportamientos e interacciones antes y durante la implementación. No sustituye al código, pero reduce ambigüedades y apoya el análisis técnico.

Un diagrama bien hecho ahorra discusiones que de otro modo tomarían horas en reuniones, y permite que alguien nuevo en el equipo entienda la estructura del sistema sin tener que leer todo el código. El riesgo aparece en el extremo opuesto, cuando se diagrama por costumbre y se termina generando documentación que nadie vuelve a consultar.

1.27. Modelado de sistemas

El modelado representa una abstracción del sistema para comprender sus componentes, reglas y relaciones. Ayuda a discutir el diseño con usuarios y desarrolladores antes de tomar decisiones costosas. Un modelo debe ser suficiente para la decisión que apoya, evitando detalle innecesarios.

Un buen ejercicio antes de programar es preguntarse qué decisión se está apoyando con el modelo; si la respuesta no es clara, probablemente el diagrama se está haciendo por trámite y no porque realmente aporte algo a esa etapa del proyecto.

1.27.1. Herramienta StarUML

StarUML es una herramienta para crear y mantener diagramas de software. Facilita organizar modelos, aplicar notación UML y exportar documentación visual. Su uso contribuye a conservar coherencia entre los diagramas y a comunicar el diseño en un formato reconocible.

Trabajar con una herramienta dedicada, en lugar de dibujar los diagramas a mano o en un editor genérico, ayuda a mantener consistencia en la notación y facilita actualizar un diagrama cuando cambia un requisito, sin tener que rehacerlo desde cero.

1.28. Diagramas UML

Un diagrama es una representación gráfica de información, datos o conceptos que utiliza símbolos, formas y líneas para mostrar relaciones, estructuras o procesos de manera visual y comprensible. Los diagramas son herramientas poderosas utilizadas en una amplia variedad de

campos, desde la ingeniería y la informática hasta la educación y los negocios, para comunicar ideas de forma clara y efectiva.

En el desarrollo del proyecto, los diagramas UML sirvieron como base para estructurar los módulos del sistema, documentar los flujos de trabajo y establecer las relaciones entre los componentes de software. A continuación se describen los tipos de diagramas UML utilizados en el proyecto.

1.28.1. Diagrama de casos de uso

El diagrama de casos de uso muestra actores externos y las funciones que esperan del sistema. Sirve para delimitar el alcance funcional y comunicar requisitos desde la perspectiva del usuario. Cada caso de uso debe expresar un objetivo reconocible y no una acción técnica aislada.

Este diagrama suele ser el primero que se dibuja porque ayuda a que usuarios y desarrolladores se pongan de acuerdo sobre qué debe hacer el sistema antes de discutir cómo se va a construir. Un caso de uso demasiado técnico casi siempre es señal de que se está mezclando el qué con el cómo.

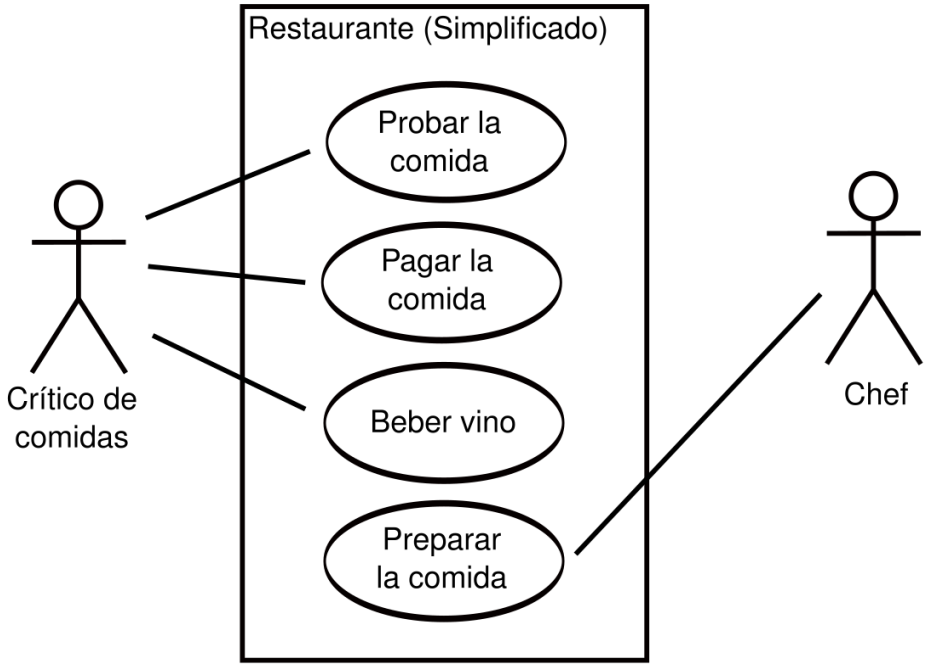


Figura 6. Ejemplo de casos de uso

Fuente: *Tomado de Wikipedia*
https://es.wikipedia.org/wiki/Archivo:UML_diagrama_caso_de_uso.svg

1.28.2. Diagrama de paquetes

El diagrama de paquetes agrupa elementos relacionados y muestra dependencias entre conjuntos de clases, interfaces u otros paquetes. Facilita entender la modularidad del sistema y limitar acoplamientos. Es especialmente útil cuando la solución crece y necesita una organización clara.

Cuando el sistema crece y aparecen decenas de clases, este diagrama permite dar un paso atrás y ver la organización general en lugar de perderse en el detalle. También ayuda a detectar dependencias cruzadas que, si no se controlan, dificultan modificar un módulo sin tocar otros.

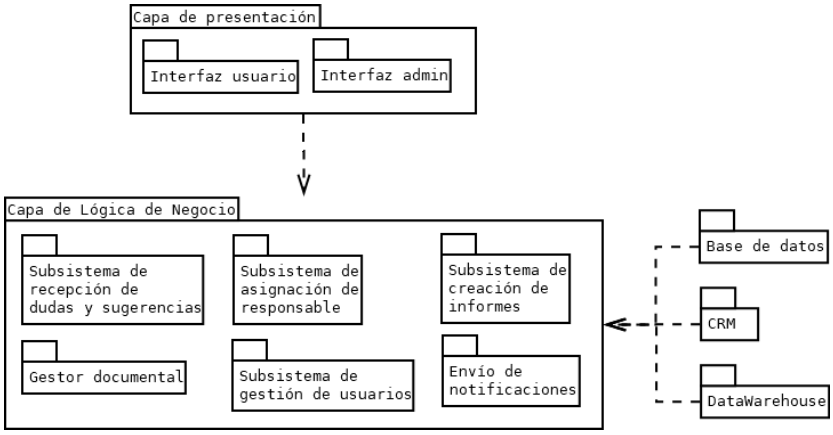


Figura 7. Ejemplo de diagrama de paquetes

Fuente: *Tomado de DiagramasUML https://diagramasuml.com/paquetes/*

1.28.3. Diagrama de clases

El diagrama de clases representa clases, atributos, operaciones y relaciones como asociación, herencia o composición. Ayuda a definir el modelo de dominio y a planificar la persistencia de datos. Debe reflejar responsabilidades coherentes y evitar relaciones innecesariamente complejas.

Diseñar este diagrama con calma antes de programar suele evitar tener que reestructurar la base de datos más adelante. Una relación mal pensada aquí, como una herencia forzada solo para reutilizar código, termina complicando el mantenimiento del sistema.

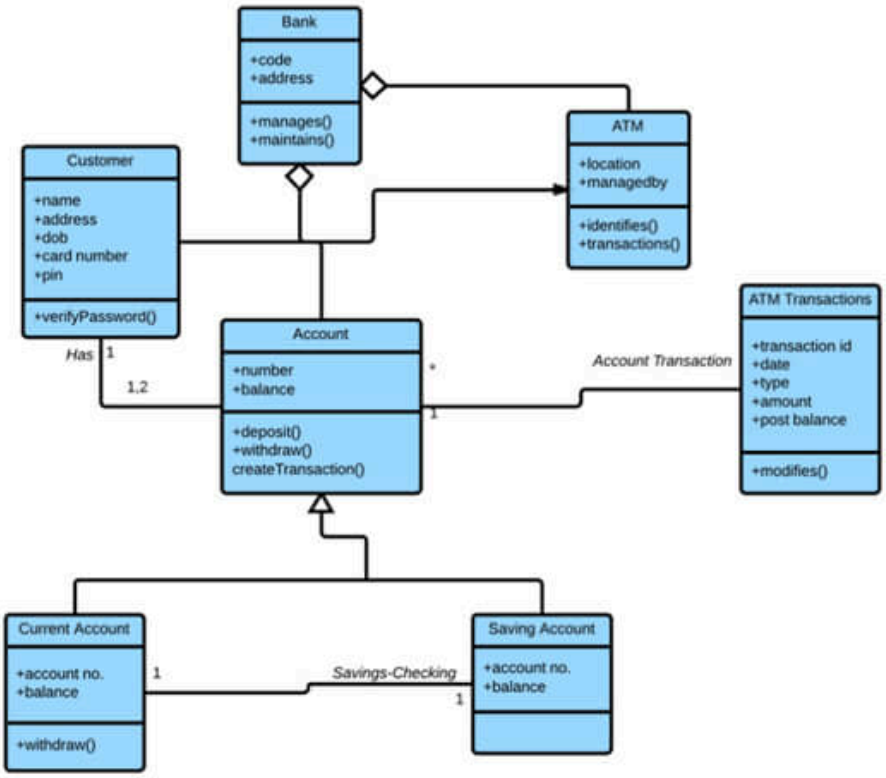


Figura 8. Ejemplo de diagrama de clases

Fuente: *Tomado de VENNGAGE https://es.venngage.com/blog/diagrama-de-clases/*

1.28.4. Diagrama de secuencia

El diagrama de secuencia muestra el orden temporal de los mensajes entre actores y objetos. Permite analizar cómo se coordina una operación, desde una solicitud inicial hasta la respuesta. Es útil para identificar validaciones, dependencias y puntos de integración.

Este diagrama resulta particularmente útil cuando una operación involucra varios componentes, por ejemplo cuando una imagen capturada pasa por preprocesamiento, inferencia del modelo y registro en base de datos antes de mostrar un resultado al usuario. Visualizar ese orden ayuda a detectar en qué punto podría fallar la comunicación entre partes.

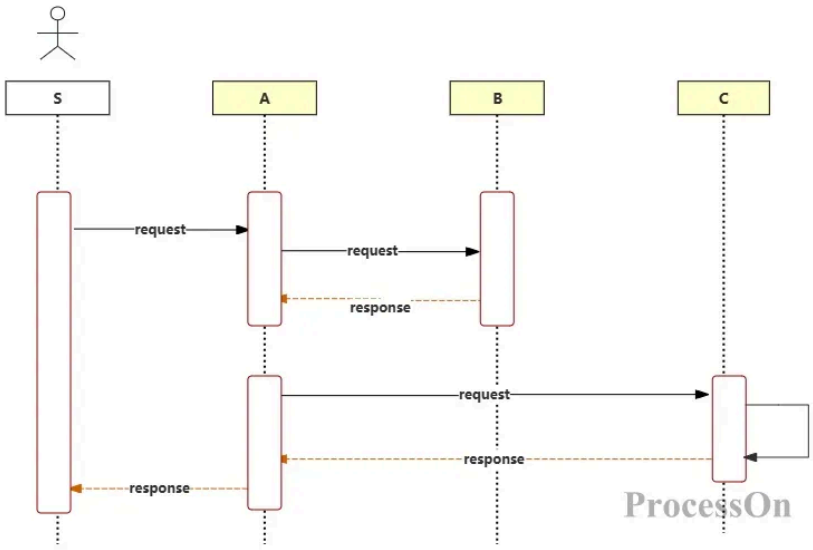


Figura 9. Ejemplo de diagrama de secuencia

Fuente: *Tomado de ProcessOn*
https://www.processon.io/es/blog/sequence-diagram-creation-tutorial

1.28.5. Diagrama de colaboración

El diagrama de colaboración, también denominado diagrama de comunicación, enfatiza los vínculos entre objetos y los mensajes que intercambian. Complementa al diagrama de secuencia porque resalta la estructura de la interacción. Ayuda a comprender qué componentes cooperan para cumplir una función.

Aunque hoy se usa menos que el de secuencia, sigue siendo útil cuando interesa más entender qué objetos están conectados entre sí que el orden exacto en que se comunican, por ejemplo al analizar el acoplamiento entre módulos del sistema.

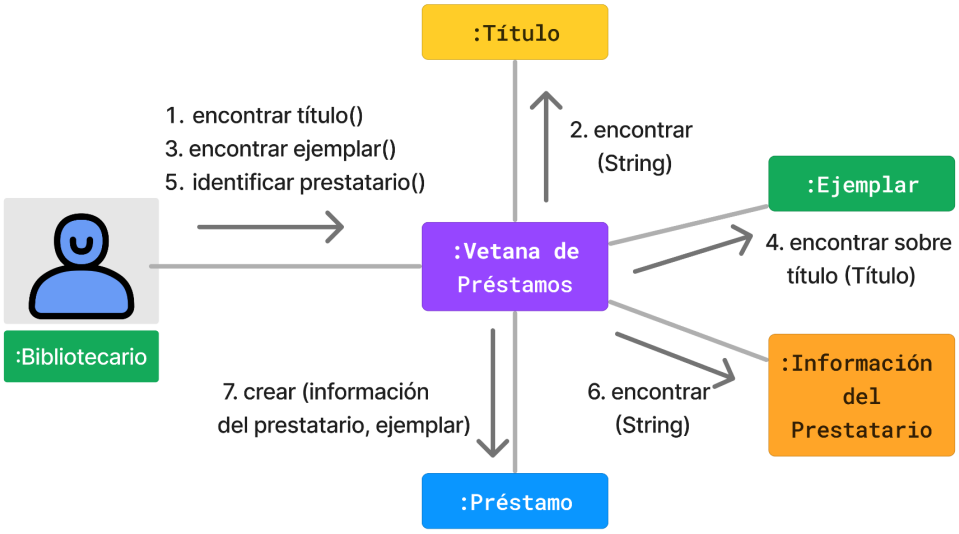


Figura 10. Ejemplo de diagrama de colaboración

Fuente: *Tomado de CODEANDO_SIMPLE*
https://codeandosimple.com/es/uml-6-diagrama-colaboracion

8.4. Diagrama de estados

El diagrama de estados describe los estados posibles de una entidad y las transiciones producidas por eventos o condiciones. Puede emplearse para modelar el ciclo de vida de una incidencia, una detección o un registro de revisión. Aporta claridad cuando el comportamiento depende del estado actual.

Este tipo de diagrama resulta útil, por ejemplo, para representar cómo cambia el estado de una incidencia: desde que se detecta, pasando por su revisión, hasta que se marca como resuelta o descartada. Sin ese modelo es fácil que el sistema permita transiciones que no deberían ocurrir, como cerrar una incidencia que nunca fue revisada.

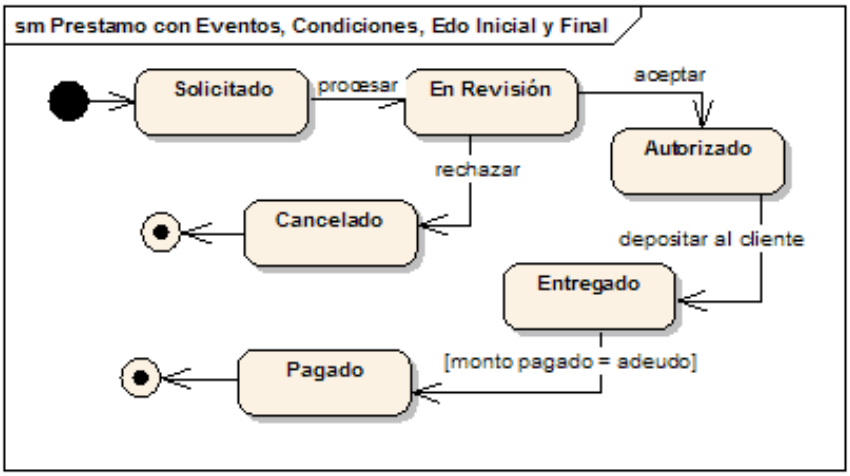


Figura 11. Ejemplo de diagrama de estado

Fuente: *Tomado de LiderDeProyecto https://www.liderdeproyecto.com/uml/uml007.html*

1.29. Inteligencia Artificial

La inteligencia artificial puede transformar imágenes de una línea productiva en resultados de clasificación o detección que apoyen la decisión humana, el registro de evidencia y la mejora continua.

Aplicar IA en un contexto de inspección visual no significa reemplazar al personal que revisa los productos, sino darle una herramienta que procese cientos de imágenes de forma consistente y le señale dónde poner atención. La decisión final de aceptar o rechazar un producto sigue recayendo, en la mayoría de los casos, en un criterio humano informado por ese apoyo.

1.30. Fundamentos de inteligencia artificial

La inteligencia artificial reúne técnicas que permiten a los sistemas realizar tareas asociadas con percepción, razonamiento, aprendizaje o toma de decisiones. Su objetivo no es reemplazar automáticamente el criterio humano, sino procesar información de manera

consistente y aportar resultados que puedan ser revisados. La calidad de sus resultados depende de los datos, el modelo y el contexto de uso.

Es común pensar que un modelo con buenos resultados en pruebas funcionará igual de bien en producción, pero eso no siempre ocurre; las condiciones reales de una línea de producción, con cambios de luz o de velocidad, pueden ser distintas a las del conjunto de datos usado para entrenarlo.

1.31. Técnicas de inteligencia artificial

Las técnicas de IA incluyen sistemas basados en reglas, aprendizaje automático, aprendizaje profundo, procesamiento de lenguaje natural y visión artificial. En aprendizaje supervisado, un modelo se entrena con ejemplos etiquetados para reconocer patrones y predecir clases o valores. La técnica debe seleccionarse según el problema, la disponibilidad de datos y la necesidad de explicación o velocidad.

1.32. Modelos y algoritmos

Un modelo es una representación aprendida o programada que recibe datos de entrada y genera una salida. Los algoritmos son procedimientos usados para entrenar, optimizar o ejecutar dicho modelo. Su desempeño debe medirse con datos no utilizados en el entrenamiento, de modo que la evaluación represente la capacidad de generalización y no solo la memorización de ejemplos.

Un error frecuente es evaluar el modelo con los mismos datos que se usaron para entrenarlo, lo cual da una sensación de precisión que luego no se sostiene con imágenes nuevas.

Separar un conjunto de prueba independiente, y no tocarlo hasta el final, es lo que permite saber si el modelo realmente aprendió a generalizar o solo memorizó los ejemplos.

1.33. Herramientas de inteligencia artificial

Las herramientas de IA incluyen lenguajes, bibliotecas, plataformas de entrenamiento, repositorios de datos y servicios de despliegue. Estas herramientas permiten preparar conjuntos de datos, anotar imágenes, entrenar modelos, calcular métricas y ejecutar inferencias. Su selección debe considerar compatibilidad, reproducibilidad, seguridad y recursos computacionales disponibles.

Elegir estas herramientas implica más que comparar qué tan rápido entrenan un modelo; también hay que considerar si el equipo cuenta con la experiencia para mantenerlas, si los datos se pueden versionar de forma ordenada y si el servicio de despliegue va a estar disponible cuando el sistema esté en operación diaria.

1.34. Redes neuronales convolucionales

Las redes neuronales convolucionales, también llamadas CNN, son modelos de aprendizaje profundo especializados en analizar imágenes. Utilizan capas de convolución que aplican filtros sobre regiones locales para identificar patrones como bordes, texturas, colores y formas. A medida que la información avanza por la red, las capas combinan características simples para reconocer patrones más complejos, por ejemplo una etiqueta, un envase o un defecto visible.

Lo que hace atractivas a las CNN para tareas de inspección visual es que no requieren definir manualmente qué características buscar en una imagen; el propio entrenamiento va

ajustando los filtros hasta que la red reconoce patrones relevantes, aunque eso también significa que su comportamiento es más difícil de explicar que el de una regla programada a mano.

1.34.1. Funcionamiento de las redes neuronales convolucionales

Una CNN recibe una imagen expresada como una matriz de píxeles. Las capas convolucionales generan mapas de características; las capas de activación introducen capacidad no lineal; y las operaciones de reducción disminuyen dimensionalidad conservando información relevante. En tareas de detección, una etapa final estima clases y posiciones de objetos. El aprendizaje ajusta los parámetros del modelo para disminuir el error entre predicciones y etiquetas reales.

Entender este proceso ayuda a diagnosticar por qué un modelo falla en ciertos casos: si las capas iniciales no logran distinguir bordes o texturas relevantes, ninguna capa posterior podrá construir sobre esa información faltante, sin importar qué tan compleja sea la arquitectura.

1.34.2. Aplicación de las CNN en control de calidad

En control de calidad, una CNN puede analizar imágenes capturadas de forma constante e identificar características asociadas a productos conformes o no conformes. La utilidad del modelo depende de contar con imágenes de buena calidad, iluminación representativa, clases definidas y etiquetas correctas. Las detecciones deben conservar evidencia visual y un nivel de confianza para permitir revisión humana, auditoría y mejora del conjunto de datos.

En la práctica, la mayor parte del esfuerzo no está en entrenar el modelo sino en construir un conjunto de imágenes representativo, con ejemplos suficientes de defectos poco frecuentes.

Un modelo entrenado con pocas imágenes de un defecto raro difícilmente lo detectará de forma confiable cuando aparezca en producción.

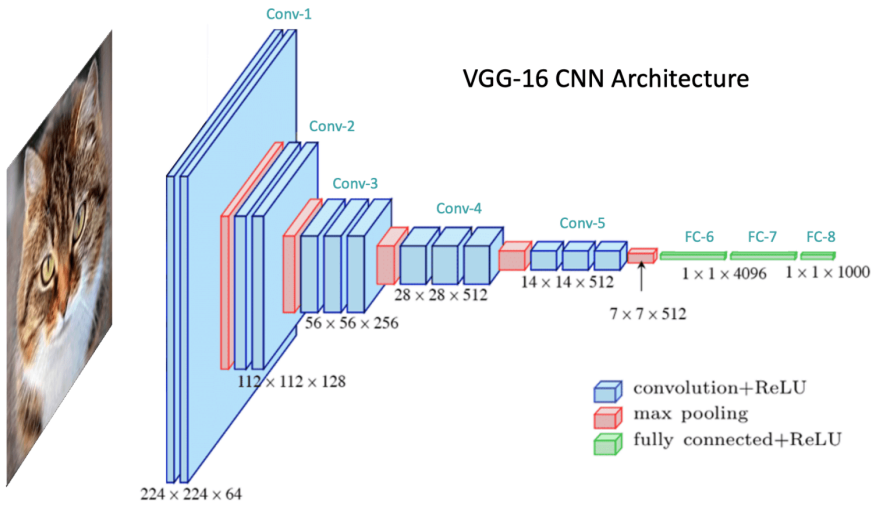


Figura 12. Arquitectura de las Redes Neuronales Convolucionales

Fuente: *Tomado de "Understanding Convolutional Neural Networks (CNN)" por LearnOpenCV (s.f.). https://learnopencov.com/understanding-convolutional-neural-networks-cnn/*

1.35. Métricas de calidad

Las métricas permiten cuantificar el desempeño de un modelo y del sistema. La precisión expresa qué proporción de detecciones positivas fue correcta; la exhaustividad o recall indica cuántos casos reales fueron detectados; y la medida F1 equilibra ambas. En detección de objetos también se emplean IoU y mAP. Las métricas deben interpretarse junto con errores relevantes para la operación, como falsos negativos de defectos críticos.

En control de calidad conviene prestar especial atención al recall sobre defectos críticos, porque un falso negativo, es decir, dejar pasar un producto defectuoso, suele costar más que un

falso positivo que solo genera una revisión adicional. Optimizar únicamente la precisión general puede ocultar ese tipo de errores.

1.36. Integración del modelo inteligente

La integración del modelo conecta captura de imágenes, preprocesamiento, inferencia, reglas de negocio, almacenamiento y visualización. Un flujo típico recibe una imagen, ejecuta el modelo, valida la confianza, registra el resultado y presenta una alerta o evidencia cuando corresponde. La integración debe manejar errores de red, disponibilidad del servicio, latencia y seguridad de los datos.

Un modelo que funciona bien en un entorno de pruebas puede fallar en producción si la integración no contempla qué pasa cuando el servicio de inferencia se cae momentáneamente o cuando la cámara entrega una imagen borrosa; el sistema debe tener una respuesta definida para esos casos y no simplemente detenerse.

1.37. Normativas

Las normativas orientan prácticas para desarrollar y operar sistemas de manera segura, confiable y verificable. Su aplicación ayuda a identificar controles necesarios y a documentar decisiones. En proyectos de software, los estándares deben adaptarse al alcance real, evitando afirmar cumplimiento formal si no se realizó una certificación o evaluación completa.

Seguir una normativa no debería tratarse como llenar una lista de requisitos para pasar una auditoría, sino como una guía para tomar decisiones razonables sobre riesgos reales del proyecto. Decir que un sistema "cumple" con un estándar sin haber pasado por una evaluación formal puede generar una falsa sensación de seguridad.

1.37.1. Seguridad de la información

La seguridad de la información protege la confidencialidad, integridad y disponibilidad de los datos. La confidencialidad evita accesos no autorizados; la integridad protege contra cambios indebidos; y la disponibilidad garantiza que los recursos estén accesibles cuando se requieran. Controles como autenticación, autorización, copias de seguridad, registros de auditoría y cifrado reducen riesgos.

En un sistema que almacena evidencia de inspecciones, perder la integridad de un registro puede ser tan grave como perder su confidencialidad: si alguien puede modificar un resultado después de guardado, la trazabilidad que se buscaba con todo el sistema deja de tener sentido.

1.37.2. Seguridad de software

La seguridad de software incorpora controles durante el diseño, construcción, pruebas y operación. Incluye validación de entradas, gestión segura de sesiones, control de permisos, tratamiento de errores, actualización de dependencias y protección de interfaces. La seguridad debe considerarse desde los requisitos y no únicamente al final del desarrollo.

Agregar seguridad al final del desarrollo casi siempre sale más caro que pensarla desde el diseño; validar una entrada que ya se usó en decenas de lugares del código obliga a revisar todos esos puntos, mientras que definirla bien desde el inicio evita ese trabajo repetido.

1.37.3. Estándares ISO

Los estándares ISO e ISO/IEC proporcionan marcos de referencia para gestionar calidad, seguridad y otros aspectos organizacionales. No constituyen por sí solos una implementación técnica, pero ayudan a estructurar controles, criterios de evaluación y documentación. Su selección debe corresponder a los riesgos y objetivos del sistema.

Estos estándares funcionan mejor como punto de partida que como receta cerrada; conviene revisar cuáles de sus recomendaciones aplican realmente al tamaño y al contexto del proyecto, en lugar de intentar cubrir cada punto sin evaluar si aporta valor.

1.37.3.1. ISO IEC 27002

ISO/IEC 27002 ofrece orientaciones para controles de seguridad de la información. Incluye temas como políticas, gestión de activos, control de acceso, criptografía, seguridad operativa, relaciones con proveedores e incidentes. Puede emplearse como guía para identificar medidas aplicables a usuarios, datos, servidores y respaldos.

De esta guía suele ser más útil enfocarse primero en el control de acceso y en la gestión de respaldos, porque son los puntos donde un descuido tiene consecuencias inmediatas, antes de abordar controles más orientados a procesos organizacionales de mayor escala.

1.37.3.2. ISO IEC 25010

ISO/IEC 25010 define un modelo de calidad del producto de software. Considera adecuación funcional, eficiencia del desempeño, compatibilidad, usabilidad, fiabilidad,

seguridad, mantenibilidad y portabilidad. Este modelo permite establecer criterios de calidad y planificar pruebas que no se limiten a comprobar que una función existe.

Usar este modelo obliga a pensar en calidad más allá de si una función existe; por ejemplo, un sistema de inspección puede tener toda la funcionalidad requerida pero fallar en fiabilidad si se cae ante una carga alta de imágenes, lo cual también cuenta como un problema de calidad.

1.38. Pruebas de software

Las pruebas de software son actividades sistemáticas destinadas a verificar requisitos, descubrir defectos y obtener evidencia sobre la calidad de un producto. Una estrategia de pruebas define qué se validará, con qué datos, quién lo hará, cuáles son los resultados esperados y cómo se registrarán las incidencias.

Definir la estrategia de pruebas antes de empezar a programar evita el escenario común donde las pruebas se dejan para el final y terminan reducidas a probar solo lo obvio, dejando sin cubrir los casos límite que suelen ser los que fallan en producción.

1.38.1. Ejecución de pruebas

La ejecución de pruebas consiste en aplicar casos definidos, comparar resultados obtenidos con resultados esperados y documentar hallazgos. Los casos deben incluir condiciones normales, límites, entradas inválidas y escenarios de error. La evidencia puede incluir capturas, registros, datos de prueba y estado de corrección de cada incidencia.

Probar únicamente con datos ideales da una falsa sensación de que el sistema funciona; los problemas reales suelen aparecer con entradas vacías, archivos corruptos o usuarios que no siguen el flujo esperado, y esos casos merecen tanto espacio en el plan de pruebas como los escenarios normales.

1.38.1. Prueba de caja negra

La prueba de caja negra evalúa el comportamiento externo del sistema sin considerar la estructura interna del código. Se diseña a partir de requisitos, entradas y salidas esperadas. Es apropiada para validar formularios, reglas de negocio, APIs, permisos y flujos desde el punto de vista del usuario.

Este tipo de prueba resulta especialmente útil cuando quien prueba no participó en el desarrollo, porque obliga a validar el sistema desde la perspectiva de quien lo va a usar, sin dar por sentado ningún conocimiento sobre cómo está construido por dentro.

1.39. Análisis financiero del proyecto

El análisis financiero estima los recursos necesarios para construir y operar una solución y evalúa si los beneficios esperados justifican la inversión. Debe basarse en supuestos explícitos, costos verificables y un horizonte temporal definido. Sus resultados orientan decisiones, pero no sustituyen la gestión de riesgos técnicos y operativos.

Antes de presentar cualquier número conviene dejar claro qué se está asumiendo: si el precio de una licencia puede cambiar, si el equipo requerirá capacitación adicional o si el volumen de uso proyectado es razonable. Sin esos supuestos explícitos, cualquier cifra final resulta difícil de defender ante quien decide invertir o no.

1.39.1. Estimación de costos

La estimación de costos identifica recursos humanos, software, infraestructura, capacitación, mantenimiento y otros gastos asociados. Una estimación útil separa costos iniciales y recurrentes, explica los supuestos y se actualiza cuando cambian el alcance o los recursos. La transparencia en los supuestos es más importante que una precisión aparente sin sustento.

Es fácil subestimar los costos recurrentes porque no se ven al inicio del proyecto; el mantenimiento del modelo, el reentrenamiento periódico y el soporte técnico continuo suelen pesar tanto o más que la inversión inicial en hardware y desarrollo.

1.39.1.1. Modelo COCOMO II

COCOMO II es un modelo de estimación que relaciona tamaño de software, factores de escala y multiplicadores de esfuerzo para aproximar personas-mes y duración. Puede aplicarse en distintos momentos del proyecto según la información disponible. Su resultado es una referencia para planificar y debe ajustarse con datos reales del equipo y el contexto.

Aplicar este modelo con datos genéricos de la industria da una primera referencia, pero conviene ajustarlo con información real del equipo apenas se tenga, porque la productividad puede variar bastante según la experiencia de las personas involucradas y las herramientas disponibles.

COCOMO II estima el esfuerzo a partir del tamaño del software, los factores de escala y los multiplicadores de esfuerzo. La expresión general utilizada para el modelo postarquitectónico es la siguiente:

$$E = A \times (KSLOC)^{B} \times \prod EM_i$$

En esta fórmula, E representa el esfuerzo en personas-mes; A es un coeficiente de calibración; KSLOC corresponde a miles de líneas de código fuente; B expresa el efecto de los factores de escala; y EM_i representa cada multiplicador de esfuerzo asociado al producto, la plataforma, el personal y el proyecto.

$$B = 0.91 + 0.01 \times \Sigma SF_i$$

El tiempo de desarrollo se estima con la duración resultante del esfuerzo y con el factor de calendario del proyecto. Esta relación permite construir un cronograma que incluya análisis, diseño, construcción, pruebas y corrección de incidencias.

$$TDEV = C \times E^{(D + 0.2 \times (B - 0.91))}$$

1.39.1.1.1. Punto de función

El punto de función es una medida del tamaño funcional que considera entradas, salidas, consultas, archivos internos e interfaces externas. Busca estimar el alcance desde la perspectiva de las funciones entregadas al usuario, independientemente del lenguaje de programación. Una medición consistente requiere reglas claras de conteo y revisión de los componentes identificados.

Esta medida resulta útil cuando se necesita comparar el tamaño de dos proyectos hechos en tecnologías distintas, algo que contar líneas de código no permite hacer de forma justa. Su principal limitación es que requiere reglas de conteo bien definidas, y dos personas sin el mismo criterio pueden llegar a números distintos.

El cálculo de puntos de función ajusta el conteo funcional según características generales del sistema. La fórmula mostrada en el documento de referencia puede expresarse de la siguiente manera:

$$PF\ ajustado = PF\ sin\ ajustar \times (0.85 + 0.01 \times FI)$$

PF sin ajustar corresponde a la suma ponderada de entradas, salidas, consultas, archivos internos e interfaces externas. FI representa el factor de influencia asociado a características técnicas. El resultado es una medida funcional que puede apoyar la estimación del tamaño del software.

1.39.1.1.2. Cálculo de esfuerzo

El cálculo de esfuerzo estima el trabajo requerido para desarrollar una solución, generalmente en personas-mes u horas. Considera tamaño, complejidad, experiencia, restricciones y herramientas disponibles. El resultado debe usarse para planificar actividades y no como una garantía exacta, porque los proyectos cambian durante su ejecución.

Un error común es tratar el resultado de este cálculo como un número fijo e inamovible; en la práctica sirve mejor como un rango de referencia que se actualiza a medida que el equipo aprende más sobre la complejidad real del proyecto.

1.39.1.1.3. Cálculo del tiempo de desarrollo

El tiempo de desarrollo estima la duración necesaria para completar el trabajo con una capacidad de equipo determinada. No siempre se reduce de manera proporcional al aumentar

personas, debido a costos de coordinación, aprendizaje e integración. Un cronograma realista incorpora pruebas, correcciones, documentación y actividades de despliegue.

Duplicar el número de personas en un proyecto casi nunca reduce el tiempo a la mitad, porque coordinar a más gente también consume tiempo; por eso conviene tratar estas estimaciones como una guía y no como una promesa exacta al momento de comprometerse con una fecha de entrega.

1.39.2. Indicadores de rentabilidad

Los indicadores de rentabilidad comparan inversión, costos y beneficios esperados a lo largo del tiempo. Son herramientas para apoyar una decisión financiera y dependen de la calidad de los flujos proyectados. Deben acompañarse de análisis de sensibilidad cuando existen supuestos inciertos.

Ningún indicador por sí solo cuenta toda la historia; un proyecto puede verse rentable en el papel y aun así fracasar si los supuestos sobre adopción o ahorro de tiempo resultan demasiado optimistas frente a lo que ocurre en la operación real.

1.39.2.1. VAN Valor Actual Neto

El Valor Actual Neto, VAN, calcula el valor presente de los flujos futuros descontados menos la inversión inicial. Un VAN positivo indica que, bajo la tasa y los supuestos utilizados, el proyecto genera valor por encima de la rentabilidad mínima exigida. La interpretación debe declarar el horizonte, la tasa de descuento y las fuentes de los beneficios estimados.

Un VAN positivo no garantiza que el proyecto sea una buena decisión si la tasa de descuento usada no refleja el riesgo real de la inversión; conviene probar el cálculo con distintas tasas para ver qué tan sensible es el resultado a ese supuesto.

$$VAN = \Sigma\ [Flujo\ neto\ de\ efectivo_t\ /\ (1 + r)^t] - Inversión\ inicial$$

1.39.2.2. TIR Tasa Interna de Retorno

La Tasa Interna de Retorno, TIR, es la tasa de descuento que hace que el VAN sea igual a cero. Se compara con una tasa mínima requerida o costo de oportunidad para apoyar la decisión de inversión. La TIR debe analizarse junto con VAN, riesgos y consistencia de los flujos, especialmente cuando existen cambios de signo o escenarios variables.

Cuando los flujos de efectivo cambian de signo más de una vez, es posible obtener más de una TIR válida matemáticamente, lo cual complica su interpretación; en esos casos conviene apoyarse más en el VAN que en la TIR para tomar la decisión final.

$$0 = \Sigma\ [Flujo\ neto\ de\ efectivo_t\ /\ (1 + TIR)^t] - Inversión\ inicial$$

CAPÍTULO 2:

JUSTIFICACIÓN DEL PROYECTO

2.1. Justificación Técnica.

Las justificaciones técnicas describen las herramientas seleccionadas para el desarrollo del sistema, destacando su funcionalidad, ventajas y el motivo de su elección dentro del proyecto de control de calidad en la industria alimentaria.

- **Laravel**: Se utiliza porque implementa el patrón MVC (Modelo-Vista-Controlador), lo que permite separar la lógica de negocio, la interfaz y el control del sistema; esto facilita el mantenimiento, escalabilidad y organización del código, además de ofrecer herramientas integradas para autenticación, rutas y seguridad, haciendo más eficiente el desarrollo del sistema web.
- **PostgreSQL**: Se elige debido a su alta confiabilidad, integridad y seguridad en el manejo de datos; permite gestionar grandes volúmenes de información de manera eficiente, soporta transacciones complejas y garantiza consistencia, lo cual es fundamental para almacenar registros de producción y control de calidad.
- **Blade** y **Vue.js**: Se emplean en el frontend para construir interfaces dinámicas y reactivas; Blade facilita la generación de vistas integradas con Laravel, mientras que Vue.js permite actualizar datos en tiempo real sin recargar la página, mejorando la experiencia del usuario en el monitoreo de la producción.
- **YOLOv8**: Se utiliza como núcleo del sistema porque permite detectar objetos (productos defectuosos) en tiempo real mediante visión artificial, ofreciendo alta precisión y velocidad, lo que resulta ideal para entornos industriales donde se requiere respuesta inmediata.

- **Python**, **PyTorch** y **Ultralytics**: Se emplean para el desarrollo del modelo de inteligencia artificial; Python facilita la programación por su simplicidad, PyTorch permite entrenar redes neuronales de manera flexible y eficiente, y Ultralytics proporciona una implementación optimizada de YOLOv8 que mejora la precisión y el rendimiento del modelo.
- **OpenCV**: Se utiliza para la captura y procesamiento de imágenes, permitiendo la conexión con cámaras en la línea de producción y el análisis continuo de video, lo que hace posible detectar defectos en tiempo real.

2.2. Justificación Económica.

Industria de Alimentos Ltda. (INAL), se beneficiará económicamente mediante la reducción de pérdidas generadas por productos defectuosos que actualmente avanzan en la línea de producción sin ser detectados, evitando costos innecesarios de reproceso, desperdicio de materiales.

La empresa minimizará los gastos operativos relacionados a la inspección manual, optimizando el uso del personal en áreas de mayor productividad. Al detectar defectos de forma automática y en tiempo real, se reduce el margen de error humano que actualmente genera pérdidas económicas difíciles de cuantificar y controlar.

El tiempo de detección de un producto dañado está directamente relacionado al costo que este genera dentro del proceso productivo. En consecuencia, el presente proyecto se orienta a reducir ese tiempo de detección mediante la automatización inteligente, disminuyendo el impacto

económico de cada falla y permitiendo a la empresa tomar decisiones oportunas basadas en datos reales.

2.3. Justificación Social.

El sistema beneficia directamente a los operarios y supervisores de la línea de producción, ya que automatiza la tarea de inspección visual, reduciendo la carga laboral repetitiva, disminuyendo la fatiga acumulada durante las jornadas de trabajo y evitando la presión que implica detectar defectos de forma manual de manera continua. También beneficia al personal del área de control de calidad, quienes contarán con información precisa y en tiempo real para tomar decisiones más rápidas y fundamentadas, mejorando la coordinación entre las distintas áreas de la empresa.

De manera indirecta, el proyecto favorece a los consumidores finales de los productos, quienes recibirán productos en mejores condiciones al existir un control de calidad más riguroso y automatizado, lo cual protege su salud y refuerza su confianza en la marca. La empresa tendrá un beneficio socialmente ante sus clientes osea a las tiendas o lugares que son distribuidos osea al mercado en general. El proyecto garantizará una seguridad y bienestar para todos los que interactúan con la empresa, optimizando el trabajo diario del personal osea de los trabajadores de la empresa garantizando un producto mejor recibido.

De manera más amplia, el proyecto también busca beneficiar al sector industrial, ya que promueve el uso de tecnologías basadas en visión artificial dentro de los procesos productivos, esto permite que otras empresas puedan tomar este tipo de soluciones como referencia,

fomentando la modernización, la mejora en la calidad de los productos y el avance tecnológico en el contexto industrial de Bolivia.

2.4. Justificación Legal.

La justificación legal se fundamenta en el cumplimiento de normativas vigentes relacionadas con la calidad de productos alimenticios, la seguridad industrial y el manejo adecuado de la información dentro de sistemas tecnológicos. El desarrollo del sistema propuesto se alinea con las regulaciones establecidas por entidades nacionales encargadas de garantizar la inocuidad alimentaria y la protección del consumidor.

El proyecto se encuentra respaldado por las disposiciones del Servicio Nacional de Sanidad Agropecuaria e Inocuidad Alimentaria (SENASAG), organismo encargado de regular y supervisar la calidad e inocuidad de los productos alimenticios en Bolivia. Algunas leyes, un decreto y una resolución se darán a conocer a continuación:

- Ley N° 830 de Sanidad Agropecuaria e Inocuidad Alimentaria (06/09/2016): Establece el marco normativo nacional en materia de inocuidad, sanidad agropecuaria, y la creación de tasas por servicios prestados.
- Ley N° 2061 (16/03/2000): Crea el SENASAG, encargada de administrar el régimen de sanidad agropecuaria e inocuidad alimentaria en los tramos productivos y de procesamiento.
- Decreto Supremo N° 25729: Reglamenta la organización y funcionamiento del SENASAG, facultándolo para certificar la inocuidad alimentaria de productos de consumo nacional, importación y exportación.

- Resolución Administrativa SENASAG N° 0484/2023: Regula específicamente los procedimientos de inocuidad alimentaria en 2023

También la Industria de Alimentos Ltda.(INAL), debe implementar y cumplir con:

- Buenas Prácticas de Manufactura (BPM): Exigidas para obtener el registro sanitario.
- Sistema HACCP (Análisis de Peligros y Puntos Críticos de Control): Aunque a menudo voluntario, es altamente recomendado y exigido para certificar calidad en exportaciones o licitaciones públicas.
- Normas ISO 22000:2018: Sistema de gestión de inocuidad alimentaria.

CAPÍTULO 3:

DISEÑO TEÓRICO DE LA INVESTIGACIÓN

3.1. Problemática.

En el caso de Industria de Alimentos Ltda. (INAL), una empresa dedicada a la producción y comercialización de alimentos, el proceso de control de calidad presenta diversas limitaciones derivadas de su naturaleza manual. Entre estas se destacan la posibilidad de los errores humanos, fatiga visual acumulada durante la jornadas laborales, se tiene una baja precisión en la identificación de defectos y dificultades para mantener un monitoreo constante en tiempo real a lo largo de toda la línea de producción.

La problemática se identifica en escenarios de alta producción, donde la velocidad de la línea dificulta una inspección visual adecuada, generando que productos con defectos físicos, deformaciones o empaques irregulares continúen en el proceso productivo sin ser detectados. No tener herramientas tecnológicas automatizadas limita la capacidad de registrar, analizar y gestionar de manera centralizada la información relacionada con los defectos identificados, impidiendo un seguimiento sistemático.

En la cadena de suministros y producción cuenta con una deficiencia en el sistema integrado, lo que dificulta la centralización de la información en sus diversas líneas de lácteos y helados, esta situación genera ineficiencias en el control de calidad, incrementa el margen de error en la detección de productos no conformes, eleva los costos operativos por reprocesos y pérdidas de materia prima.

3.2. Formulación del problema.

¿Cómo reducir los errores en la detección de productos dañados dentro de la línea de producción en la Industria de Alimentos Ltda. (INAL).?

3.3. Delimitación del contenido.

Para el desarrollo del sistema, estos son los módulos que se llevarán a desarrollar para el sistema:

- Módulo de Gestión de Sesiones: Permitirá el registro, autenticación e inicio de sesión de los usuarios dentro del sistema, así como la administración de roles (administrador, supervisor, operador y analista), garantizando el acceso seguro y controlado a cada funcionalidad según permisos establecidos.
- Módulo de Detección Automática: Será el núcleo del sistema, encargado de analizar imágenes capturadas en tiempo real mediante cámaras en la línea de producción, utilizando redes neuronales convolucionales para identificar productos defectuosos como empaques dañados, deformaciones o sellados incorrectos.
- Módulo de Monitoreo en Tiempo Real: Permitirá visualizar el estado actual de la línea de producción a través de un panel interactivo, mostrando las cámaras en vivo, los productos inspeccionados y las detecciones realizadas por el sistema en el momento en que ocurren.
- Módulo de Alertas Automáticas: Se encargará de generar notificaciones inmediatas cuando se detecte un producto defectuoso, registrando el evento junto con evidencia fotográfica, fecha y hora, permitiendo una rápida intervención por parte del personal.
- Módulo de Inventario de Productos: Permitirá llevar un control organizado de los productos inspeccionados, clasificándolos en conformes y no conformes, facilitando su consulta, búsqueda y gestión dentro del sistema.
- Módulo de Trazabilidad: Permitirá realizar el seguimiento completo de los productos, registrando información desde su detección hasta su distribución, incluyendo el historial de defectos y el destino final de cada lote o producto.

- Módulo de Reportes: Permitirá generar reportes estadísticos e indicadores de desempeño del proceso de control de calidad, mostrando información como frecuencia de defectos, tipos más comunes y comparaciones por períodos de tiempo, con opción de exportación para análisis.

3.4. Delimitación temporal.

El desarrollo del sistema está programado para iniciar en Abril de 2026 y concluir a finales de Junio del mismo año, abarcando un período total de aproximadamente tres meses, en los cuales se contempla el análisis de requerimientos, diseño, desarrollo del sistema.

3.5. Delimitación espacial.

El desarrollo e implementación del sistema se llevará a cabo en dos lugares diferentes. La fase de desarrollo, programación, entrenamiento del modelo y pruebas se realizará en las instalaciones de la universidad, donde se contará con los recursos tecnológicos necesarios para el trabajo de investigación y desarrollo. La fase de implementación y pruebas en entorno real se ejecutará en las instalaciones de Industria de Alimentos Ltda. (INAL).

3.6. Objetivos.

3.6.1. Objetivo general.

Desarrollar un sistema que controle la calidad mediante redes neuronales convolucionales para la identificación automática de productos dañados con el proceso de inspección e integrar el monitoreo en tiempo real en la línea de producción en la Industria de Alimentos Ltda. (INAL), para la gestión 1/2026.

3.6.2. Objetivos específicos

- Desarrollar una red neuronal convolucional para la detección automática de productos dañados, dirigida a la Industria de Alimentos Ltda.(INAL), mediante entrenamiento con imágenes de sus productos.
- Controlar el inventario de productos conformes y no conformes, para llevar un control organizado del estado de los productos dirigido al supervisor de producción, mediante el registro de los resultados dados por la red neuronal.
- Integrar los reportes e indicadores de desempeño del sistema, para proporcionar información confiable a la toma de decisiones gerenciales, dirigido a la gerencia de INAL Ltda, mediante el uso de dashboards y paneles de visualización generados por el sistema.
- Inspeccionar el monitoreo en tiempo real que permita visualizar el estado de la línea de producción y las detecciones realizadas por el modelo, para el operador de producción, mediante el desarrollo de una interfaz gráfica.
- Identificar las alertas automáticas con registro e historial, para que este pueda notificar de manera inmediata ante eventos críticos del sistema, para el operador de producción, mediante un módulo de notificaciones integrados al sistema.

CAPÍTULO 5:

DESCRIPCIÓN DEL PROYECTO

5.1. Introducción a la Metodología de Análisis SCRUM

La metodología SCRUM fue elegida más que todo por su flexibilidad y que permite trabajar por entregas funcionales de manera progresiva. Esto resulta adecuado para este tipo de proyecto, ya que al ser desarrollado por una sola persona, es necesario asumir distintos roles y organizar el trabajo de forma ordenada. SCRUM facilita la planificación de las tareas, la implementación de cada módulo y la mejora continua del sistema. En este caso, se aplicó principalmente en la parte administrativa, permitiendo desarrollar y organizar las diferentes funcionalidades del sistema de manera más eficiente.

5.2. Fase 1 – Pre Game

En esta fase se aplican los principios de la metodología SCRUM para realizar el diseño de la parte administrativa del sistema. En esta etapa se define cómo estará estructurado el sistema, organizando las funcionalidades que se van a desarrollar.

5.2.1. Análisis de Requerimientos

Los requerimientos incluyen la creación de perfiles de usuario, un sistema de búsqueda avanzada, la integración de la base de datos y la implementación de funciones de seguridad, como la autenticación de usuarios.

5.2.1.1. Requerimientos Funcionales

ID	Requisito Funcional	Descripción
RF-1	Registro de Usuarios	El sistema permitirá registrar diferentes tipos de usuarios, como ser administrador, operativo, supervisor, analista y distribuidor. Cada uno tendrá accesos distintos según su rol dentro del sistema. En el caso del administrador, contará con permisos completos para gestionar y administrar todas las funciones del sistema.
RF-2	Inicio de Sesión	El sistema debe requerir inicio de sesión con credenciales para acceder a cualquier módulo, garantizando que solo el personal autorizado pueda utilizar la plataforma.
RF-3	Gestión de usuarios y roles	El sistema debe permitir la creación y administración de usuarios con roles diferenciados al momento de registrarse como ser administrador, supervisor y operador y Analista cada uno con accesos y permisos específicos sobre los módulos del sistema.
RF-4	Detección automática de productos dañados	El sistema debe analizar imágenes capturadas en tiempo real desde las cámaras instaladas en la línea de producción e identificar automáticamente productos con defectos visuales como empaques rotos, deformaciones y sellados incorrectos.
RF-5	Monitoreo en tiempo real	El sistema debe mostrar en el panel web el estado actual de la línea de producción de manera continua, permitiendo al personal visualizar las detecciones en el momento en que ocurren.
RF-6	Gestión de alertas automáticas	El sistema debe notificar de manera inmediata al personal supervisor cuando se detecte un producto defectuoso, registrando el evento con evidencia fotográfica y la hora exacta de detección.
RF-7	Módulo de inventario de productos	El sistema debe llevar un registro organizado y diferenciado de los productos conformes y no conformes detectados durante el proceso productivo, permitiendo consultar el estado del inventario en cualquier momento.
RF-8	Registro de evidencia fotográfica e historial	El sistema debe almacenar automáticamente las imágenes de cada producto defectuoso detectado, junto con el tipo de defecto identificado, manteniendo un historial completo de todas las detecciones realizadas.

RF-9	Trazabilidad de distribución	El sistema debe permitir registrar y visualizar a qué destino o sucursal fueron enviados los productos aprobados, proporcionando seguimiento del recorrido del producto desde la línea de producción hasta su distribución.
RF-10	Módulo de reportes estadísticos	El sistema debe generar reportes e indicadores de desempeño del proceso de control de calidad, mostrando gráficas de frecuencia de defectos, tipos de defectos más comunes y comparativas por período de tiempo.

Tabla 5. Requisitos Funcionales

Fuente: *Elaboración propia*

5.2.2. Historias de Usuario

ID	PRIOR.	RIESG.	USUARIO	TÍTULO	DESCRIPCIÓN	ESTIM.
1	Muy Alta	Alto	Administrador	Inicio de sesión	Como administrador quiero iniciar sesión con mis datos para acceder de forma segura al sistema.	2
2	Muy Alta	Alto	Administrador	Gestión de usuarios y roles	Como administrador quiero crear usuarios con roles diferenciados, para que cada persona tenga permisos acordes a su función.	2
3	Muy Alta	Alto	Administrador	Control de acceso por rol	Como administrador quiero restringir el acceso a módulos según el rol asignado, para que cada usuario solo opere lo que le corresponde.	1
4	Muy Alta	Alto	Operador	Detección automática de productos dañados	Como operador quiero que el sistema analice imágenes en tiempo real, para identificar automáticamente productos con defectos visuales.	3
5	Muy Alta	Alto	Operador	Configuración del modelo de detección	Como operador quiero configurar los parámetros del modelo de detección, para adaptar el sistema a cada línea de producción.	2
6	Muy Alta	Alto	Operador	Captura y procesamiento de imágenes	Como operador quiero que el sistema capture imágenes desde las cámaras mediante OpenCV para alimentar el modelo de detección de forma continua.	3
7	Muy Alta	Alto	Operador	Panel de monitoreo en tiempo real	Como operador quiero visualizar el estado actual de la línea de producción en el panel web para conocer las	3

					detecciones en el momento en que ocurren.	
8	Alta	Alto	Supervisor	Visualización de cámaras activas	Como supervisor quiero ver el feed en vivo de cada cámara instalada para conocer el estado de cada una dentro de la línea de producción.	2
9	Alta	Medio	Operador	Indicadores de producción en el panel	Como operador quiero visualizar los indicadores de producción en el panel, para conocer el avance de la inspección durante la jornada actual.	2
10	Muy Alta	Alto	Supervisor	Alerta automática ante defecto detectado	Como supervisor quiero recibir una notificación inmediata al detectar un producto defectuoso para intervenir con rapidez ante cualquier falla en la línea.	2
11	Alta	Medio	Supervisor	Historial de alertas generadas	Como supervisor quiero consultar el registro completo de alertas generadas, para revisar los eventos ocurridos durante el proceso productivo.	2
12	Alta	Alto	Analista	Registro de productos conformes y no conformes	Como analista quiero acceder al registro de productos conformes y no conformes para consultar el estado del inventario en cualquier momento.	2
13	Alta	Medio	Analista	Búsqueda y filtros en inventario	Como analista quiero filtrar el inventario por tipo de defecto para localizar rápidamente los productos que requieren atención.	2
14	Media	Medio	Analista	Registro de evidencia fotográfica e historial	Como analista quiero que el sistema almacene automáticamente la imagen de cada producto defectuoso para mantener un historial visual de las detecciones realizadas.	2
15	Alta	Medio	Supervisor	Seguimiento de distribución de productos	Como supervisor quiero registrar el destino de los productos aprobados, para hacer seguimiento del recorrido desde la línea de producción hasta su distribución.	3
16	Media	Medio	Analista	Consulta de trazabilidad por lote	Como analista quiero consultar el historial completo de un lote de producción, para verificar el destino de distribución que fue asignado.	2
17	Alta	Medio	Administrador	Generación de reportes estadísticos	Como administrador quiero generar reportes estadísticos del proceso de control de calidad para apoyar la toma de decisiones gerenciales.	3
18	Alta	Medio	Administrador	Exportación de reportes	Como administrador quiero exportar los reportes generados en formato PDF/Excel para presentarlos en auditorías internas de INAL Ltda.	2

19	Alta	Alta	Supervisor	Registro de entrada de productos para su fabricación	Como supervisor, quiero registrar el ingreso de un producto para su uso en la fabricación para dejar constancia del inicio del proceso de inspección.	2
20	Alta	Medio	Distribuidor	Confirmación de salida de productos aprobados	Como distribuidor quiero confirmar la recepción de los productos aprobados asignados a mi ruta para registrar formalmente la salida hacia su destino final.	2
21	Alta	Medio	Distribuidor	Confirmar la entrega de los productos terminados	Como distribuidor quiero confirmar los productos entregados a sus destinos que me designen para confirmar que se llegaron a sus destinos.	2

Tabla 6. Historias de Usuario

Fuente: *Elaboración propia*

5.2.3. Producto Backlog

MÓDULO	PRIOR.	ID BACK LOG	ELEMENTO DEL BACKLOG	DESCRIPCIÓN	RESPONSABLE	ESTIM.
SESIONES	Muy Alta	1	Inicio de sesión	Como administrador quiero iniciar sesión con mis credenciales para acceder de forma segura al sistema.	José Ángel Mamani Escarzo	2
	Muy Alta	2	Gestión de usuarios y roles	Como administrador quiero crear usuarios con roles diferenciados, para que cada persona tenga permisos acordes a su función.	José Ángel Mamani Escarzo	2
	Muy Alta	3	Control de acceso por rol	Como administrador quiero restringir el acceso a módulos según el rol asignado, para que cada usuario solo opere lo que le corresponde.	José Ángel Mamani Escarzo	1
DETECCIÓN AUTOMÁTICA	Muy Alta	4	Detección automática de productos dañados	Como operador quiero que el sistema analice imágenes en tiempo real, para identificar automáticamente productos con defectos visuales.	José Ángel Mamani Escarzo	3

	Muy Alta	5	Configuración del modelo de detección	Como operador quiero configurar los parámetros del modelo de detección, para adaptar el sistema a cada línea de producción.	José Ángel Mamani Escarzo	2
	Muy Alta	6	Captura y procesamiento de imágenes	Como operador quiero que el sistema capture imágenes desde las cámaras mediante OpenCV para alimentar el modelo de detección de forma continua.	José Ángel Mamani Escarzo	3
MONITOREO EN TIEMPO REAL	Muy Alta	7	Panel de monitoreo en tiempo real	Como operador quiero visualizar el estado actual de la línea de producción en el panel web para conocer las detecciones en el momento en que ocurren.	José Ángel Mamani Escarzo	3
	Alta	8	Visualización de cámaras activas	Como supervisor quiero ver el feed en vivo de cada cámara instalada para conocer el estado de cada una dentro de la línea de producción.	José Ángel Mamani Escarzo	2
	Alta	9	Indicadores de producción en el panel	Como operador quiero visualizar los indicadores de producción en el panel, para conocer el avance de la inspección durante la jornada actual.	José Ángel Mamani Escarzo	2
ALERTAS AUTOMÁTICAS	Muy Alta	10	Alerta automática ante defecto detectado	Como supervisor quiero recibir una notificación inmediata al detectar un producto defectuoso para intervenir con rapidez ante cualquier falla en la línea.	José Ángel Mamani Escarzo	2
	Alta	11	Historial de alertas generadas	Como supervisor quiero consultar el registro completo de alertas generadas, para revisar los eventos ocurridos durante el proceso productivo.	José Ángel Mamani Escarzo	2
INVENTARIO DE PRODUCTOS	Alta	12	Registro de productos conformes y no conformes	Como analista quiero acceder al registro de productos conformes y no conformes para consultar el estado del inventario en cualquier momento.	José Ángel Mamani Escarzo	2

	Alta	13	Búsqueda y filtros en inventario	Como analista quiero filtrar el inventario por tipo de defecto para localizar rápidamente los productos que requieren atención.	José Ángel Mamani Escarzo	2
	Alta	14	Registro de evidencia fotográfica e historial	Como analista quiero que el sistema almacene automáticamente la imagen de cada producto defectuoso para mantener un historial visual de las detecciones realizadas.	José Ángel Mamani Escarzo	2
TRAZABILIDAD	Alta	15	Seguimiento de distribución de productos	Como supervisor quiero registrar el destino de los productos aprobados, para hacer seguimiento del recorrido desde la línea de producción hasta su distribución.	José Ángel Mamani Escarzo	3
	Media	16	Consulta de trazabilidad por lote	Como analista quiero consultar el historial completo de un lote de producción, para verificar el destino de distribución que fue asignado.	José Ángel Mamani Escarzo	2
	Alta	19	Registro de entrada de productos para su fabricación	Como supervisor, quiero registrar el ingreso de un producto para su uso en la fabricación para dejar constancia del inicio del proceso de inspección.	José Ángel Mamani Escarzo	2
	Alta	20	Confirmación de salida de productos aprobados	Como distribuidor quiero confirmar la recepción de los productos aprobados asignados a mi ruta para registrar formalmente la salida hacia su destino final.	José Ángel Mamani Escarzo	2
	Alta	21	Confirmar la entrega de los productos terminados	Como distribuidor quiero confirmar los productos entregados a sus destinos que me designen.	José Ángel Mamani Escarzo	2
REPORTES	Alta	17	Generación de reportes estadísticos	Como administrador quiero generar reportes estadísticos del proceso de control de calidad para apoyar la toma de decisiones gerenciales.	José Ángel Mamani Escarzo	3

	Alta	18	Exportación de reportes	Como administrador quiero exportar los reportes generados en formato PDF/Excel para presentarlos en auditorías internas de INAL Ltda.	José Ángel Mamani Escarzo	2

Tabla 7. Producto Backlog

Fuente: *Elaboración propia*

5.2.4. Sprint Backlog

NºSPRINT	MÓDULO	TAREAS DEL SPRINT	ESFUERZO EN HORAS	
			ESTIMA.	REAL
1	SESIONES	Análisis de requisitos	10	8
		Diseño del módulo	30	27
		Desarrollo del módulo	34	30
		Pruebas del módulo	26	20
	TIEMPO TOTAL DEL SPRINT		**100**	**85**
2	DETECCIÓN AUTOMÁTICA	Análisis de requisitos	10	
		Diseño del módulo	28	
		Desarrollo del módulo	34	
		Pruebas del módulo	26	
	TIEMPO TOTAL DEL SPRINT		**98**	
3	MONITOREO EN TIEMPO REAL	Análisis de requisitos	14	11
		Diseño del módulo	36	33
		Desarrollo del módulo	44	
		Pruebas del módulo	32	
	TIEMPO TOTAL DEL SPRINT		**126**	**44**
4	ALERTAS AUTOMÁTICAS	Análisis de requisitos	10	8
		Diseño del módulo	26	24
		Desarrollo del módulo	35	30
		Pruebas del módulo	24	20

	TIEMPO TOTAL DEL SPRINT		**95**	**82**
5	**INVENTARIO DE PRODUCTOS**	Análisis de requisitos	12	10
		Diseño del módulo	30	25
		Desarrollo del módulo	38	33
		Pruebas del módulo	28	22
	TIEMPO TOTAL DEL SPRINT		**108**	**90**
6	**TRAZABILIDAD**	Análisis de requisitos	10	7
		Diseño del módulo	28	25
		Desarrollo del módulo	35	30
		Pruebas del módulo	24	20
	TIEMPO TOTAL DEL SPRINT		**97**	**82**
7	**REPORTES**	Análisis de requisitos	10	7
		Diseño del módulo	25	21
		Desarrollo del módulo	30	26
		Pruebas del módulo	23	20
	TIEMPO TOTAL DEL SPRINT		**88**	**74**
TOTAL			**712**	

Tabla 8. Sprint Backlog

Fuente: *Elaboración propia*

5.3. Fase 2 – Game

5.3.1. Análisis y Diseño

La fase de análisis y diseño es el punto inicial para el desarrollo del sistema, ya que permitirá comprender mejor las necesidades del proyecto y definir cómo estructurarlo. En esta etapa se identificaron los requerimientos principales del sistema, tomando en cuenta los diferentes tipos de usuarios y las funciones que cada uno debía cumplir dentro de la plataforma. También, se establecen los módulos principales que formarían parte del sistema, organizando de manera clara cada componente para que responda a los objetivos planteados.

5.3.1.1. Diagrama de Paquetes

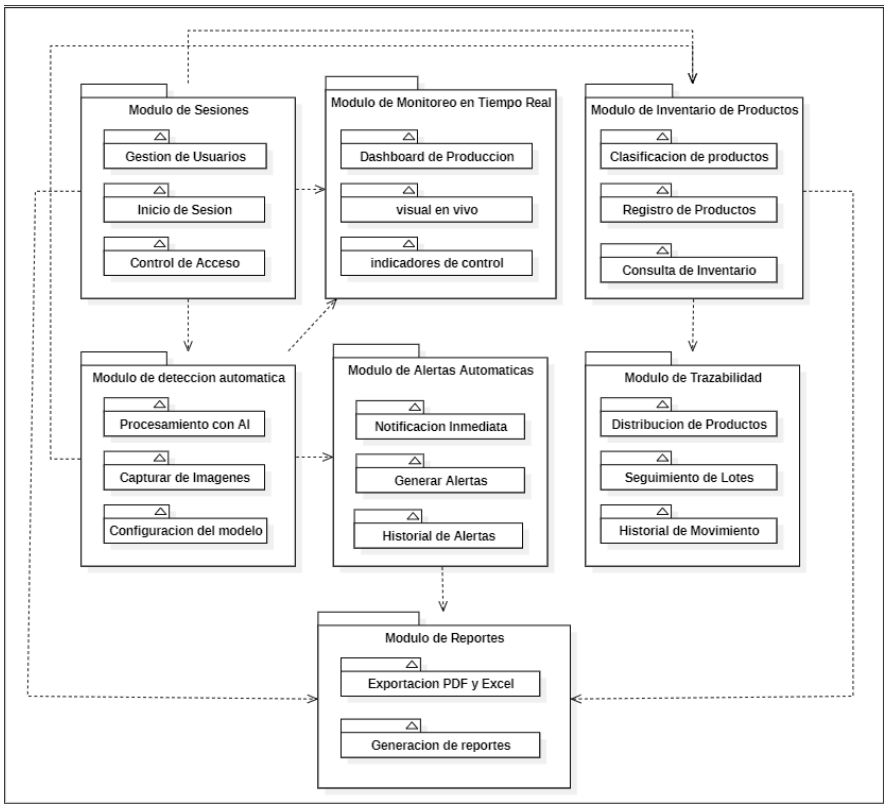


Figura 13. Diagrama de Paquetes

Fuente: *Elaboración propia*

5.3.1.2. Diagrama de Casos de Uso

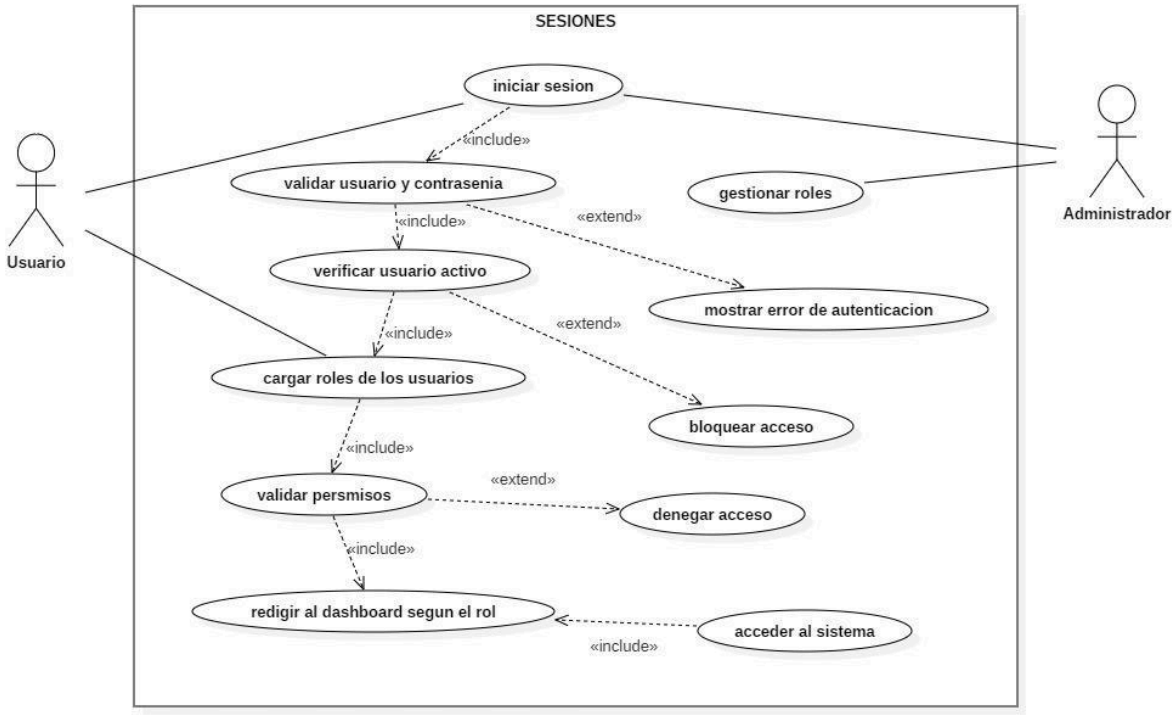


Figura 14. Diagrama de Casos de Uso - Sesiones

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Sesiones
Actor(es)	Administrador, Usuario
Tipo	Primario
Propósito	Permitir el acceso seguro al sistema mediante autenticación y control de roles.
Resumen	El usuario ingresa sus credenciales, el sistema valida la información y permite el acceso según el rol asignado.

Tabla 9. Tabla de Casos de Uso - Sesiones

Fuente: *Elaboración propia*

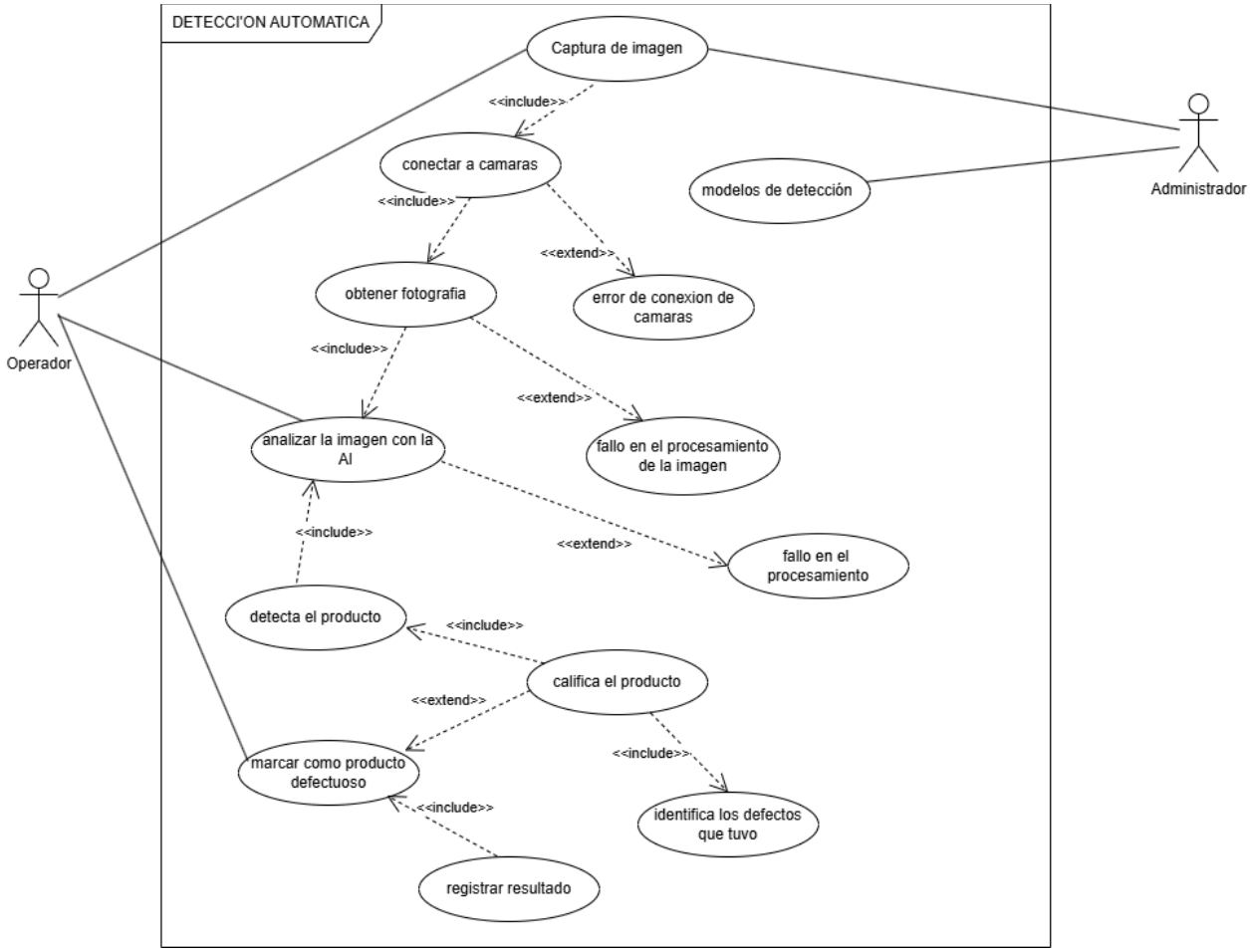


Figura 15. Diagrama de Casos de Uso - Detección Automática

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Detección Automática
Actor(es)	Administrador, Operador
Tipo	Primario
Propósito	Identificar productos defectuosos en tiempo real mediante análisis de imágenes.
Resumen	El sistema captura imágenes desde cámaras, procesa los datos y detecta defectos automáticamente utilizando redes neuronales convolucionales.

Tabla 10. Tabla de Casos de Uso - Detección Automática

Fuente: *Elaboración propia*

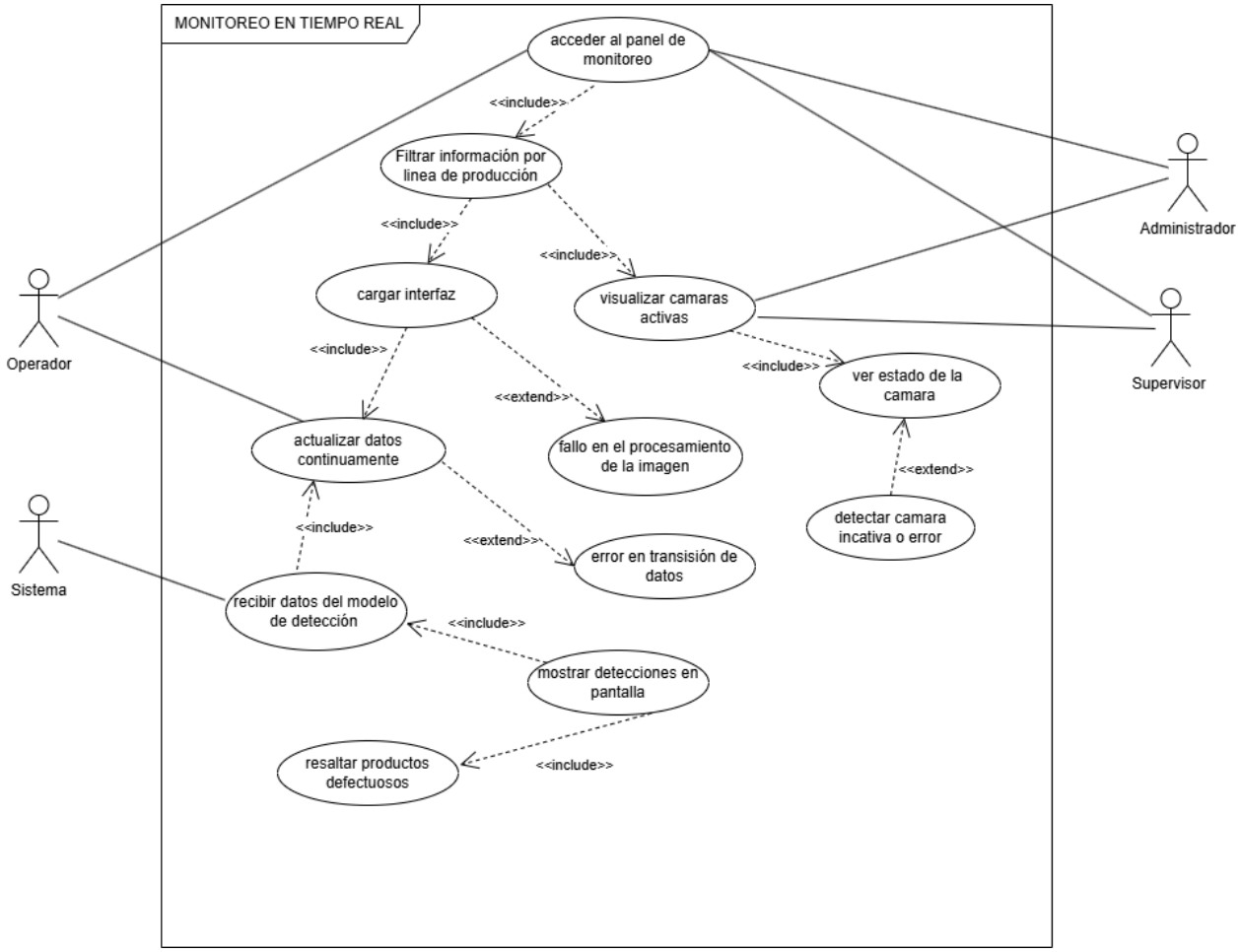


Figura 16. Diagrama de Casos de Uso - Monitoreo en tiempo real

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Monitoreo en tiempo real
Actor(es)	Administrador, Operador, Supervisor
Tipo	Primario
Propósito	Visualizar el estado de la producción y detecciones en tiempo real.
Resumen	El usuario accede a un panel donde observa cámaras en vivo, indicadores y resultados del sistema de detección.

Tabla 11. Tabla de Casos de Uso - Monitoreo en tiempo real

Fuente: *Elaboración propia*

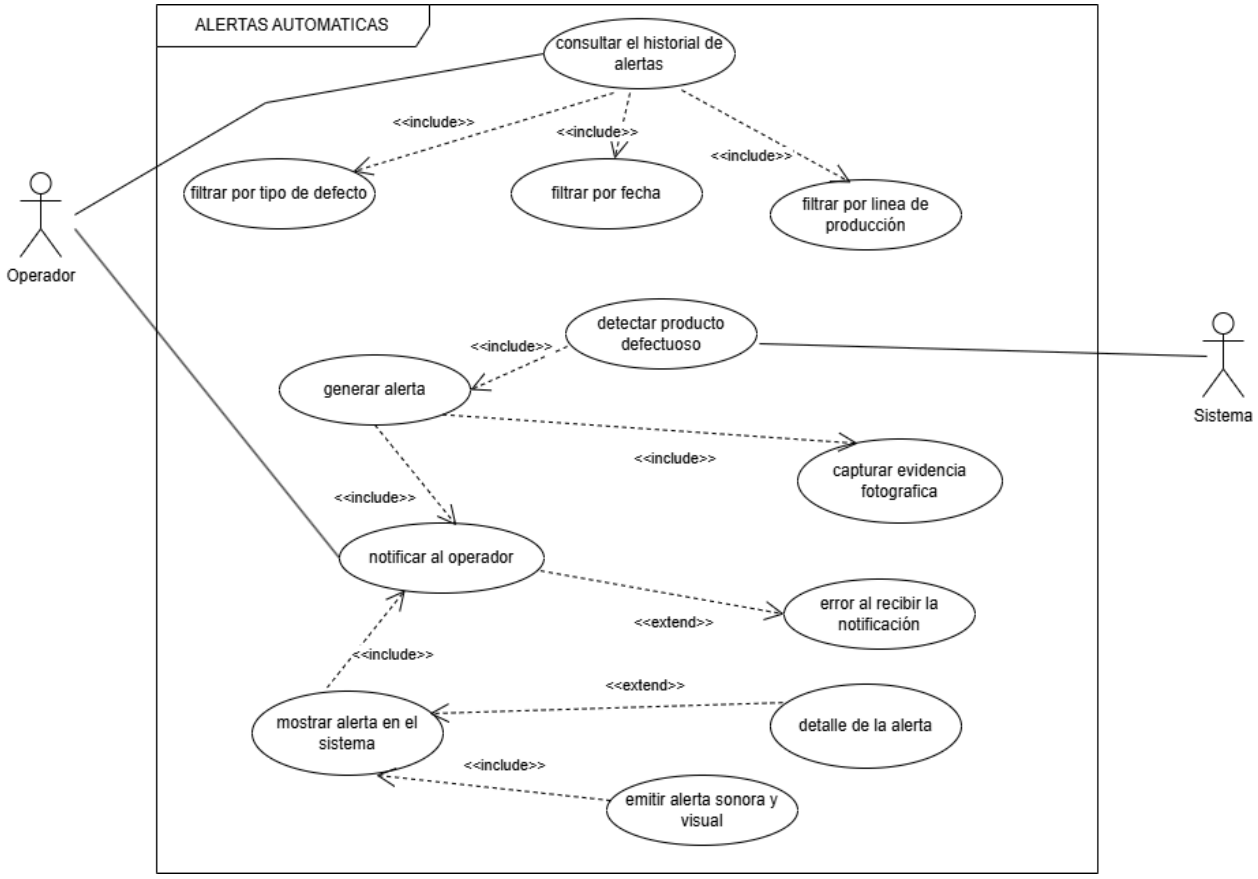


Figura 17. Diagrama de Casos de Uso - Alertas automáticas

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Alertas automáticas
Actor(es)	Operador
Tipo	Primario
Propósito	Notificar inmediatamente la detección de productos defectuosos.
Resumen	Cuando se detecta un defecto, el sistema genera una alerta con evidencia y la envía al operador para su revisión.

Tabla 12. Tabla de Casos de Uso - Monitoreo en tiempo real

Fuente: *Elaboración propia*

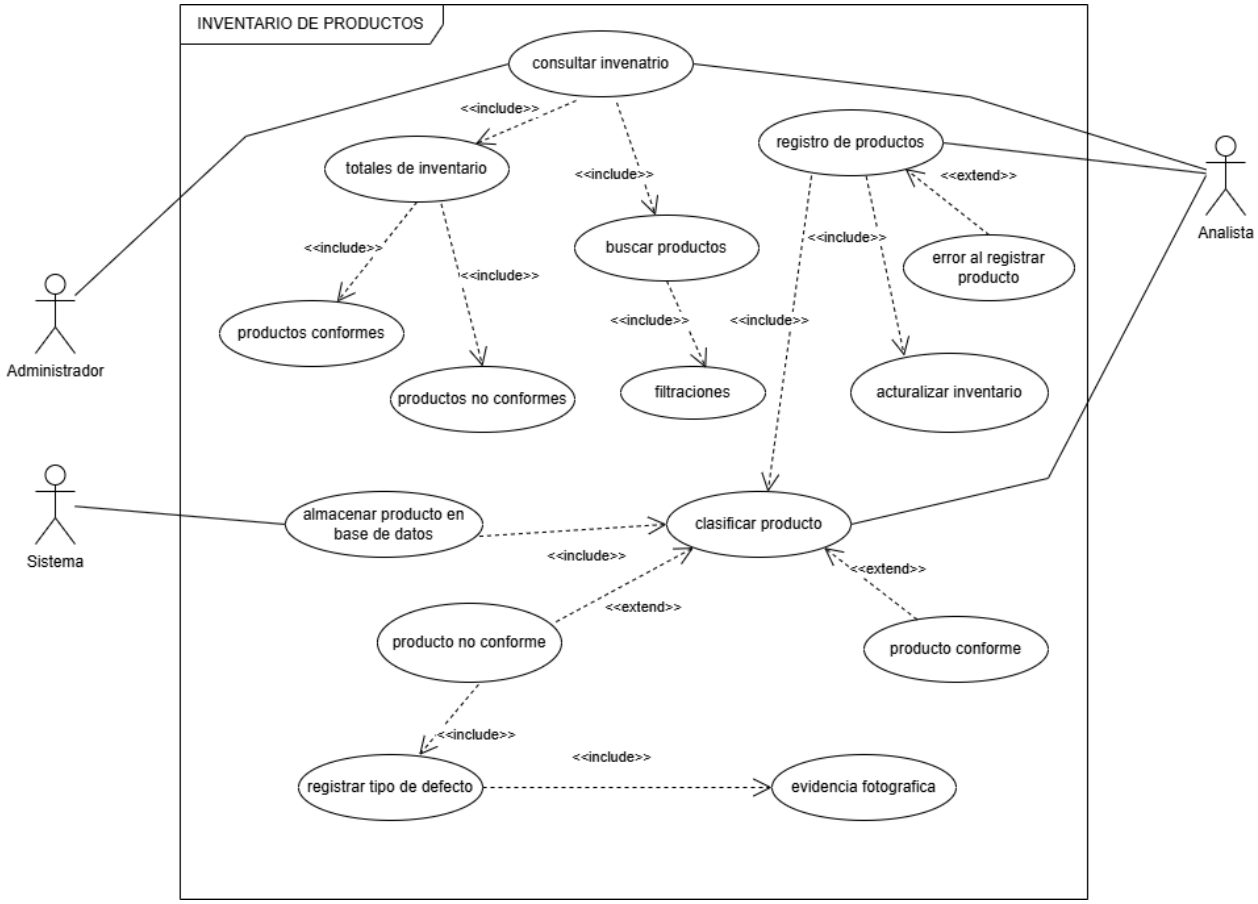


Figura 18. Diagrama de Casos de Uso - Inventario de Productos

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Inventario de Productos
Actor(es)	Administrador, Analista
Tipo	Primario
Propósito	Registrar, clasificar y consultar productos inspeccionados.
Resumen	El sistema almacena productos conformes y no conformes, permitiendo su consulta mediante filtros y búsqueda.

Tabla 13. Tabla de Casos de Uso - Inventario de Productos

Fuente: *Elaboración propia*

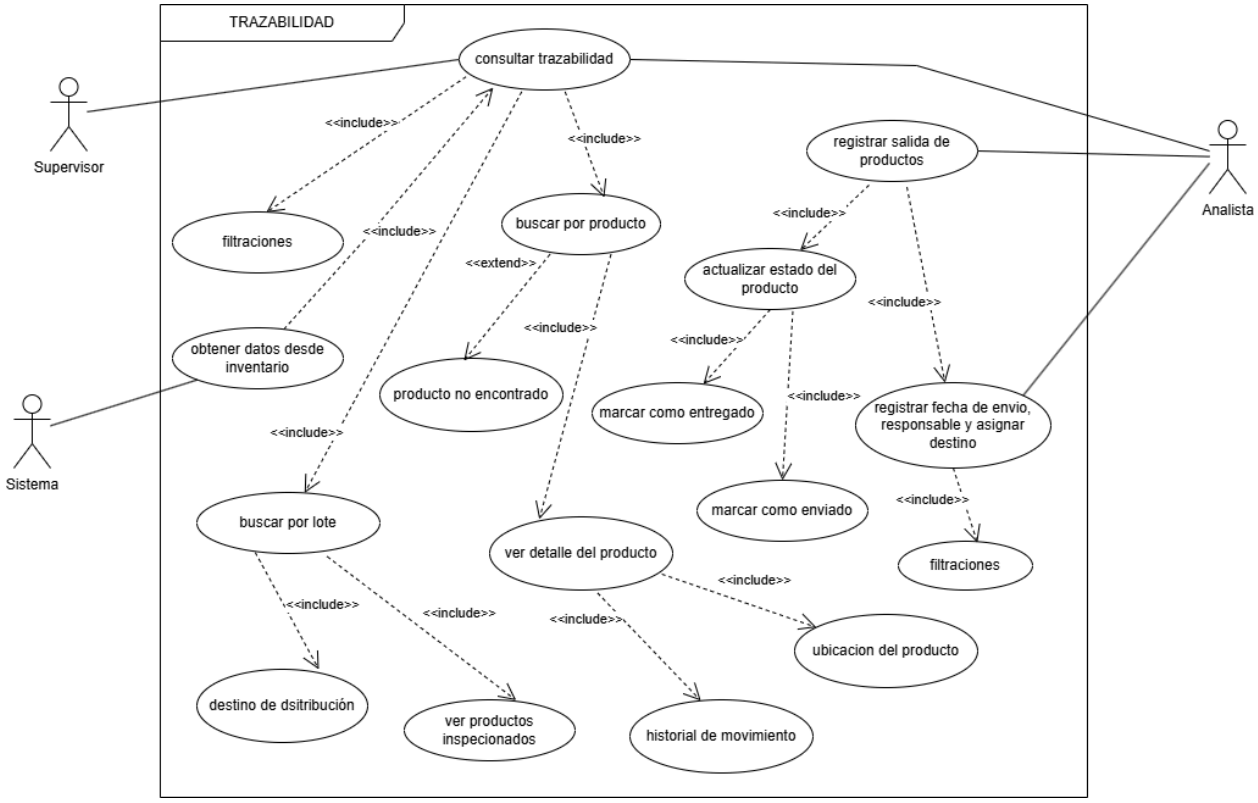


Figura 19. Diagrama de Casos de Uso - Trazabilidad

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Trazabilidad
Actor(es)	Analista, Supervisor
Tipo	Primario
Propósito	Permitir el seguimiento completo de los productos desde producción hasta distribución.
Resumen	El usuario puede consultar el historial de un producto o lote, incluyendo defectos y destino final.

Tabla 14. Tabla de Casos de Uso - Trazabilidad

Fuente: *Elaboración propia*

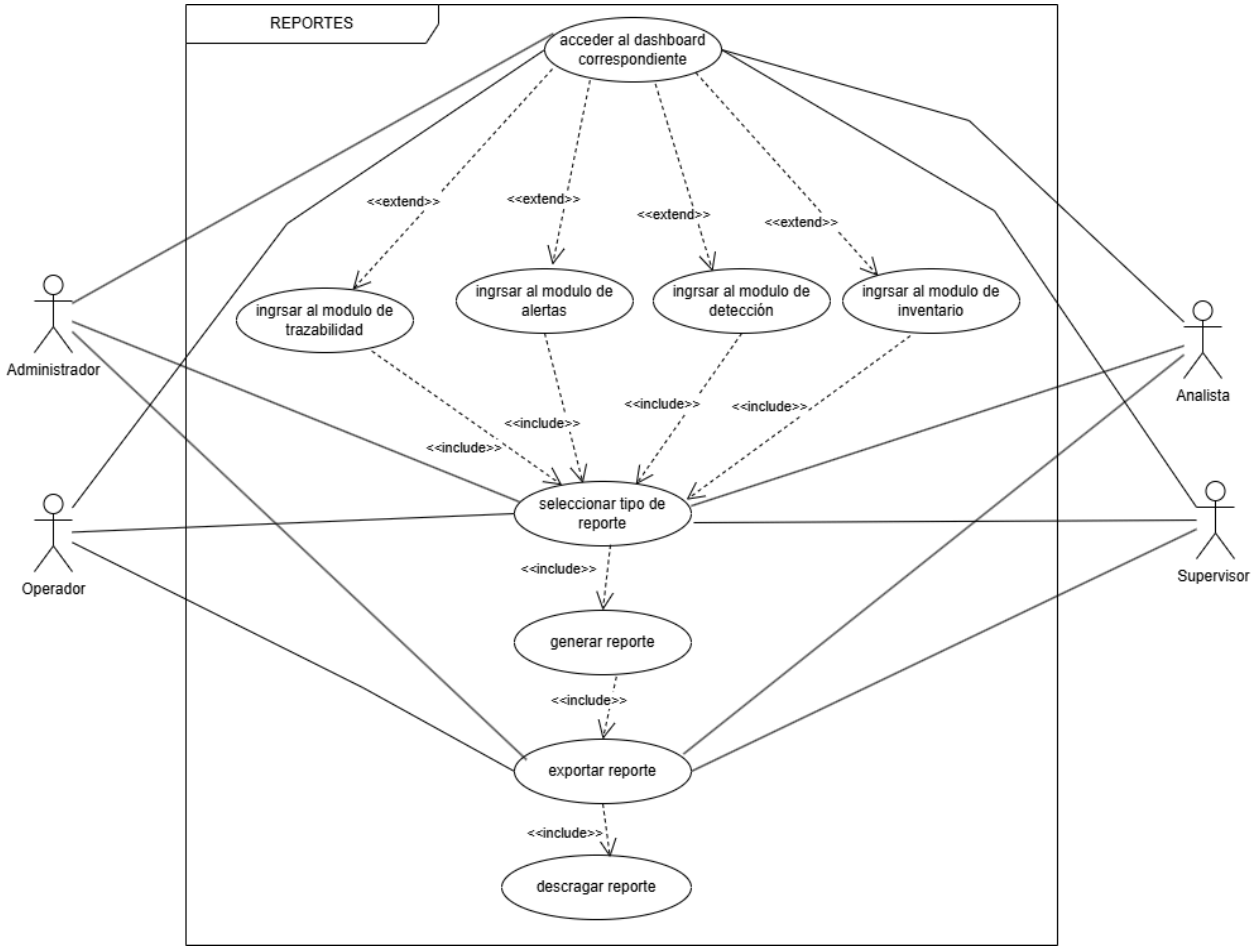


Figura 20. Diagrama de Casos de Uso - Reportes

Fuente: *Elaboración propia*

Elemento	Detalle
Caso de uso	Reportes
Actor(es)	Administrador, Analista, Supervisor, Operador
Tipo	Primario
Propósito	Generar reportes estadísticos y exportarlos para análisis y toma de decisiones.
Resumen	El usuario selecciona parámetros, el sistema procesa datos y genera reportes en formatos como PDF o Excel.

Tabla 15. Tabla de Casos de Uso - Reportes

Fuente: *Elaboración propia*

5.3.1.3. Diagrama de Secuencia

- **Sesiones**

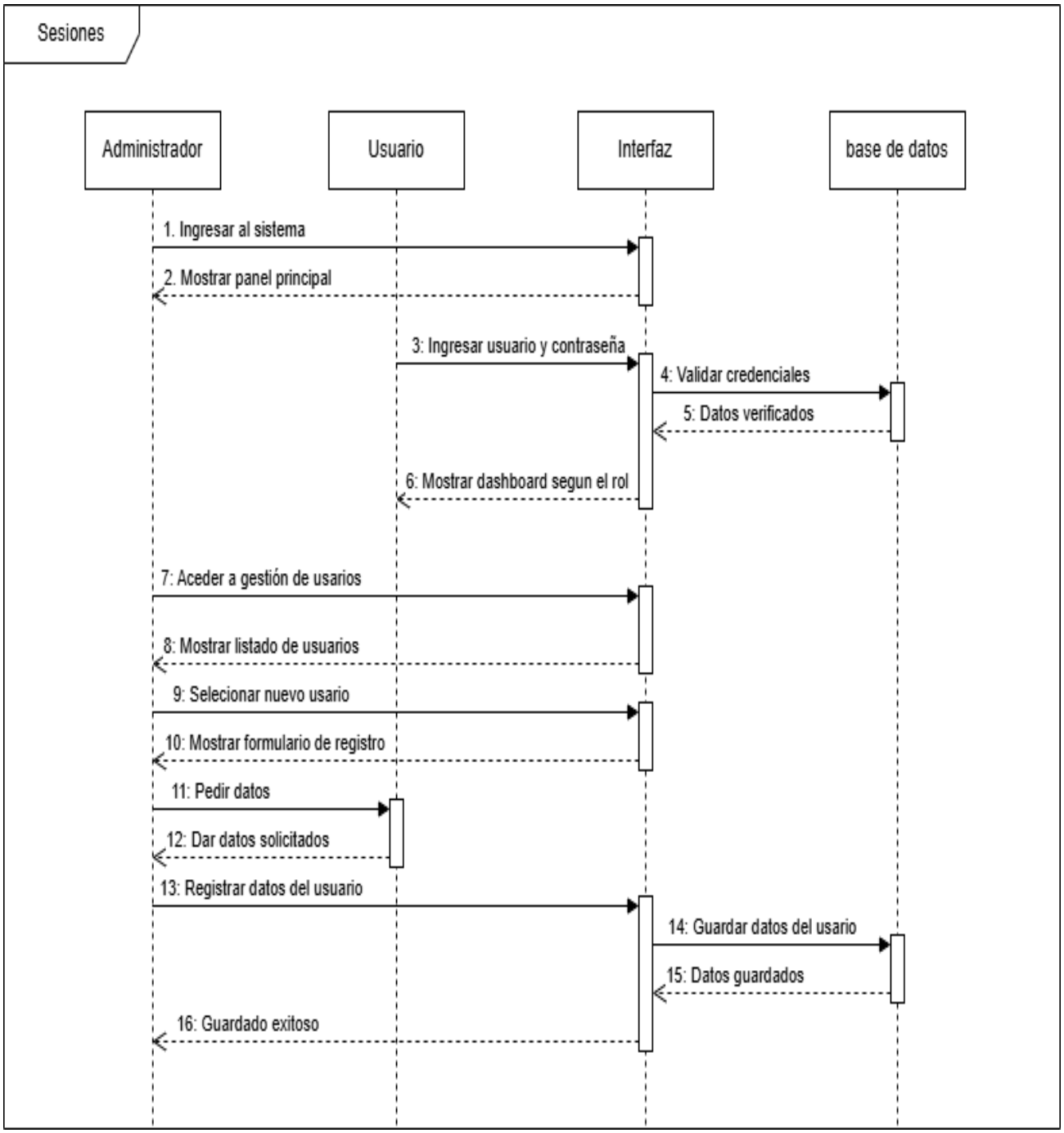


Figura 21. Diagrama de Secuencia - Sesiones

Fuente: *Elaboración propia*

- **Detección Automática**

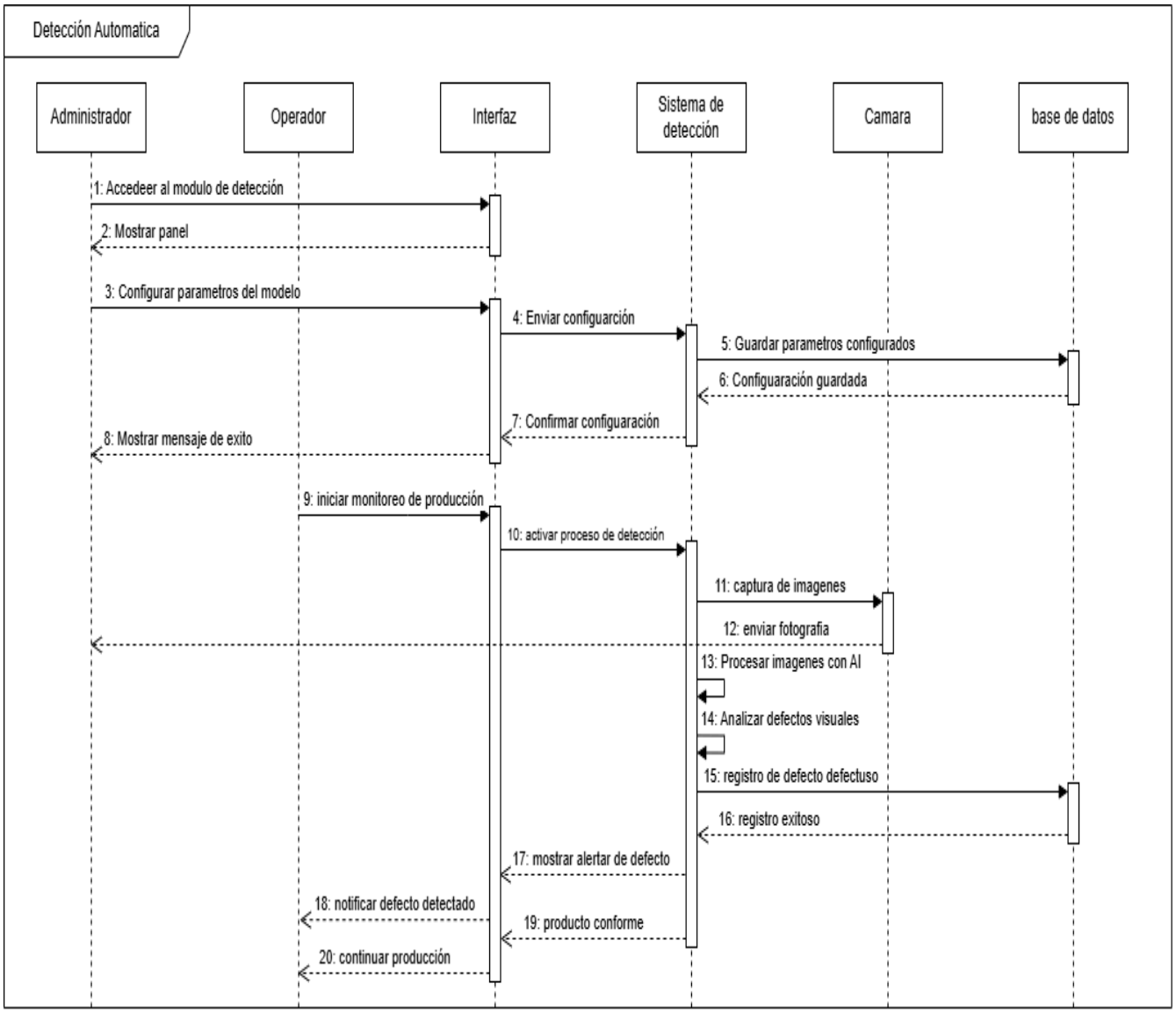


Figura 22. Diagrama de Secuencia - Detección Automática

Fuente: *Elaboración propia*

- **Monitoreo en Tiempo Real**

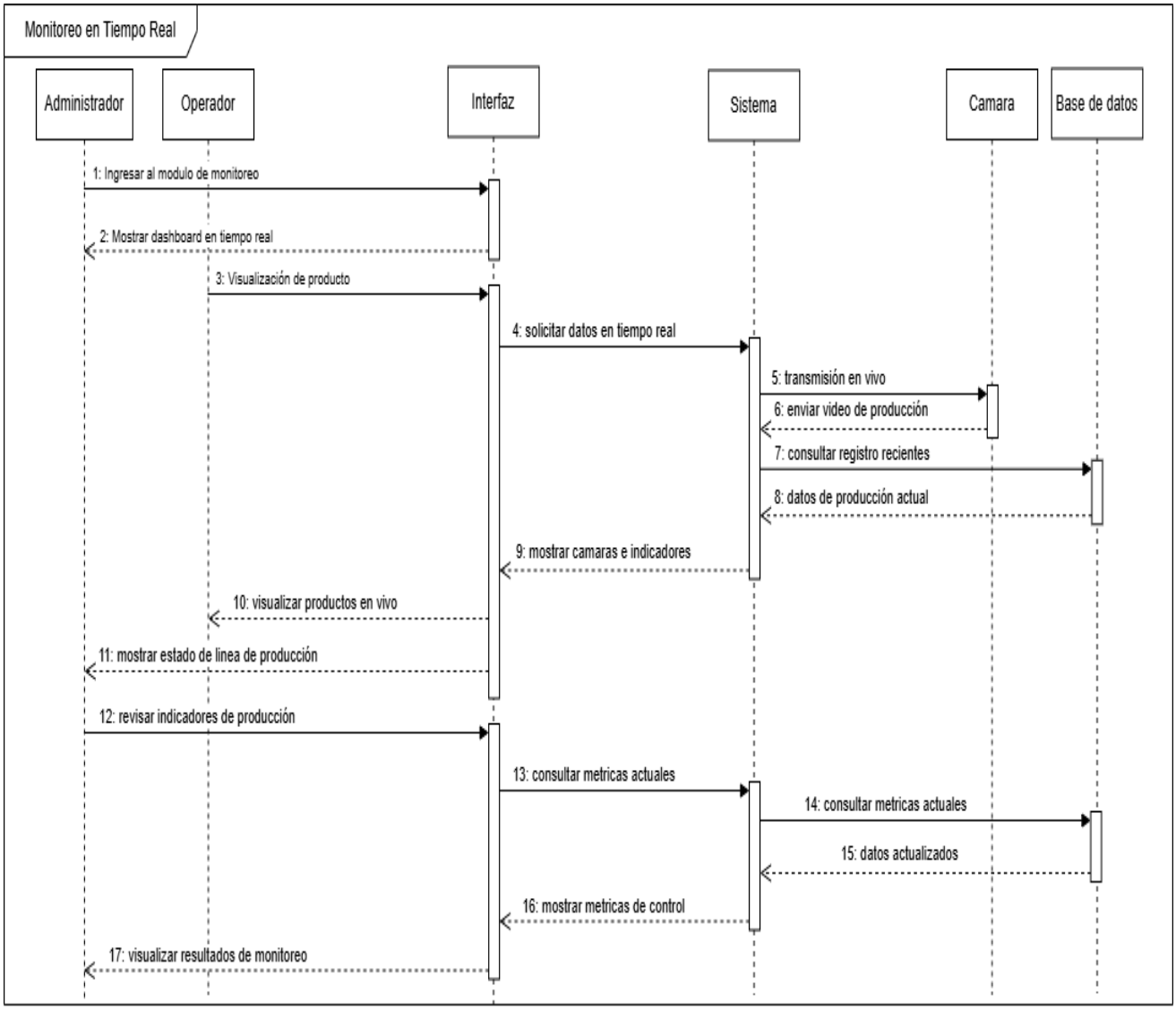


Figura 23. Diagrama de Secuencia - Monitoreo en tiempo real

Fuente: *Elaboración propia*

- **Alertas Automáticas**

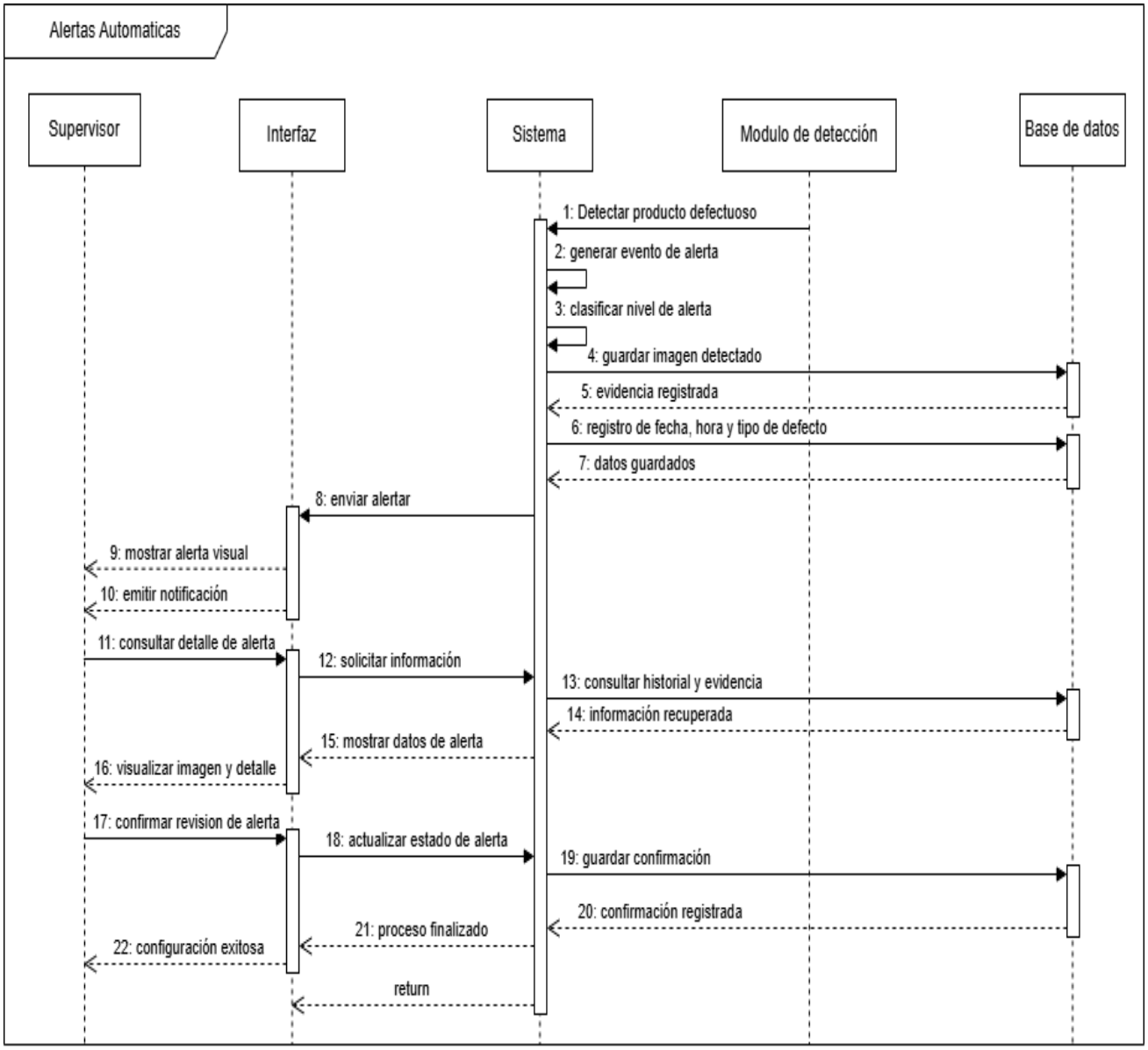


Figura 24. Diagrama de Secuencia - Alertas Automáticas

Fuente: *Elaboración propia*

- **Inventario de Productos**

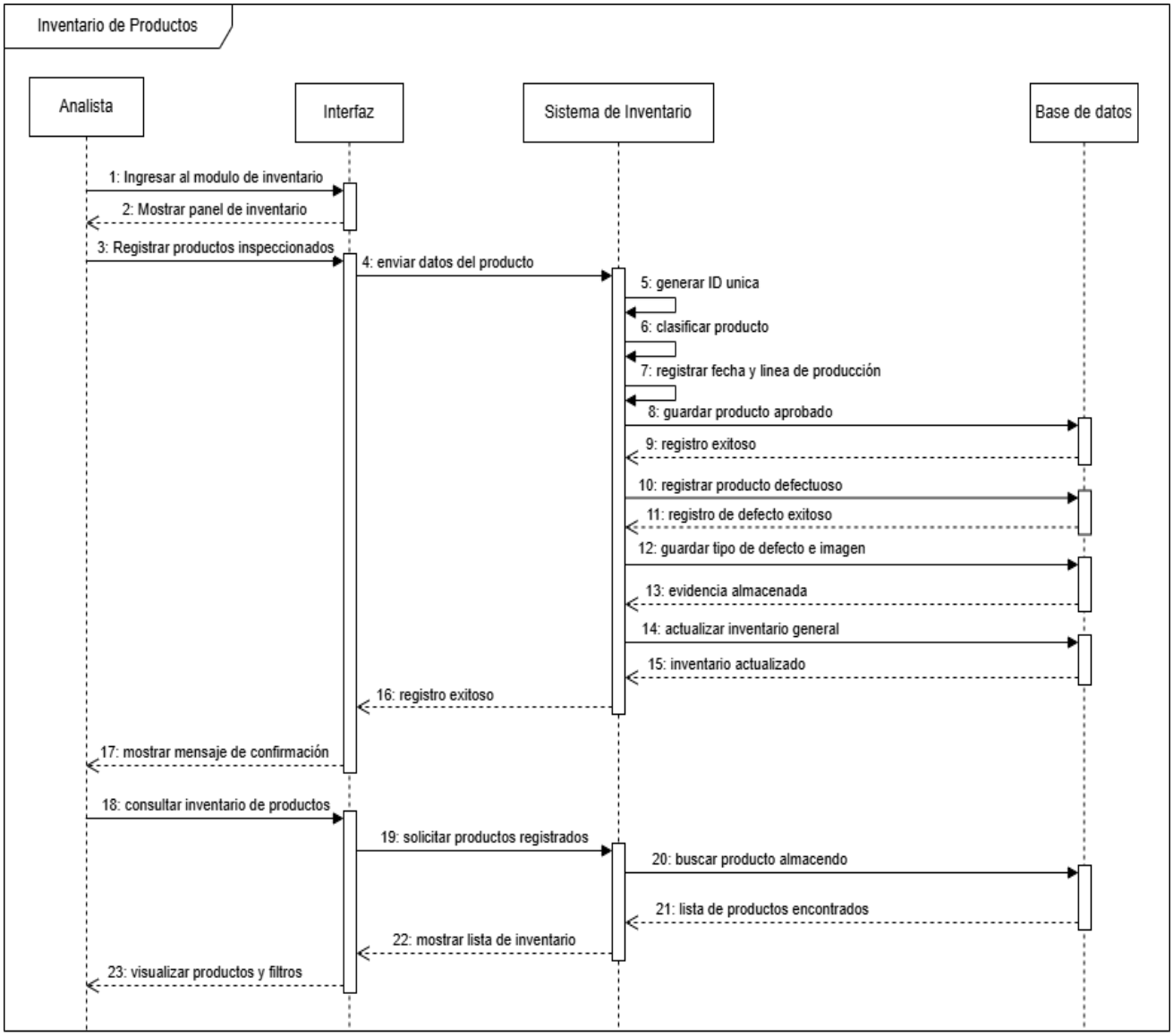


Figura 25. Diagrama de Secuencia - Inventario de Productos

Fuente: *Elaboración propia*

- **Trazabilidad**

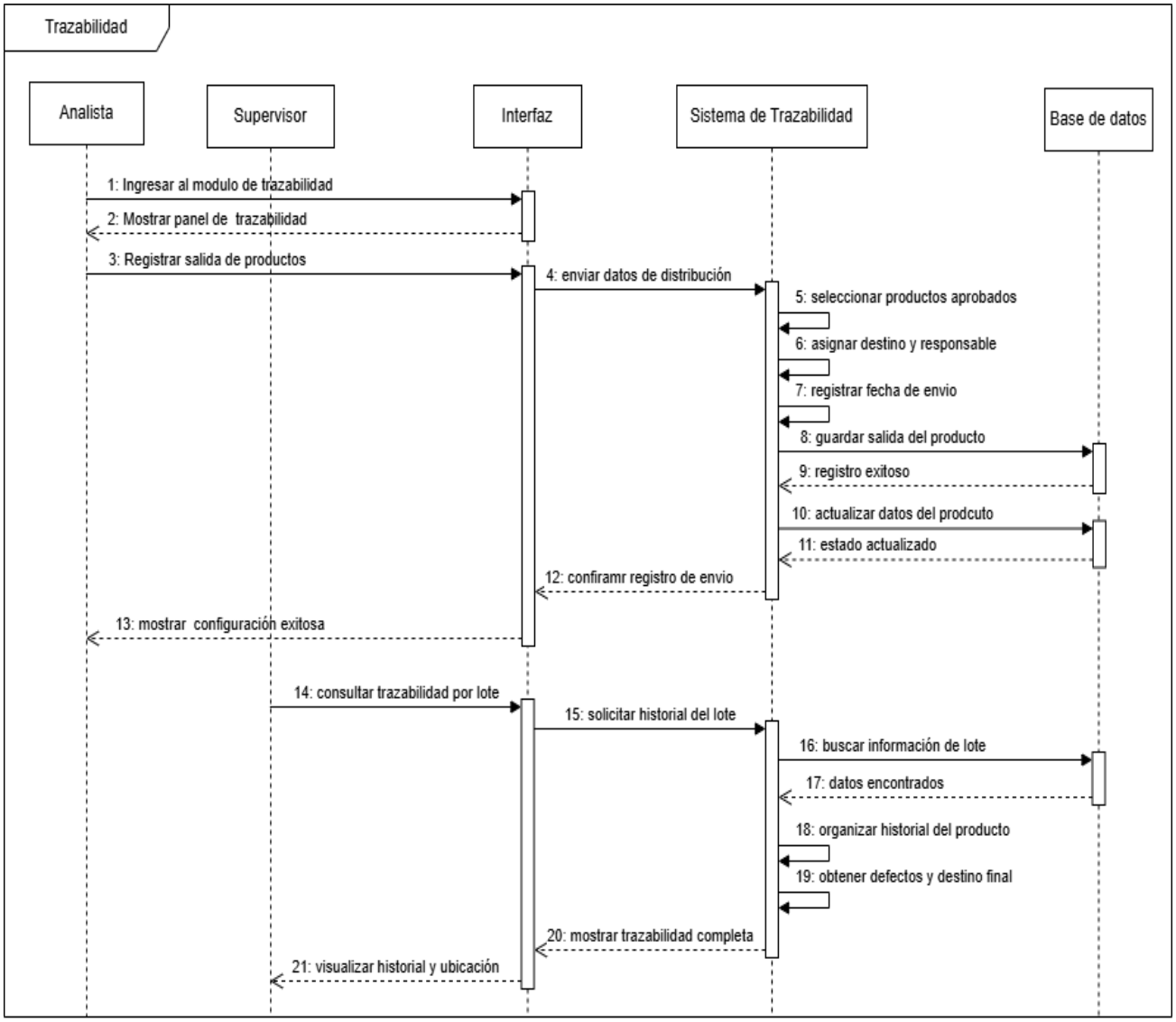


Figura 26. Diagrama de Secuencia - Trazabilidad

Fuente: *Elaboración propia*

- **Reportes**

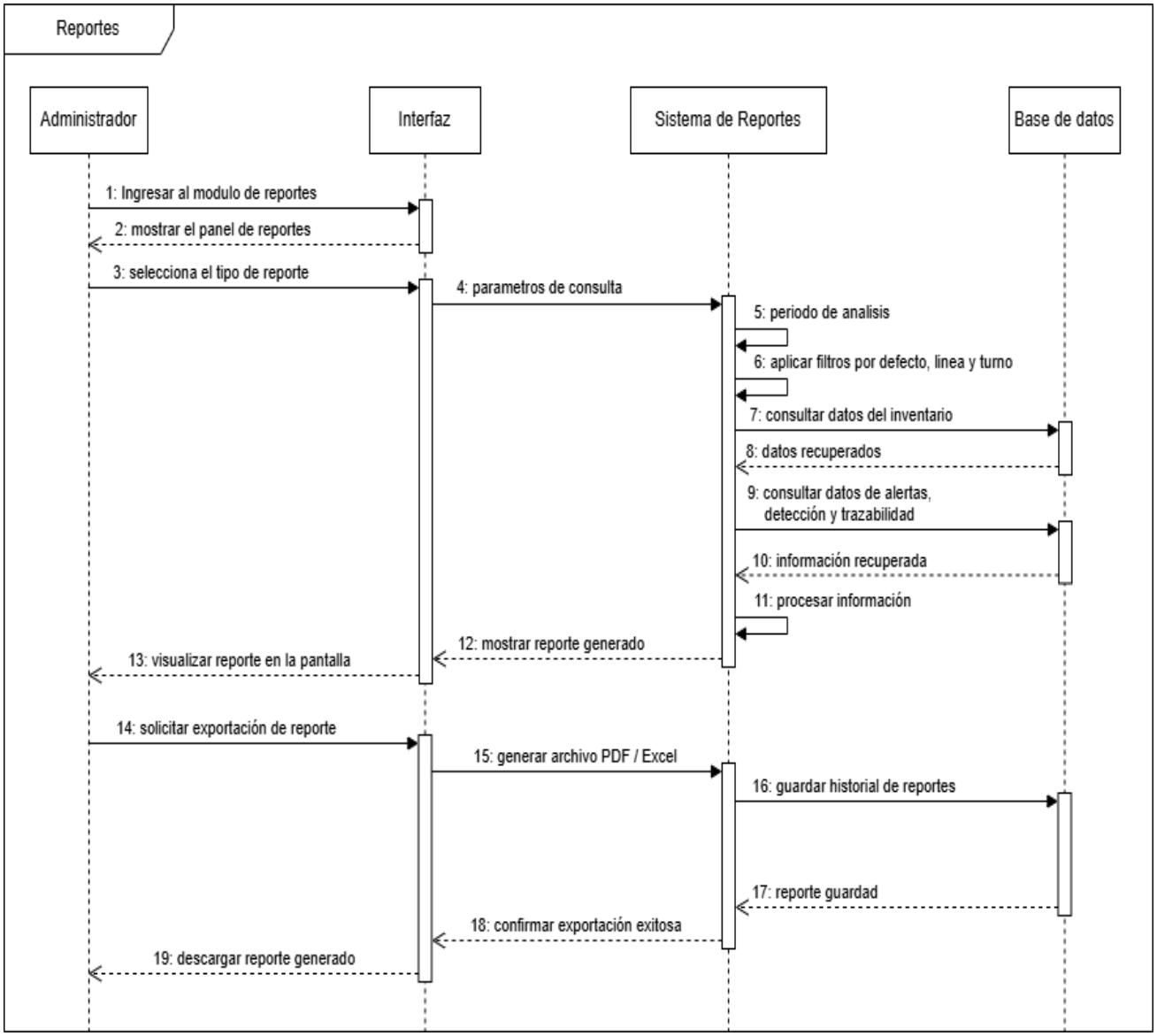


Figura 27. Diagrama de Secuencia - Reportes

Fuente: *Elaboración propia*

5.3.1.4. Diagrama de Colaboración

- **Sesiones**

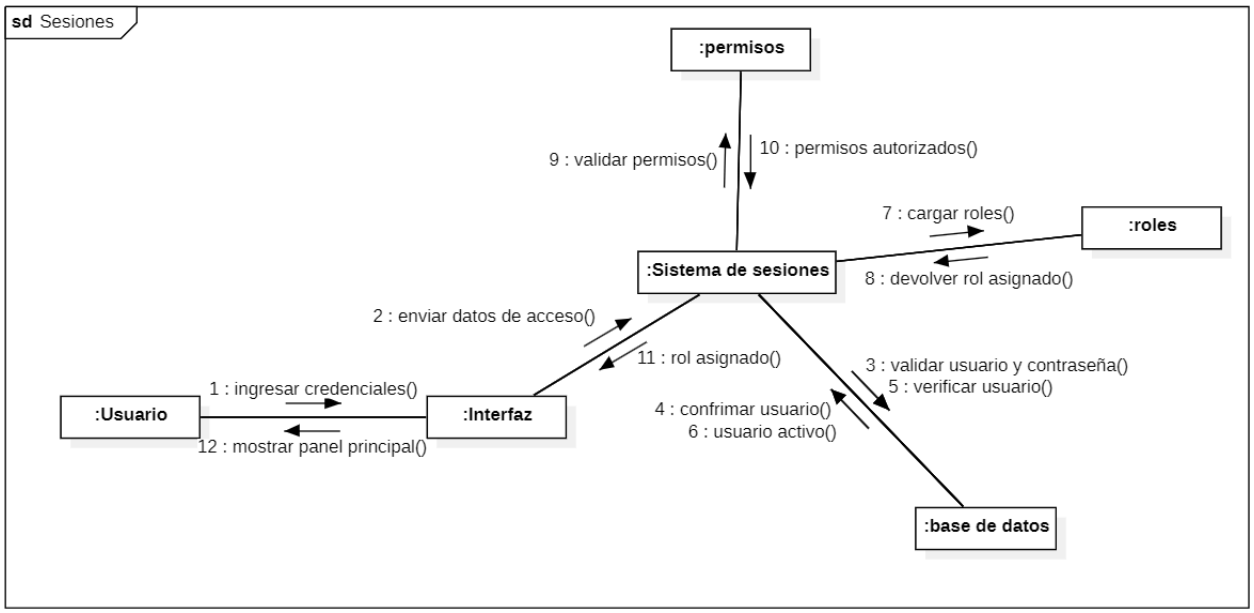


Figura 28. Diagrama de Colaboración - Sesiones
Fuente: *Elaboración propia*

- **Detección Automática**

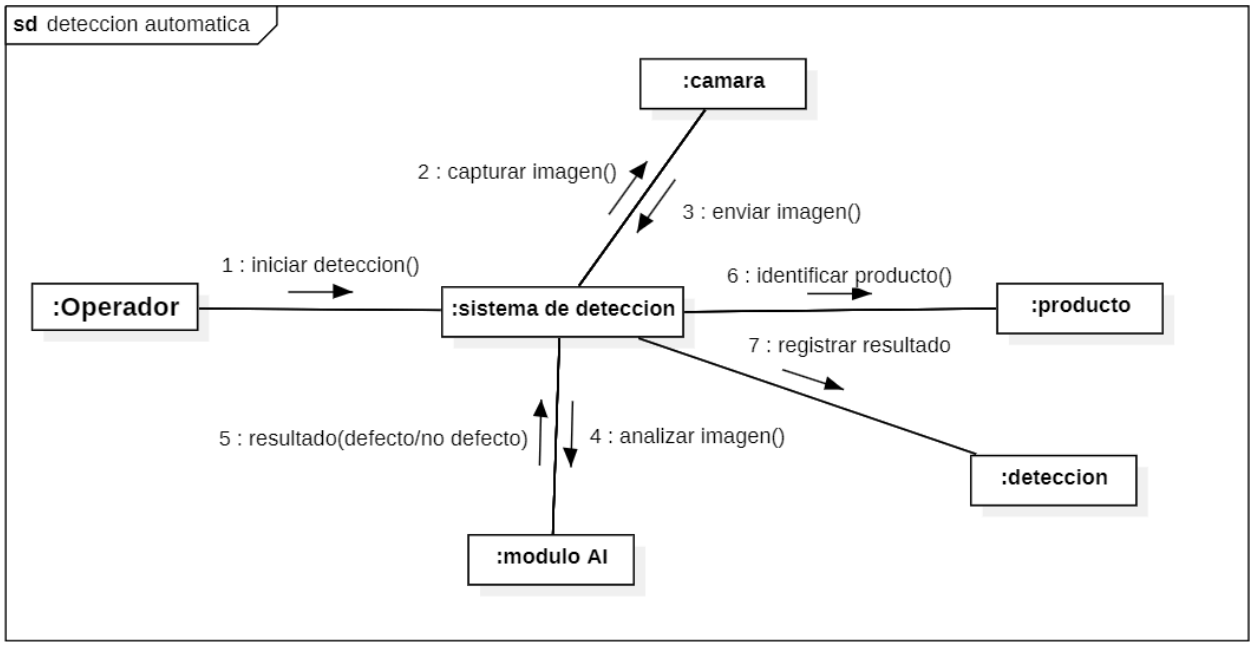


Figura 29. Diagrama de Colaboración - Detección Automática
Fuente: *Elaboración propia*

- **Monitoreo en Tiempo Real**

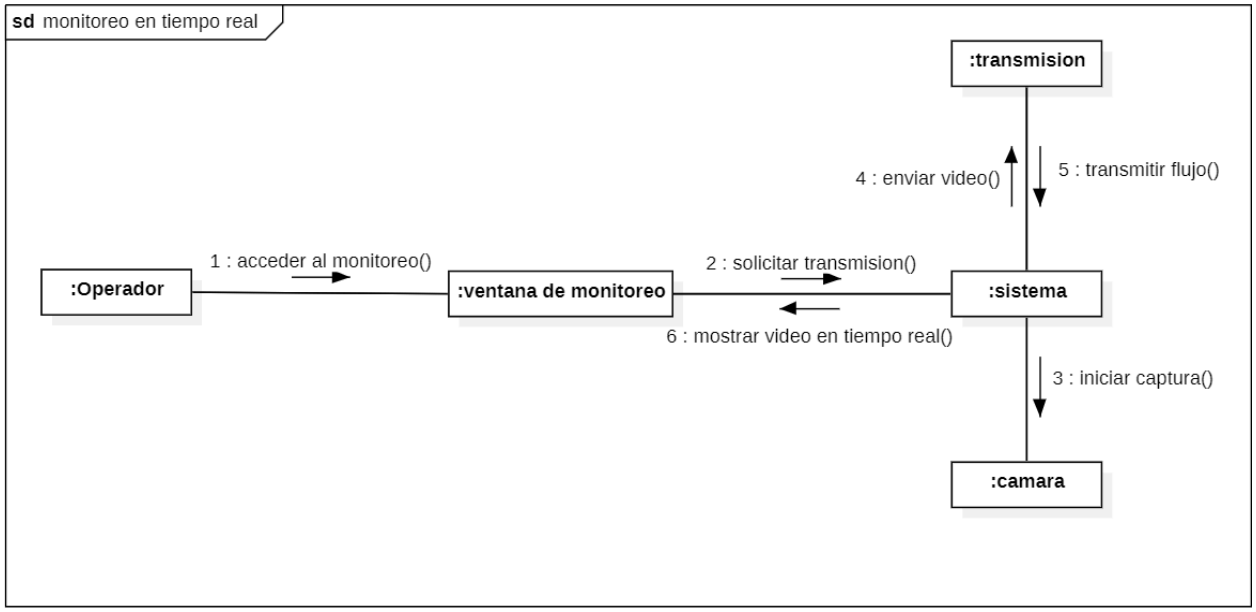


Figura 30. Diagrama de Colaboración - Monitoreo en tiempo real
Fuente: *Elaboración propia*

- **Alertas Automáticas**

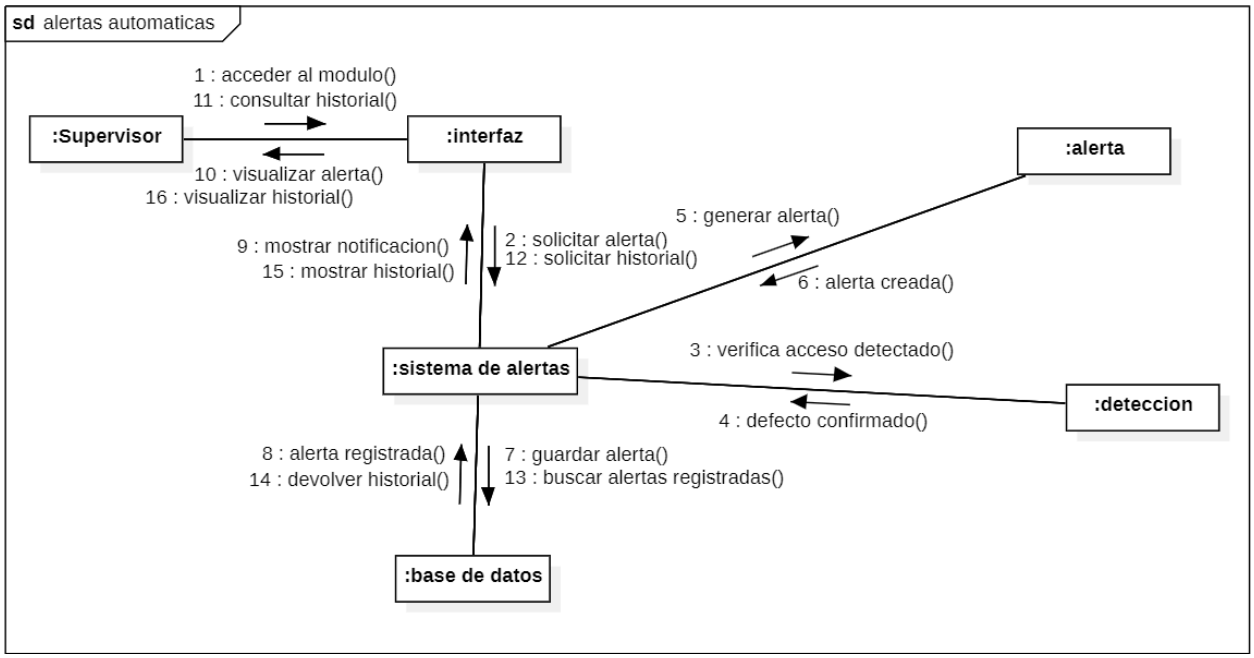


Figura 31. Diagrama de Colaboración - Alertas Automáticas
Fuente: *Elaboración propia*

- **Inventario de Productos**

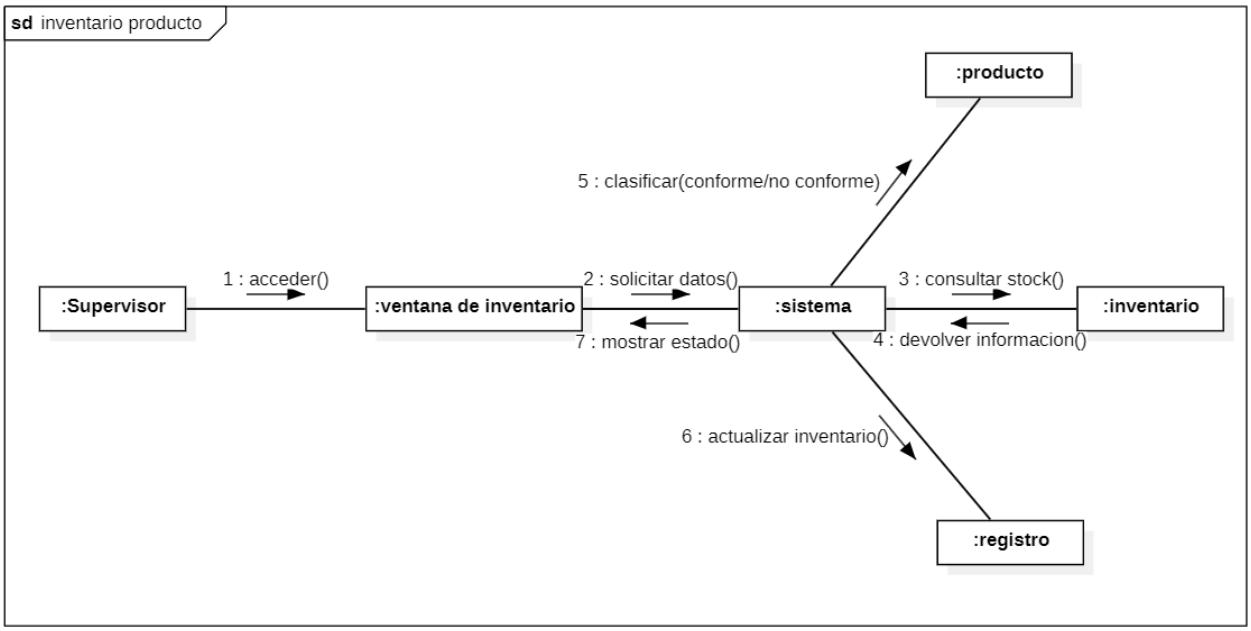


Figura 32. Diagrama de Colaboración - Inventario de Productos
Fuente: *Elaboración propia*

- **Trazabilidad**

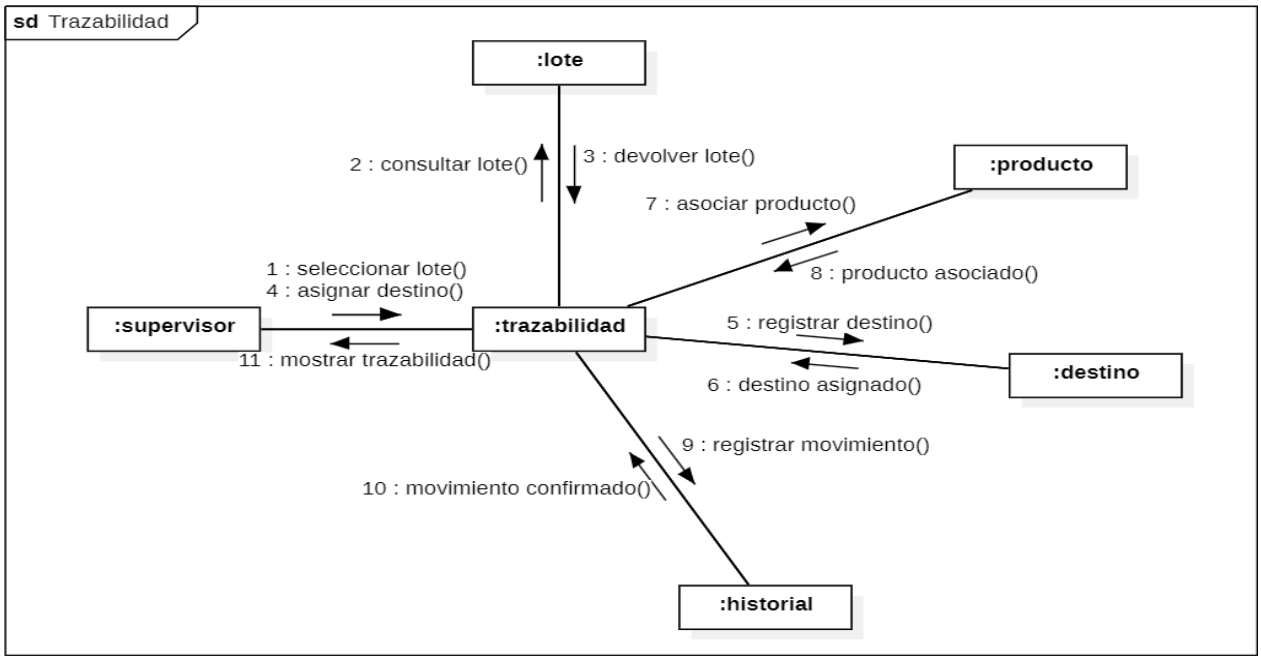


Figura 33. Diagrama de Colaboración - Trazabilidad
Fuente: *Elaboración propia*

- **Reportes**

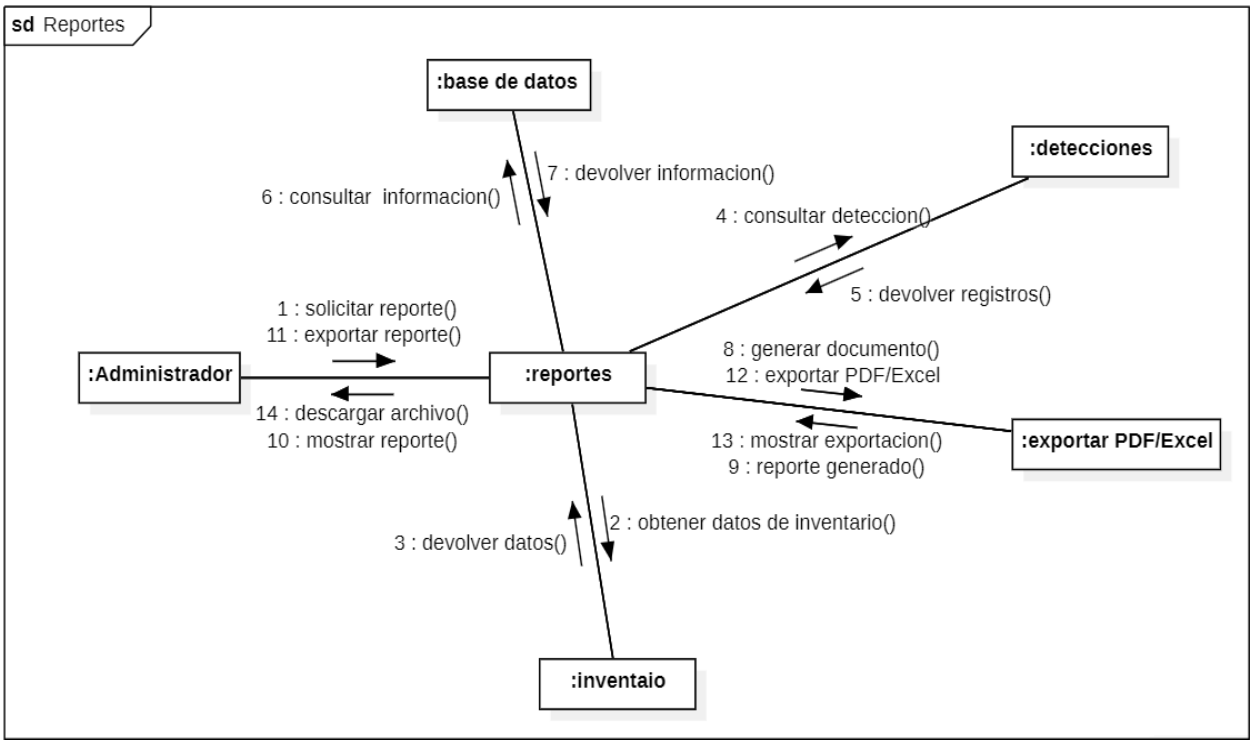


Figura 34. Diagrama de Colaboración - Reportes
Fuente: *Elaboración propia*

5.3.1.5. Diagrama de Actividades

- **Sesiones**

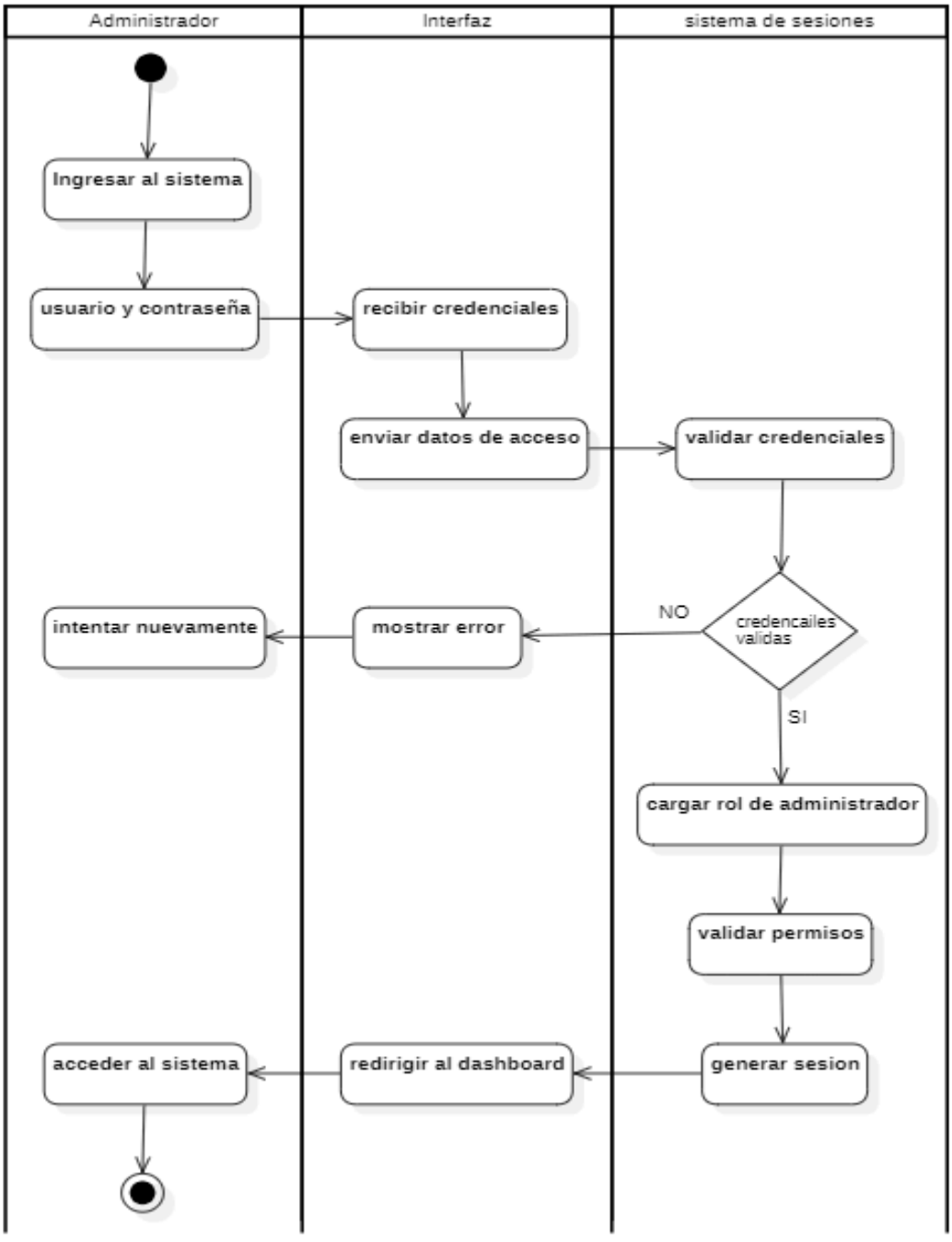


Figura 35. Diagrama de Actividades - Sesiones
Fuente: *Elaboración propia*

- **Detección Automática**

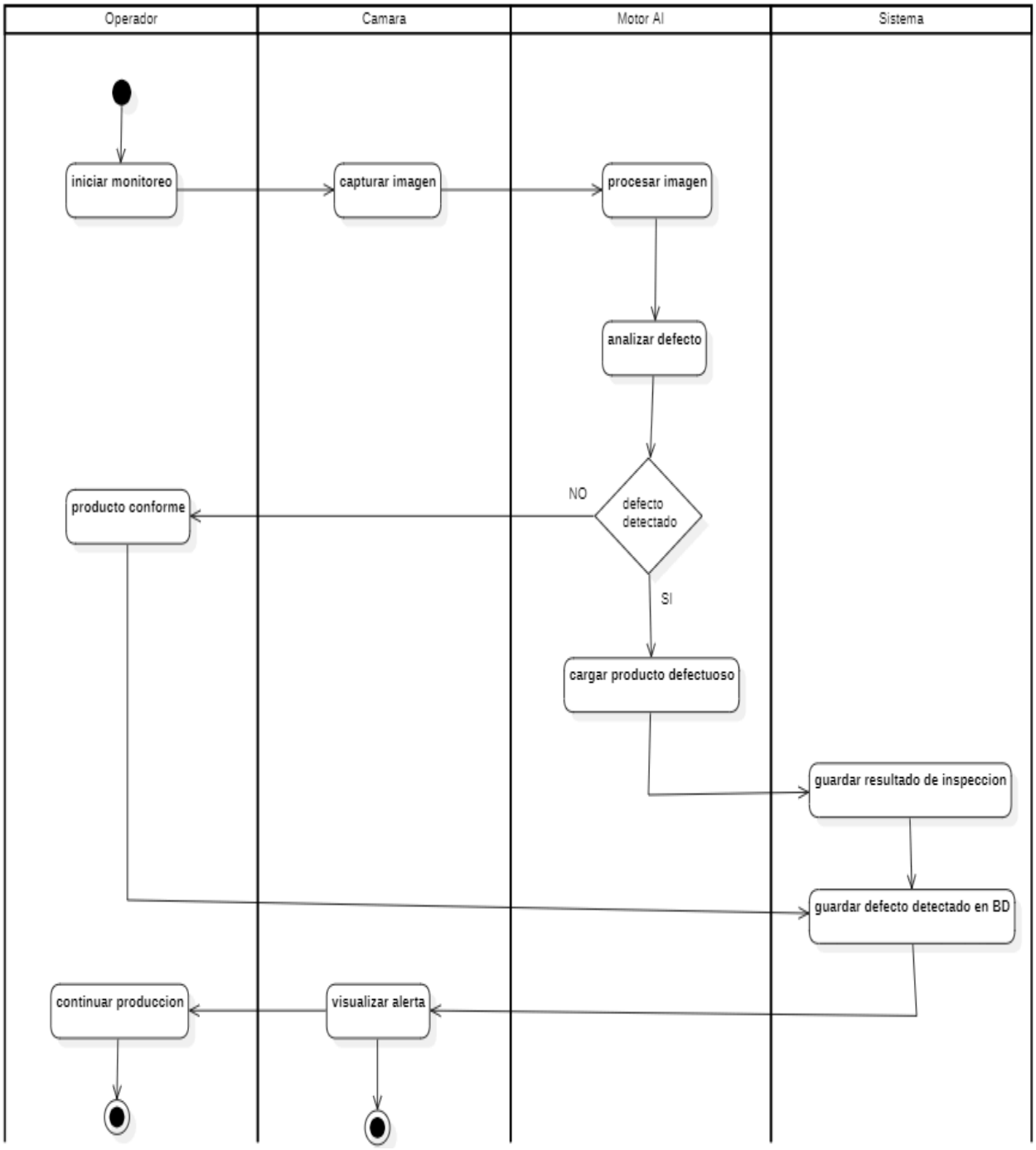


Figura 36. Diagrama de Actividades - Detección Automática
Fuente: *Elaboración propia*

- **Monitoreo en Tiempo Real**

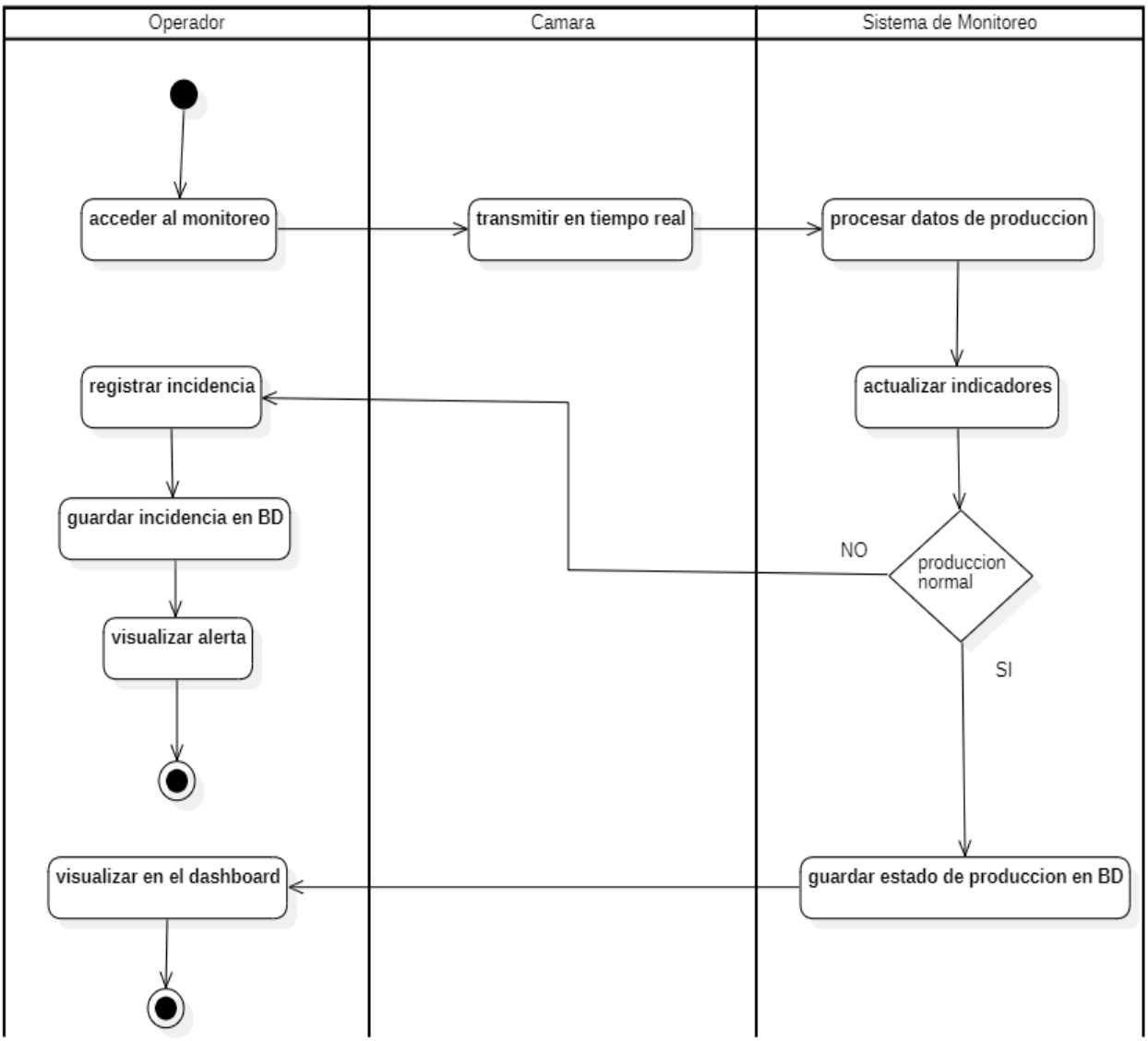


Figura 37. Diagrama de Actividades - Monitoreo en Tiempo Real
Fuente: *Elaboración propia*

- **Alertas Automáticas**

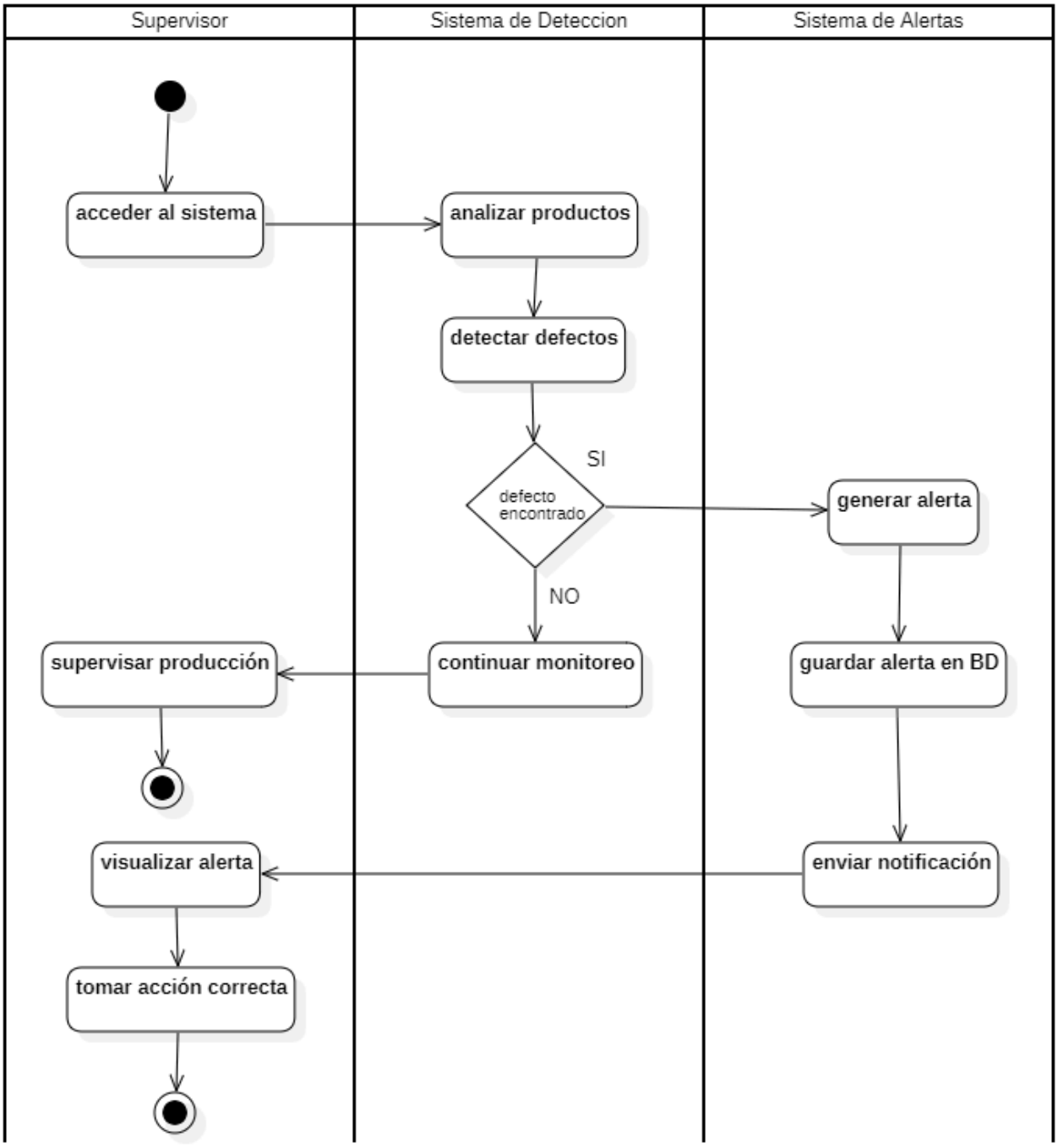


Figura 38. Diagrama de Actividades - Monitoreo en Tiempo Real
Fuente: *Elaboración propia*

- **Inventario de Producto**

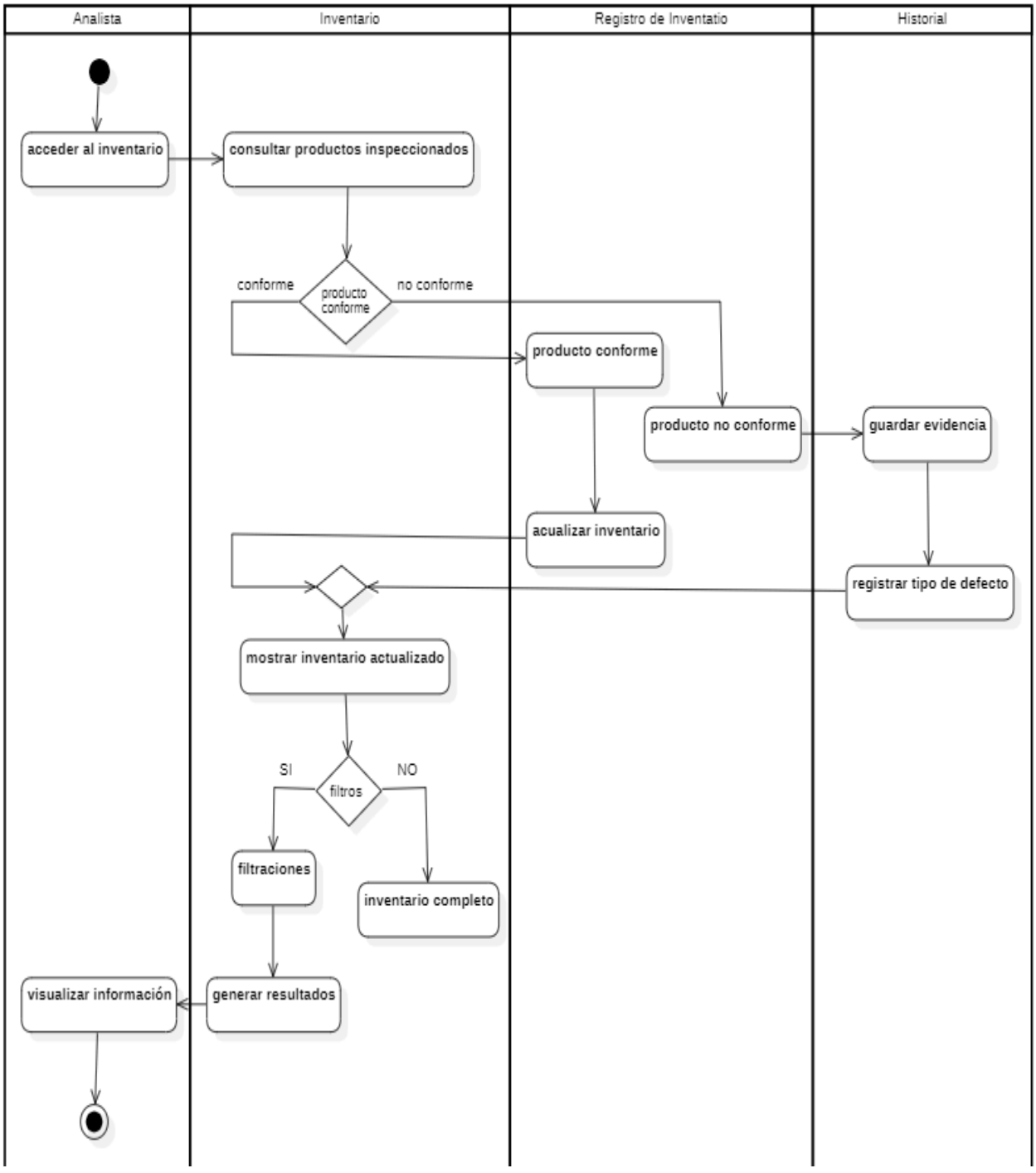


Figura 39. Diagrama de Actividades - Inventario de Productos
Fuente: *Elaboración propia*

- **Trazabilidad**

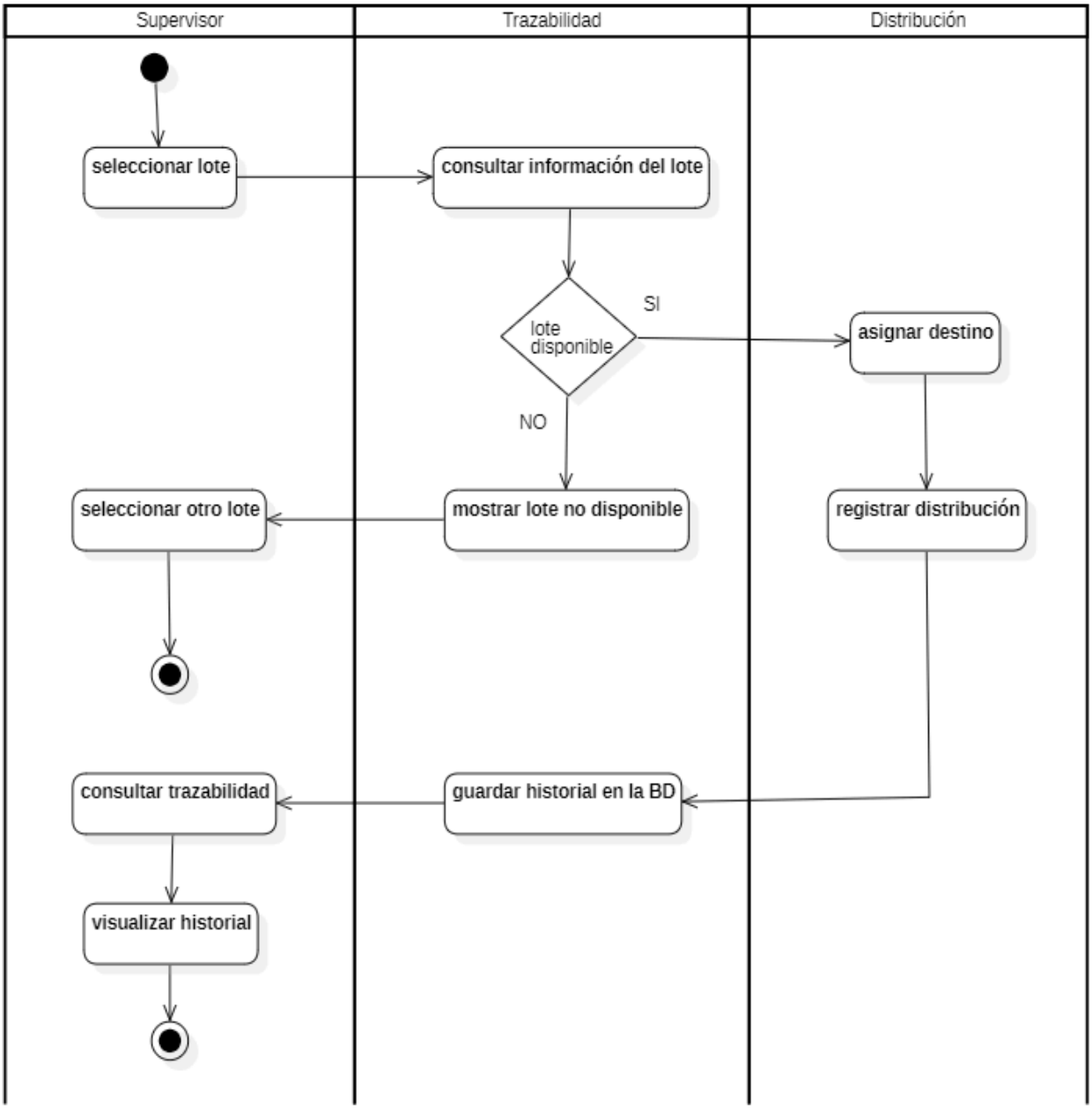


Figura 40. Diagrama de Actividades - Trazabilidad
Fuente: *Elaboración propia*

- **Reportes**

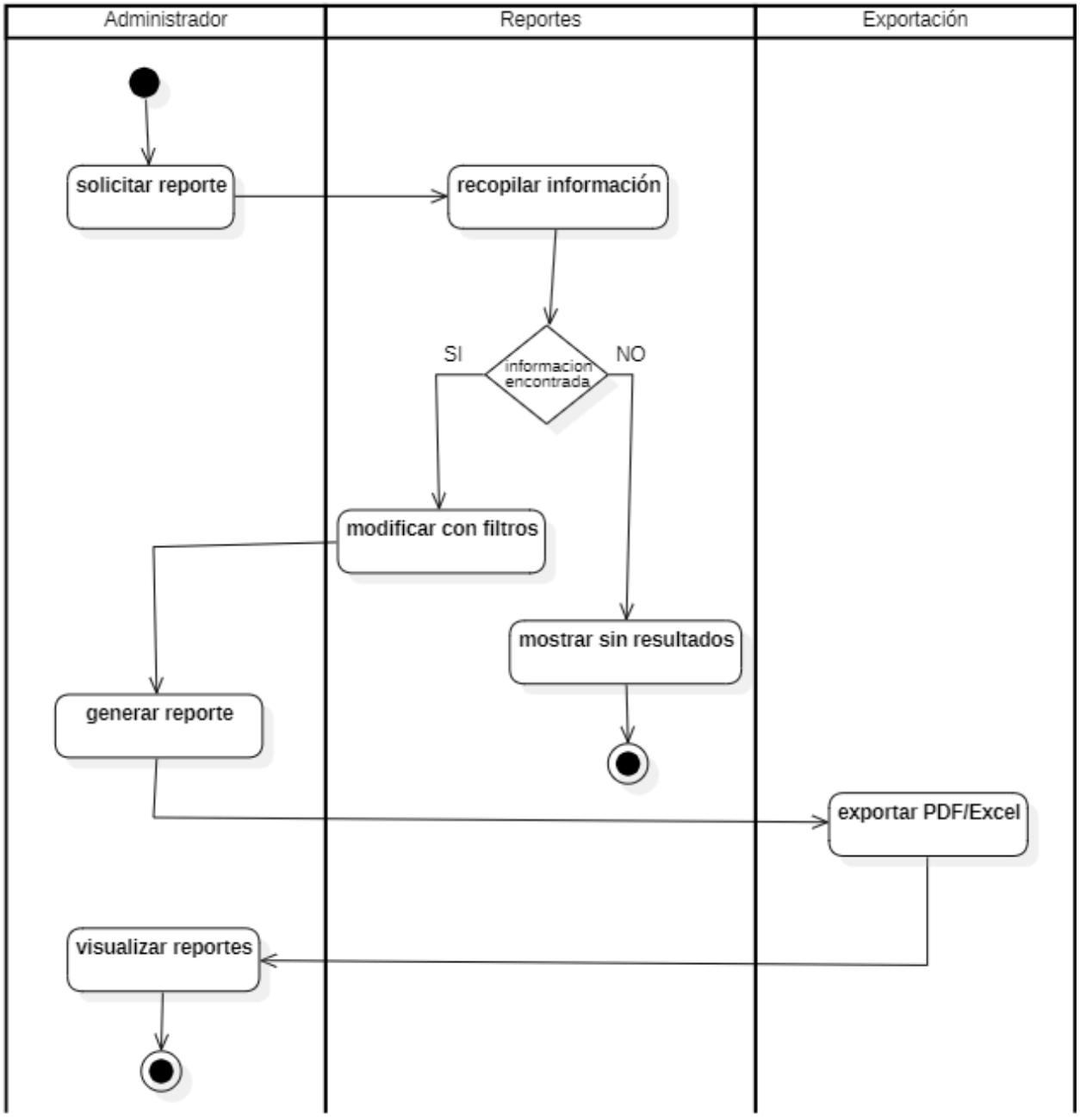


Figura 41. Diagrama de Actividades - Reportes
Fuente: *Elaboración propia*

5.3.1.6. Diagrama de Estados

- **Sesiones**

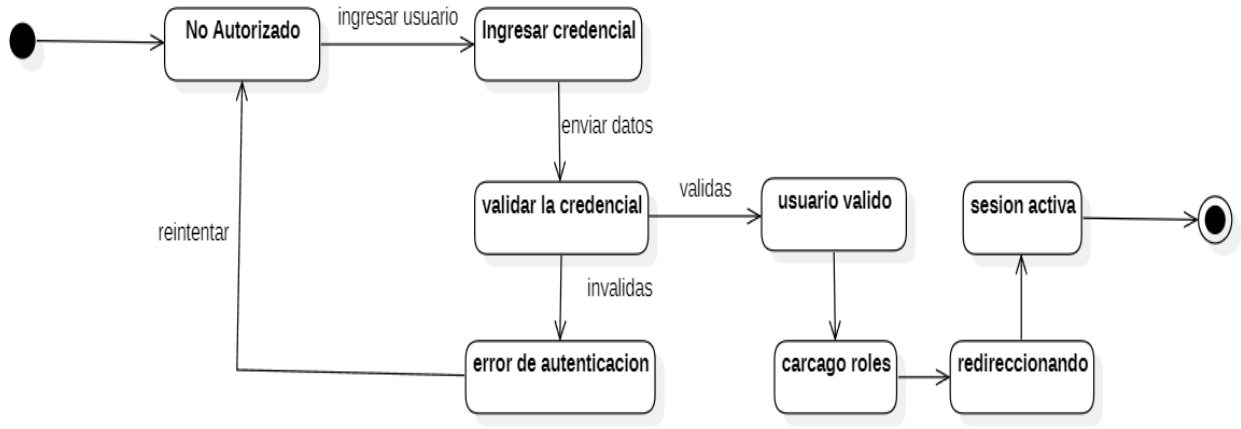


Figura 42. Diagrama de Estado - Sesiones

Fuente: *Elaboración propia*

- **Detección Automática**

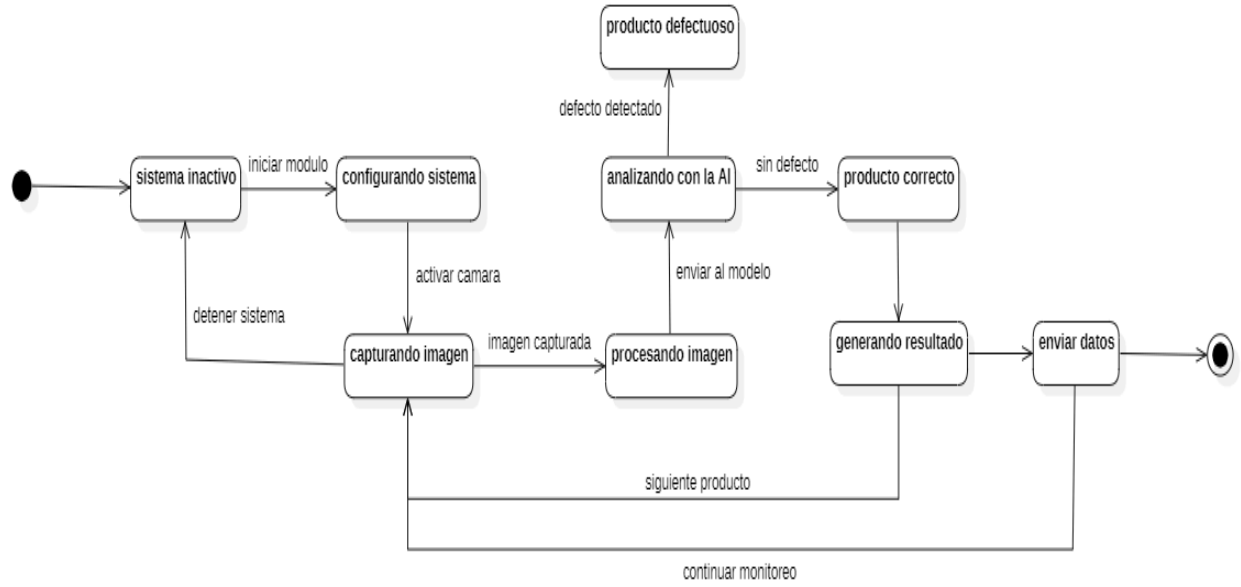


Figura 43. Diagrama de Estado - Detección Automática

Fuente: *Elaboración propia*

- **Monitoreo en Tiempo Real**

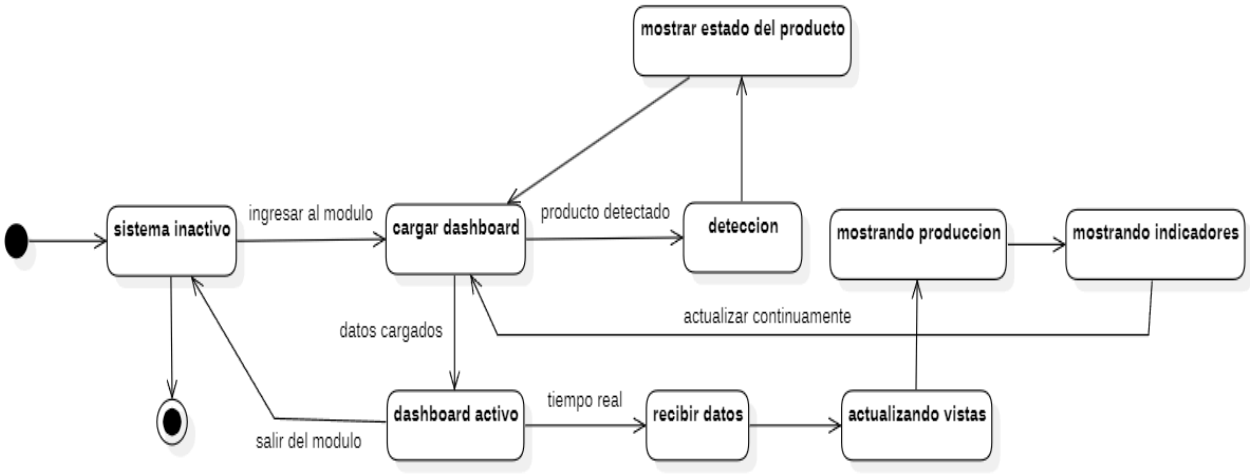


Figura 44. Diagrama de Estado - Monitoreo en tiempo real

Fuente: *Elaboración propia*

- **Alertas Automáticas**

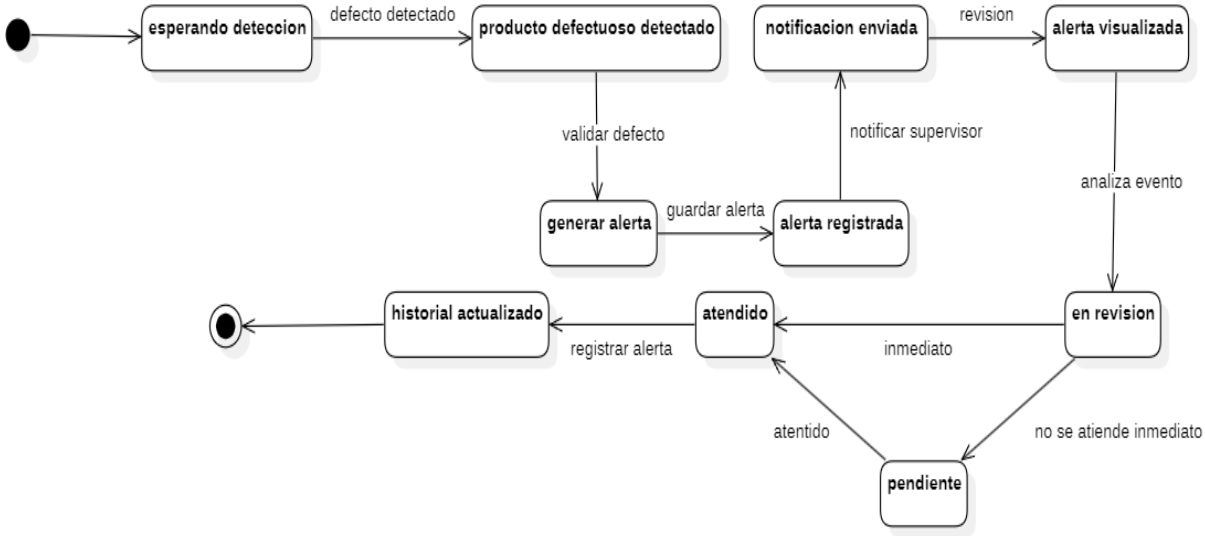


Figura 45. Diagrama de Estado - Alertas Automáticas

Fuente: *Elaboración propia*

- **Inventario Productos**

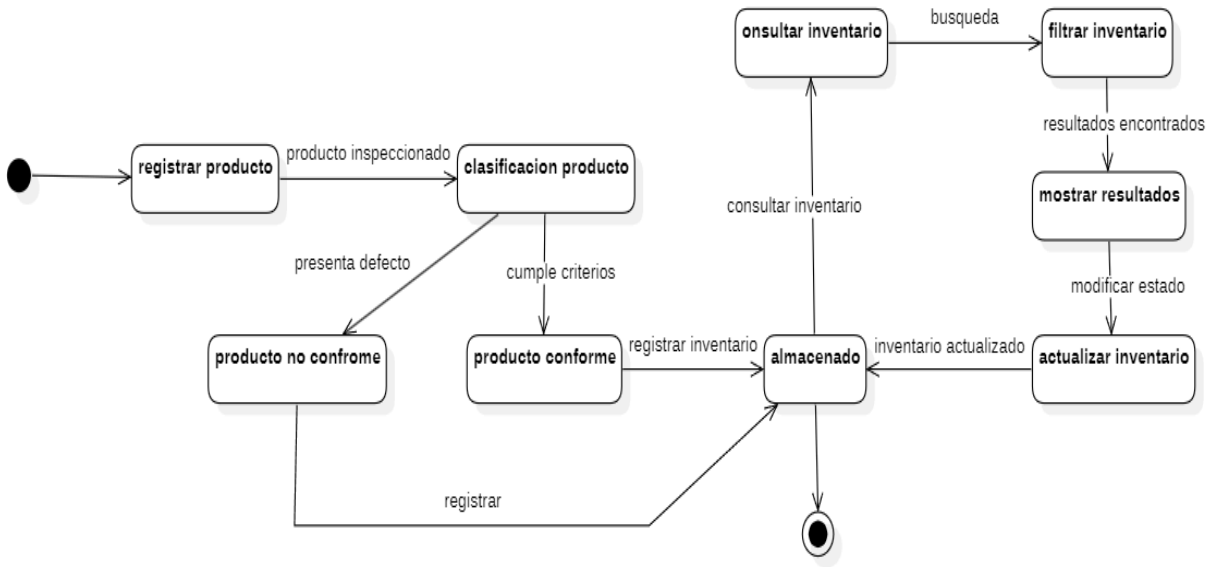


Figura 46. Diagrama de Estado - Inventario Productos

Fuente: *Elaboración propia*

- **Trazabilidad**



Figura 47. Diagrama de Estado - Trazabilidad

Fuente: *Elaboración propia*

- **Reportes**

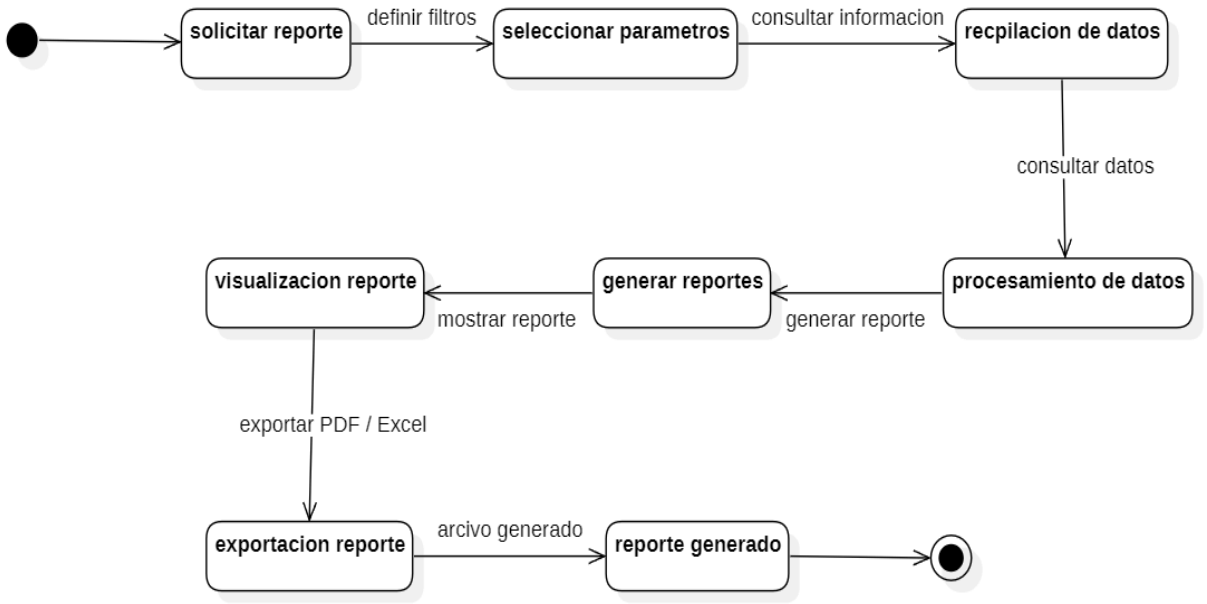


Figura 48. Diagrama de Estado - Reportes

Fuente: *Elaboración propia*

5.3.1.7. Diagrama de Clases de Alto Nivel

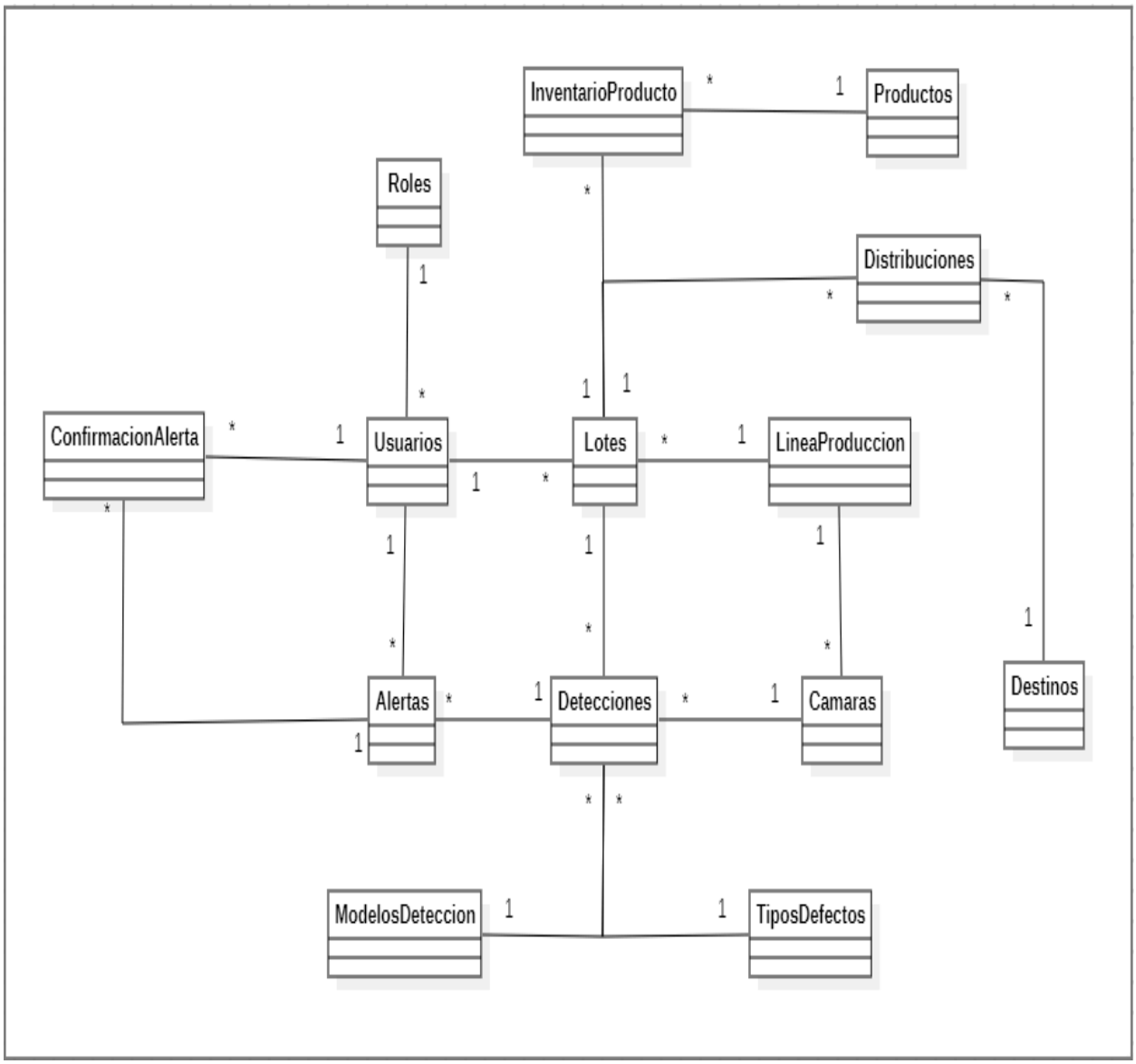


Figura 49. Diagrama de Clases de Alto Nivel

Fuente: *Elaboración propia*

5.3.1.8. Diagrama de Clases de Bajo Nivel

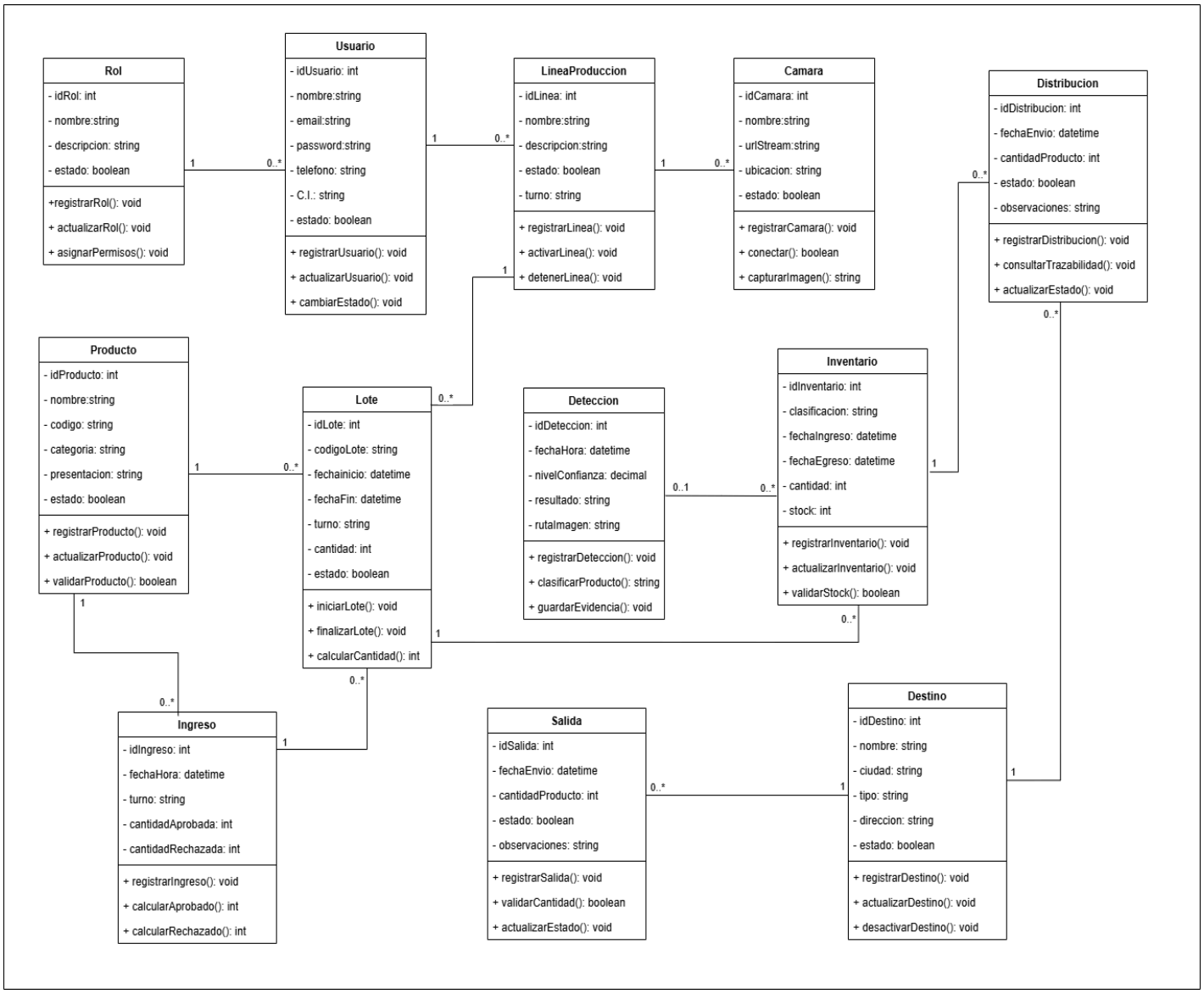


Figura 50. Diagrama de Clases de Bajo Nivel

Fuente: *Elaboración propia*

5.3.1.9. Diagrama Físico

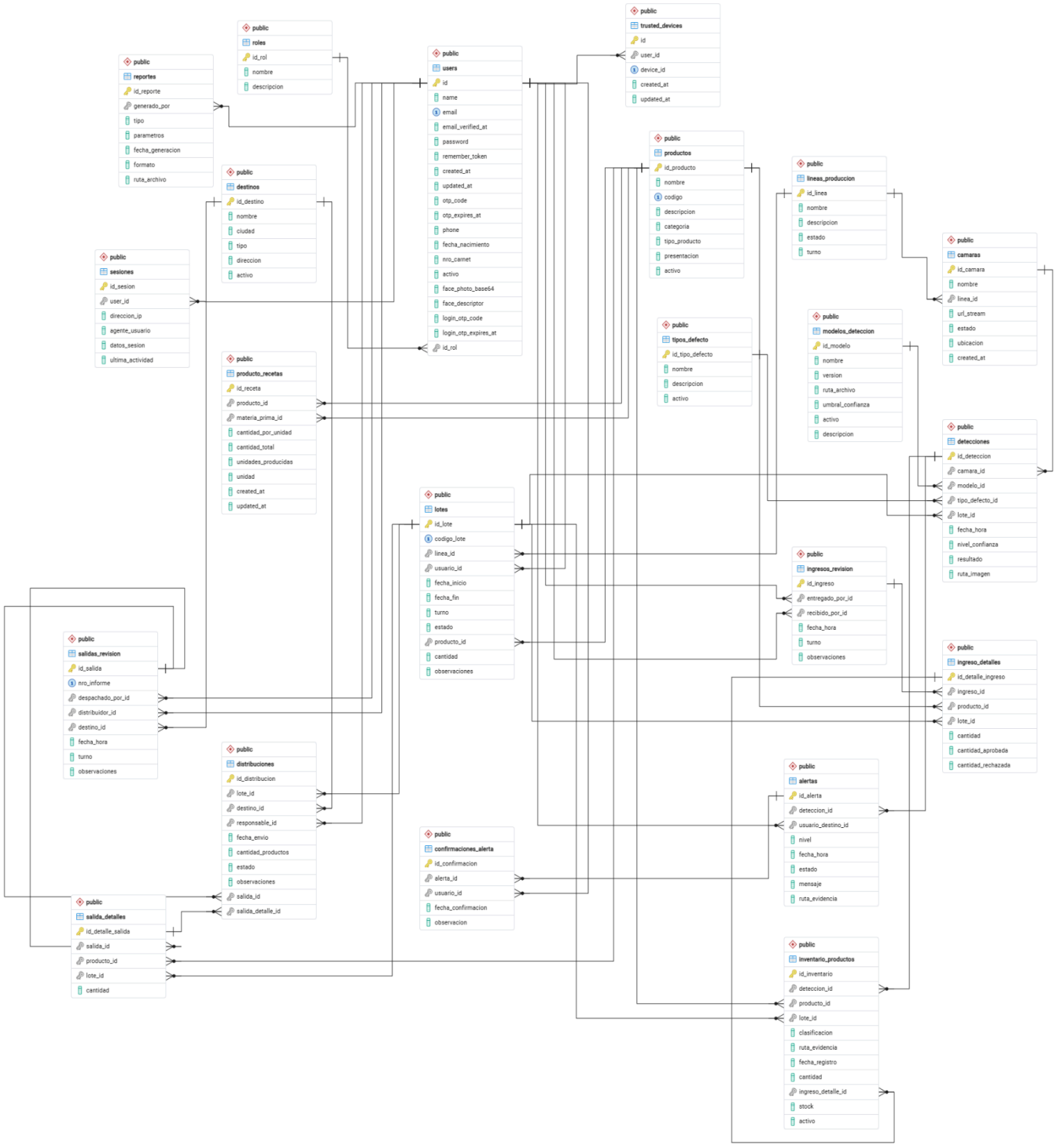


Figura 51. Diagrama Físico

Fuente: *Elaboración propia*

5.3.1.10. Mapa Navegacional del sistema

- **Mapa navegacional - Administrador**

El administrador tiene acceso completo al sistema, incluyendo la gestión de usuarios, configuraciones generales y generación de reportes. Su rol es asegurar el correcto funcionamiento organizacional y técnico.

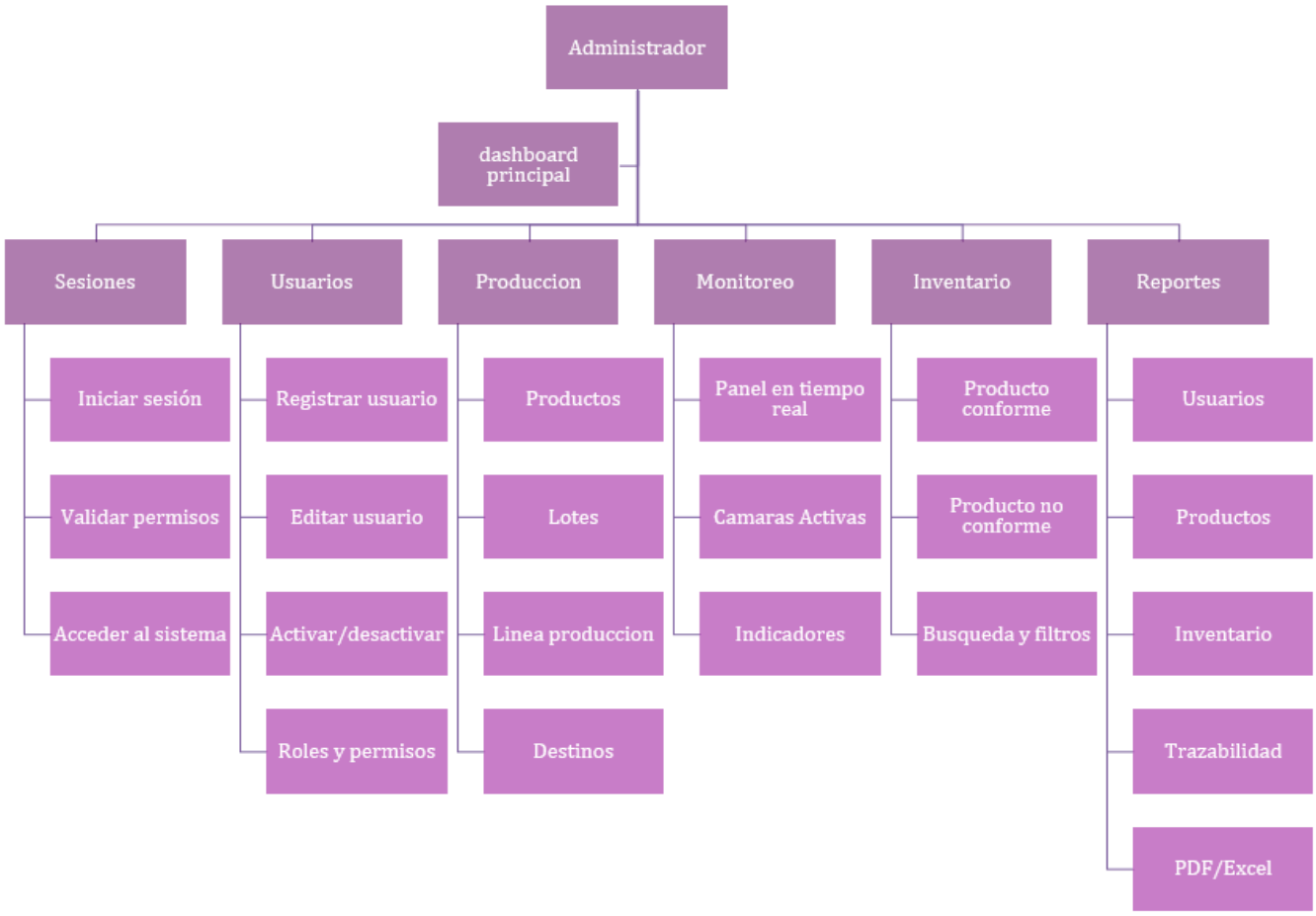


Figura 52. Mapa Navegacional - Administrador

Fuente: *Elaboración propia*

- **Mapa navegacional - Supervisor**

El supervisor tiene acceso a la distribución de los productos tanto como los ingresos y salidas, supervisión de que las cámaras funcionen correctamente, generación de reportes.

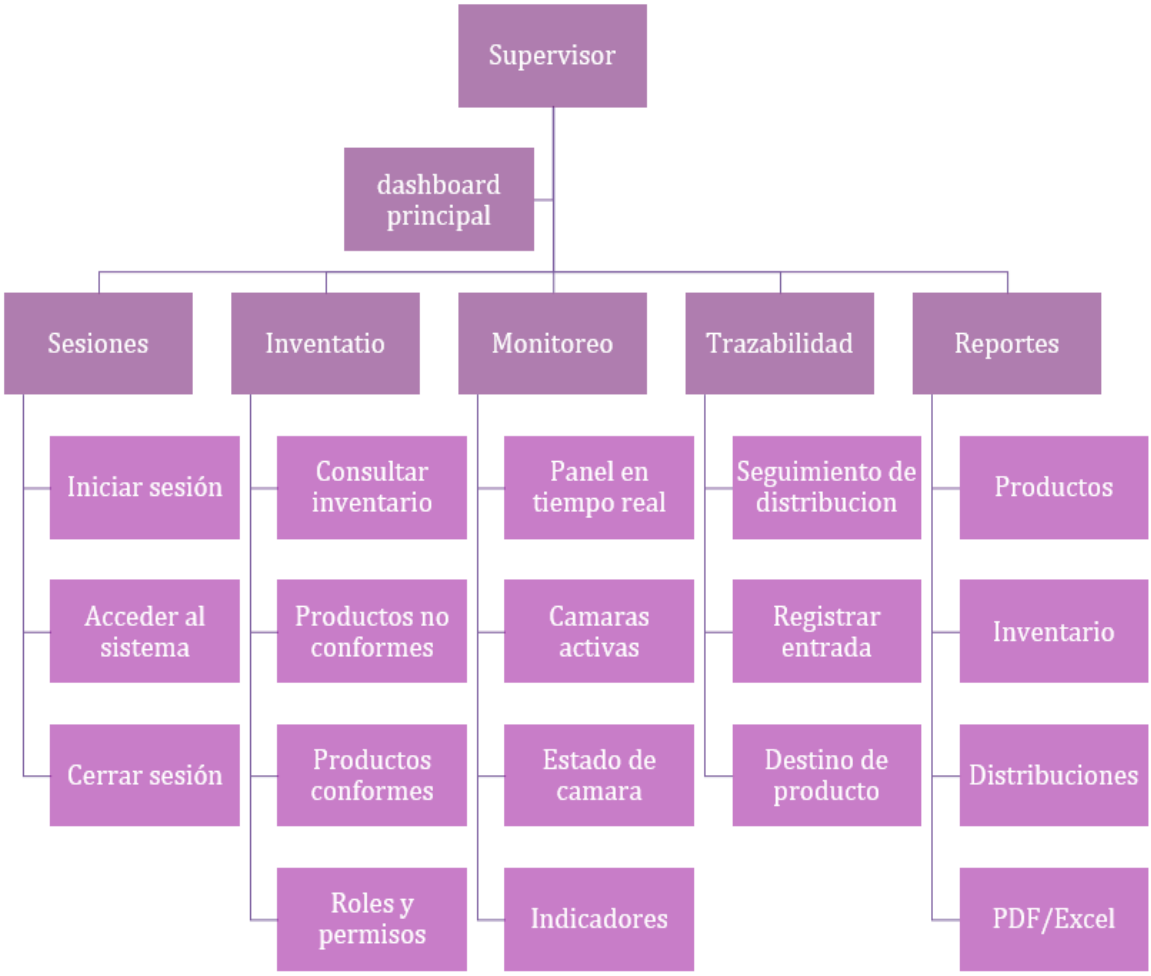


Figura 53. Mapa Navegacional - Supervisor

Fuente: *Elaboración propia*

- **Mapa navegacional - Operador**

El operador tiene acceso a los monitoreos puestos en cada línea de producción como cada rincón de la industria, verificación de que las cámaras funcionen y control sobre la línea de producción.

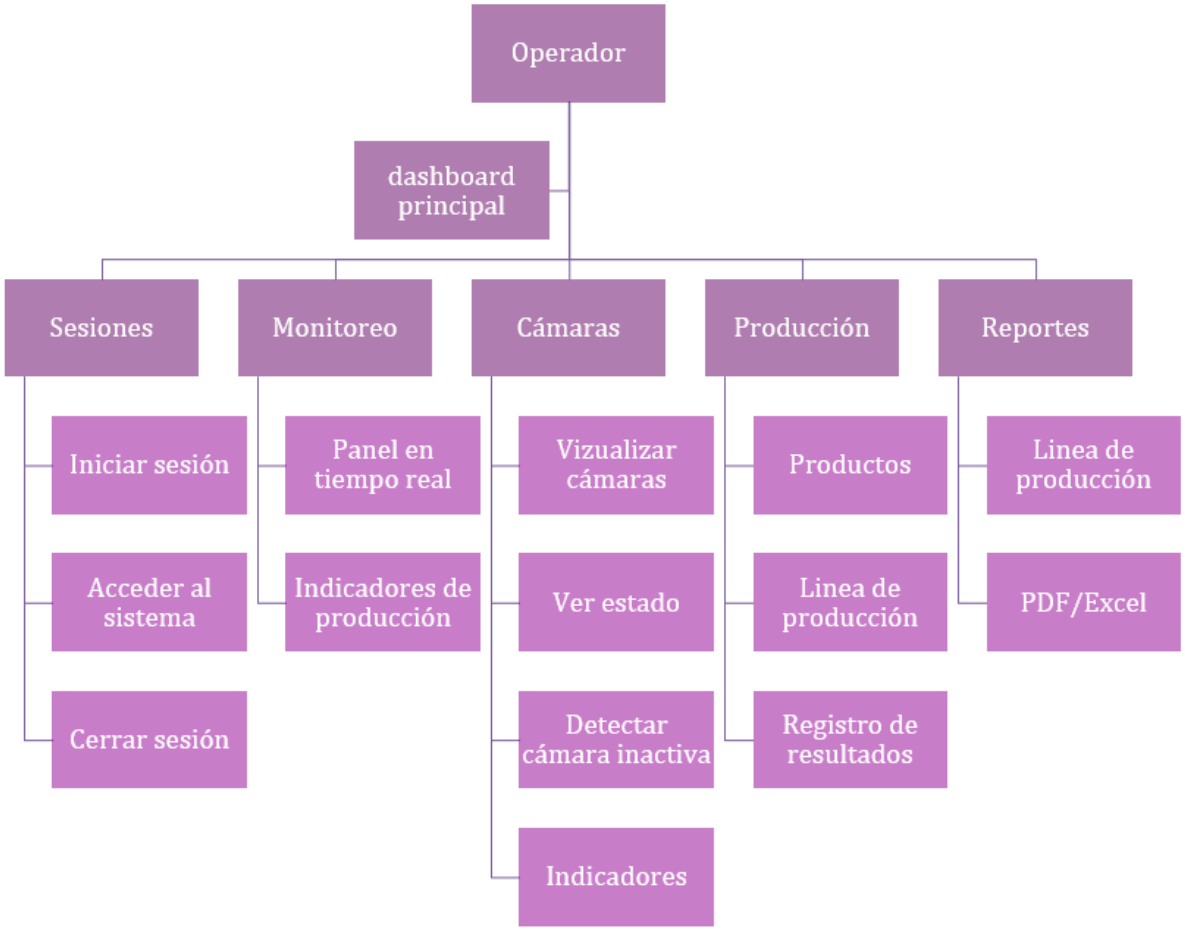


Figura 54. Mapa Navegacional - Operador

Fuente: *Elaboración propia*

- **Mapa navegacional - Distribuidor**

El Distribuidor tiene acceso a un apartado donde le muestre sus entregas y al destino que tiene que ser entregado, y después de entregar el producto marcar como entregado para la supervisión del supervisor que lo asignó.

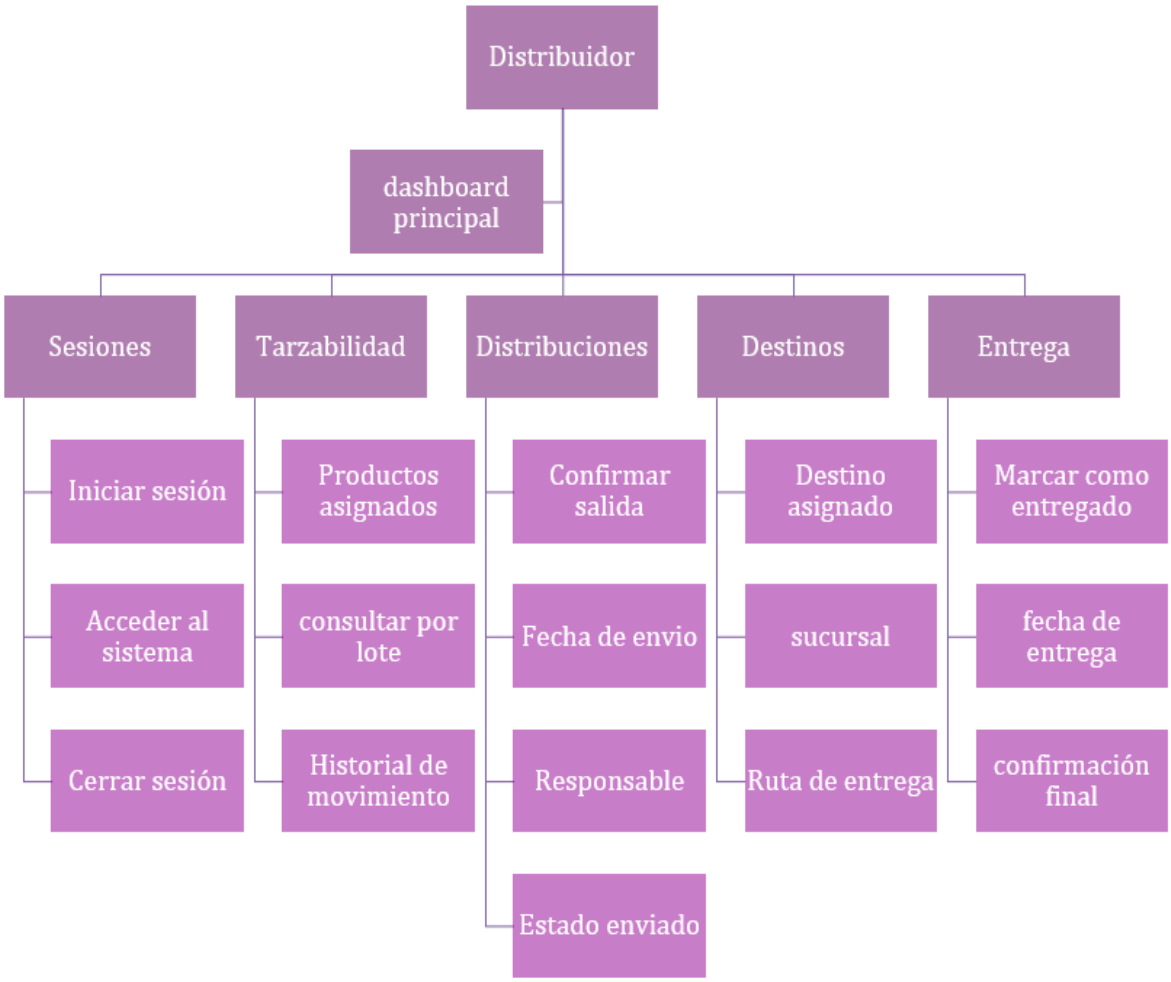


Figura 55. Mapa Navegacional - Distribuidor

Fuente: *Elaboración propia*

5.3.1.11. Arquitectura del Software

El sistema desarrollado se construyó siguiendo una arquitectura en capas basada en el patrón de diseño Modelo-Vista-Controlador (MVC), implementado a través del framework Laravel. Esta estructura arquitectónica permitió organizar de manera clara y estructurada los diferentes componentes del sistema de inspección y detección de productos dañados en la línea de producción de Industria de Alimentos Ltda. (INAL). La adopción del patrón MVC no solo favoreció una codificación más ordenada y modular, sino que también facilitó el mantenimiento, la escalabilidad y las futuras ampliaciones del sistema, aspectos especialmente relevantes dado el carácter crítico del entorno industrial en el que opera.

Durante la construcción del sistema, se definieron tres capas fundamentales que estructuran el flujo de información y la lógica de procesamiento del sistema:

- **Capa de Datos (Modelo)**

Representada por los modelos de Laravel, esta capa es la responsable de interactuar directamente con la base de datos PostgreSQL. Cada modelo define la estructura de sus tablas y las relaciones existentes entre ellas. Entre los modelos principales del sistema se encuentran los correspondientes a los registros de productos inspeccionados, los usuarios del sistema (operarios, supervisores y administradores), y los parámetros de configuración del proceso de inspección. PostgreSQL fue seleccionado como sistema gestor de base de datos por su alta confiabilidad, integridad referencial y capacidad para manejar grandes volúmenes de registros de producción con transacciones complejas, garantizando la consistencia de los datos almacenados.

- **Capa de Presentación (Vista)**

Esta capa fue implementada utilizando Blade y Vue.js en combinación con el framework Laravel. Blade actúa como motor de plantillas integrado con Laravel para la generación de vistas dinámicas en el servidor, mientras que Vue.js permite la actualización reactiva de datos en tiempo real sin necesidad de recargar la página, lo que resulta fundamental para el monitoreo continuo de la línea de producción. Desde esta capa, el usuario puede interactuar visualmente con el sistema a través de interfaces que permiten visualizar el estado de la línea de producción en tiempo real, revisar los registros e historial de productos inspeccionados, acceder a las imágenes y evidencias visuales de los productos detectados como defectuosos, consultar reportes de control de calidad y gestionar configuraciones del sistema, tales como la administración de usuarios, roles y parámetros de detección.

- **Capa de Lógica de Negocio (Controlador)**

Esta capa contiene los controladores de Laravel, que actúan como intermediarios entre la capa de presentación y la capa de datos. Los controladores son los encargados de validar la entrada de datos provenientes del usuario, determinar qué vista mostrar o qué acción ejecutar en función de las solicitudes recibidas, y gestionar la comunicación con el módulo de inteligencia artificial. Este módulo, desarrollado en Python con la librería Ultralytics y el modelo YOLOv8, ejecuta la detección de productos defectuosos a partir de los fotogramas capturados por OpenCV en la línea de producción. Los controladores también coordinan el procesamiento de resultados de detección, el almacenamiento de evidencias visuales y la generación de alertas en tiempo real hacia los operarios y supervisores responsables del control de calidad.

- **Evaluación del Diseño**

El uso de la arquitectura MVC permitió desarrollar un sistema robusto, flexible y fácil de mantener. La separación entre vistas, controladores y modelos garantiza que cada componente del sistema pudiera ser trabajado, corregido o extendido de forma independiente, sin afectar el funcionamiento de las demás capas. De este modo, futuras mejoras en el modelo de detección o cambios en la interfaz de usuario pueden realizarse de manera independiente, sin comprometer la estabilidad del sistema en su conjunto.

La arquitectura en capas implementada mediante el patrón MVC constituyó una decisión de diseño acertada para el presente proyecto, ya que proporcionó una base sólida y organizada para integrar las tecnologías involucradas, hasta la gestión de datos en PostgreSQL y la presentación de resultados en tiempo real a través de interfaces Vue.js y Blade en Laravel.

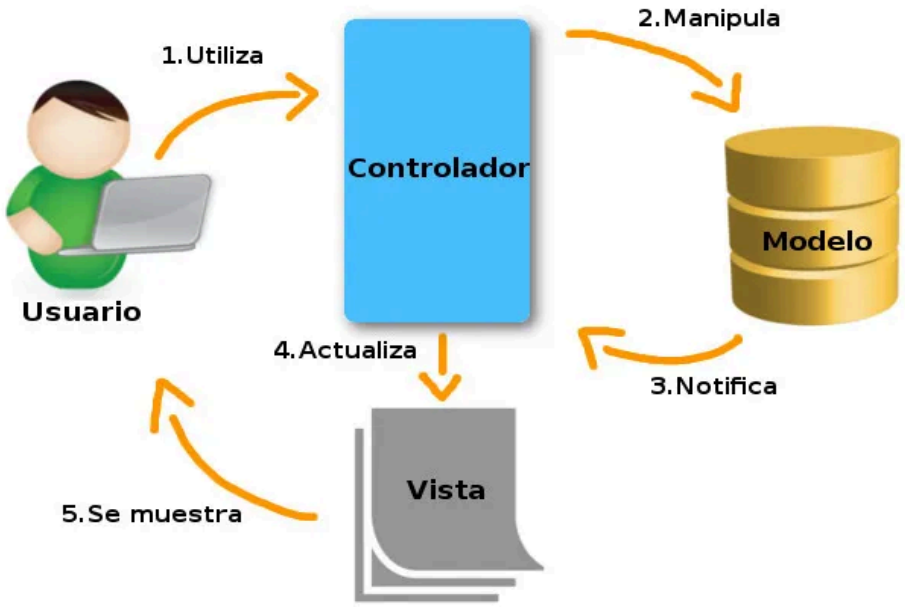


Figura 56. Arquitectura del Software

Fuente: *Tomado de Platzi https://platzi.com/blog/que-es-el-patron-mvc/*

5.4. Fase 3 – Post Game

5.4.1. Seguridad del Software

Las características de seguridad implementadas en el sistema de control de calidad de la Industria de Alimentos Ltda. (INAL) garantizan la protección de la información, el control de accesos y la integridad de los datos gestionados. A continuación se detallan las medidas de seguridad aplicadas:

Nº	Función	Atributo	Detalles
1	Validación de Contraseña Segura	Seguridad	El sistema exige que cada contraseña cumpla con criterios mínimos de complejidad, reduciendo el riesgo de accesos no autorizados mediante ataques de fuerza bruta.
2	Cifrado de Contraseñas	Seguridad	Las contraseñas de los usuarios se cifran utilizando el algoritmo bcrypt integrado en Laravel, garantizando que no puedan ser leídas directamente desde la base de datos PostgreSQL.
3	Gestión de Sesiones y Control de Accesos por Rol	Seguridad	El sistema asigna roles diferenciados (Administrador, Operador, Supervisor, Analista y Distribuidor) y restringe el acceso a cada módulo según el perfil. Cada usuario solo puede acceder a las funcionalidades autorizadas para su rol.
4	Protección CSRF y contra Inyección SQL	Seguridad	Laravel implementa tokens CSRF en todos los formularios del sistema y utiliza Eloquent ORM con consultas preparadas, protegiendo la aplicación contra ataques de inyección SQL y falsificación de solicitudes entre sitios.
5	Caducidad de Sesiones Inactivas	Seguridad	El sistema cierra automáticamente la sesión de los usuarios después de un período de inactividad configurado, previniendo accesos no autorizados en estaciones de trabajo desatendidas dentro de la planta de INAL.

6	Respaldo de la Base de Datos	Seguridad	Se realizan copias de seguridad periódicas de la base de datos PostgreSQL, evitando la pérdida de información crítica relacionada con detecciones, alertas, inventario y trazabilidad de productos.
7	Registro de Evidencia Fotográfica Segura	Seguridad	Las imágenes de productos defectuosos capturadas por el sistema se almacenan en una ruta protegida del servidor, accesible únicamente a través de rutas autenticadas de la aplicación Laravel, evitando accesos directos no autorizados.

Tabla 16. Seguridad del Software
Fuente: *Elaboración propia*

5.4.2. Pruebas de software

5.4.2.1. Prueba de Caja Negra

Durante la fase de validación del sistema, se empleó la técnica de prueba de caja negra para verificar el comportamiento funcional del sistema desde la perspectiva del usuario. El objetivo fue comprobar que las distintas funcionalidades del sistema respondan correctamente ante diferentes tipos de entradas, errores y condiciones reales de uso, evaluando especialmente la interacción con el usuario, la gestión de datos, el control de accesos por rol y el sistema web desarrollado en Laravel.

Las pruebas se organizaron por sprint, evaluando cada módulo de forma independiente una vez completado su desarrollo. Los módulos evaluados corresponden a los siete sprints planificados: Sesiones, Detección Automática, Monitoreo en Tiempo Real, Alertas Automáticas, Inventario de Productos, Trazabilidad y Reportes

5.4.2.2. Prueba de Caja Negra Sprint Nº1 Módulo Sesiones

Id	Caso de Prueba	Descripción	Fecha	Área Funcional / Sub proceso	Funcionalidad / Característica	Datos / Acciones de Entrada	Resultado Esperado	Requerimientos de Ambiente de Pruebas	Procedimientos especiales requeridos	Dependencias con otros casos de Prueba	Resultado Obtenido
1.1	iniciar sesión	Acceder al sistema con credenciales válidas.	20/04/2026	Sesiones	Autenticación	Ingresar correo y contraseña válidos..	El sistema inicia sesión y permite continuar.	Sistema activo. Usuario registrado.	Ingresar Ejecutar	Ninguna	Se inició sesión correctamente.
1.2	validar usuario y contraseña	Verificar que las credenciales pertenezcan a un usuario registrado.	20/04/2026	Sesiones	Validación de credenciales	Ingresar credenciales registradas y enviar el formulario.	El sistema valida usuario y contraseña.	Sistema activo. Usuario registrado.	Ingresar	de iniciar sesión	Se validaron las credenciales ingresadas.
1.3	verificar usuario activo	Comprobar que la cuenta se encuentre habilitada.	20/04/2026	Sesiones	Control de estado	Ingresar con un usuario activo.	El sistema confirma que el usuario está activo.	Sistema activo. Usuario registrado.	Ingresar	de validar usuarios	La cuenta fue reconocida como activa.
1.4	cargar roles de los usuarios	Obtener los roles asignados al usuario autenticado.	20/04/2026	Sesiones	Gestión de roles	Ingresar al sistema con un usuario con rol asignado.	El sistema carga los roles del usuario.	Sistema activo. Usuario registrado.	Ingresar	de verificar usuarios	Los roles del usuario fueron cargados.
1.5	validar permisos	Validar el acceso según el rol del usuario.	20/04/2026	Sesiones	Control de permisos	Acceder a una opción permitida para el rol.	El sistema autoriza la operación.	Sistema activo. Usuario registrado.	Ingresar visualizar	de cargar roles de usuarios	Los permisos fueron validados.
1.6	redirigir al dashboard segun el rol	Enviar al usuario al panel correspondiente.	20/04/2026	Sesiones	Redirección por rol	Finalizar el inicio de sesión con rol válido.	El sistema muestra el dashboard correspondiente.	Sistema activo. Usuario registrado.	Ingresar visualizar	de los permisos validados	Se mostró el dashboard según el rol.

1.7	acceder al sistema	Permitir el ingreso al sistema después de autenticar.	20/04/2026	Sesiones	Acceso al sistema	Completar el inicio de sesión exitosamente.	El usuario accede al sistema.	Sistema activo. Usuario registrado.	Completar	de redirigir al rol asignado	El acceso al sistema fue permitido.
1.8	gestionar roles	Administrar roles disponibles para los usuarios.	20/04/2026	Sesiones	Administración de roles	Ingresar como Administrador y abrir gestión de roles.	El sistema muestra opciones para gestionar los roles.	Sistema activo. Usuario registrado.	Ingresar	de acceder al sistema	Se visualizaron las opciones de roles.
1.9	mostrar error de autenticación	Informar credenciales incorrectas.	20/04/2026	Sesiones	Autenticación inválida	Ingresar usuario o contraseña incorrectos.	El sistema muestra un error de autenticación.	Sistema activo. Usuario registrado.	Ingresar	de gestionar los roles	Se mostró el error de autenticación.
1.10	bloquear acceso	Impedir el ingreso de un usuario inactivo.	20/04/2026	Sesiones	Bloqueo de acceso	Intentar ingresar con usuario inactivo.	El sistema bloquea el acceso.	Sistema activo. Usuario registrado.	Ingresar Validar	después de autenticación	El acceso fue bloqueado.
1.11	denegar acceso	Negar operaciones sin permiso suficiente.	20/04/2026	Sesiones	Denegación de permisos	Intentar abrir una opción no permitida para el rol.	El sistema deniega el acceso.	Sistema activo. Usuario registrado.	Ingresar Validar	de bloquear acceso	El acceso fue denegado por permisos.

Tabla 17. Prueba de Caja Negra Sprint Nº1 Módulo Sesiones

Fuente: *Elaboración propia*

5.4.2.3. Prueba de Caja Negra Sprint Nº5 Módulo Inventario

Id	Caso de Prueba	Descripción	Fecha	Área Funcional / Sub proceso	Funcionalidad / Característica	Datos / Acciones de Entrada	Resultado Esperado	Requerimientos de Ambiente de Pruebas	Procedimientos especiales requeridos	Dependencias con otros casos de Prueba	Resultado Obtenido
5.1	consultar inventario	Abrir inventario de productos.	27/04/2026	Inventario de Productos	Consulta de inventario	Ingresar al módulo Inventario.	El sistema muestra el inventario.	Sistema activo. Usuario Administrador.	Ingresar	Ninguna	El inventario fue mostrado.
5.2	totales de inventario	Visualizar cantidades generales.	27/04/2026	Inventario de Productos	Totales	Abrir resumen de inventario.	El sistema muestra totales de inventario.	Sistema activo. Usuario Administrador.	Ingresar	de consultar inventario	Los totales de inventario fueron mostrados.
5.3	buscar productos	Buscar producto por nombre o identificador.	27/04/2026	Inventario de Productos	Búsqueda de productos	Ingresar nombre o ID y presionar 'Buscar'.	El sistema muestra coincidencias.	Sistema activo. Usuario Administrador.	Ingresar Ejecutar	Ninguna	La búsqueda mostró coincidencias.
5.4	filtraciones	Aplicar filtros sobre productos.	27/04/2026	Inventario de Productos	Filtración	Seleccionar filtros y aplicar.	El sistema filtra los productos.	Sistema activo. Usuario Administrador.	Seleccionar	Ninguna	Los productos fueron filtrados.
5.5	registro de productos	Registrar un nuevo producto inspeccionado.	27/04/2026	Inventario de Productos	Registro de producto	Completar datos del producto y guardar.	El sistema registra el producto.	Sistema activo. Usuario Administrador.	Completar Guardar	Ninguna	El producto fue registrado.
5.6	actualizar inventario	Actualizar datos del inventario.	27/04/2026	Inventario de Productos	Actualización	Modificar datos de producto y guardar.	El sistema actualiza el inventario.	Sistema activo. Usuario Administrador.	Guardar	Ninguna	El inventario fue actualizado.

5.7	error al registrar producto	Controlar registro con datos inválidos.	27/04/2026	Inventario de Productos	Error de registro	Guardar productos con campos obligatorios.	El sistema informa error de registro.	Sistema activo. Usuario Administrador.	Guardar	Ninguna	Se informó el error al registrar el producto.
5.8	almacenar producto en base de datos	Guardar el producto en la base de datos.	27/04/2026	Inventario de Productos	Almacenamiento	Registrar producto válido.	El sistema almacena el producto.	Sistema activo. Usuario Administrador	Guardar	Ninguna	El producto fue almacenado en base de datos.
5.9	producto conforme	Registrar clasificación conforme.	27/04/2026	Inventario de Productos	Conformidad	Clasificar producto sin defectos.	El sistema marca el producto conforme.	Sistema activo. Usuario Administrador.	Ingresar visualizar	Ninguna	El producto quedó como conforme.
5.10	producto no conforme	Registrar clasificación no conforme.	27/04/2026	Inventario de Productos	No conformidad	Clasificar producto con defecto.	El sistema marca producto no conforme.	Sistema activo. Usuario Administrador.	Ingresar visualizar	Ninguna	El producto quedó como no conforme.

Tabla 18. Prueba de Caja Negra Sprint Nº5 Módulo Inventario

Fuente: *Elaboración propia*

5.4.2.4. Prueba de Caja Negra Sprint Nº6 Módulo Trazabilidad

Id	Caso de Prueba	Descripción	Fecha	Área Funcional / Sub proceso	Funcionalidad / Característica	Datos / Acciones de Entrada	Resultado Esperado	Requerimientos de Ambiente de Pruebas	Procedimientos especiales requeridos	Dependencias con otros casos de Prueba	Resultado Obtenido
6.1	Consultar trazabilidad	buscar todos los movimientos de los ingresos y salidas	4/5/2026	trazabilidad	consultar los ingresos y salidas	seleccionar el el boton de consultar trazabilidad	el sistema carga y muestra el panel de trazabilidad	sistema muestra productos registrados	buscar, adicionar, productos que se muestran	sólo de movimientos	hace los movimientos completos
6.2	buscar producto	buscar un producto específico por nombre o codigo	4/5/2026	trazabilidad	búsqueda de productos	Ingresar nombre o ID de un producto existente	el sistema muestra el producto con su historial completo	producto registrado en inventario	buscar y ingresar	de ninguna	hace la búsqueda correctamente
6.3	buscar por lote	buscar todos los productos de un lote de producción	4/5/2026	trazabilidad	búsqueda por lote	ingresar los números válidos y buscar	el sistema lista todos los productos del lote con estados y defectos	lotes con productos registrados	ingresar y buscar	de ninguna	hace la búsqueda correctamente por lote
6.4	filtraciones	aplicar filtros por tipo de defecto, fecha y línea	4/5/2026	trazabilidad	filtraciones de resultados	selecciona tipo de defecto, fechas de entrada y salidas	el sistema tiene que mostrar solo los registros	productos con distintos defectos y fechas registrados	ingresar , buscar y filtrar	de ninguna	hace las filtraciones correctas por el filtro seleccionado
6.5	obtener datos desde el inventario	el módulo carga automáticamente los productos del inventario	4/5/2026	trazabilidad	integración con inventario	ingresa al módulo de trazabilidad	el sistema tiene que mostrar los datos que están en el inventario	mostrar productos registrados	buscar	módulo de inventario de los productos	manda los datos desde el inventario

6.6	Marcar como entregado	Confirmar la entrega del producto en destino.	4/5/2026	Trazabilidad	Confirmación de entrega	Seleccionar producto con estado 'Enviado'. Presionar 'Marcar como entregado'.	El sistema actualiza el estado a 'Entregado' y registra la fecha de entrega en el historial.	Producto con estado 'Enviado' registrado.	Seleccionar confirmar	de ninguno	muestra cómo entregado al rol asignado
6.7	Registrar fecha de envío, responsable y asignar destino	Completar los datos de distribución del producto.	4/5/2026	Trazabilidad	Datos de distribución	Completar los campos que se muestran en el sistema	El sistema registra los datos de distribución y los asocia al producto enviado.	Producto seleccionado para distribución.	Completar guardar	de ninguna	registra el responsable, el distribuidor y la fecha
6.8	Destino de distribución	Visualizar el destino asignado a un producto enviado.	4/5/2026	Trazabilidad	Consulta de destino	Seleccionar producto con estado 'Enviado' y ver 'Destino de distribución'.	El sistema muestra la sucursal o destino al que fue enviado el producto.	Producto con destino de distribución asignado.	Seleccionar visualizar	de registrar fecha de envío y responsable	muestra el destino que fue asignado y guardado

Tabla 19. Prueba de Caja Negra Sprint Nº6 Módulo Trazabilidad

Fuente: *Elaboración propia*

5.4.2.4. Prueba de Caja Negra Sprint Nº7 Módulo Reportes

Id	Caso de Prueba	Descripción	Fecha	Área Funcional / Sub proceso	Funcionalidad / Característica	Datos / Acciones de Entrada	Resultado Esperado	Requerimientos de Ambiente de Pruebas	Procedimientos especiales requeridos	Dependencias con otros casos de Prueba	Resultado Obtenido
7.1	acceder al dashboard correspondiente	Abrir dashboard según el rol del usuario.	11/05/2026	Reportes	Dashboard	Ingresar al sistema con rol válido.	El sistema muestra el dashboard correspondiente.	Sistema activo. Datos de módulos disponibles.	Ingresar	Ninguna	El dashboard correspondiente fue mostrado.
7.2	ingresar al módulo de inventario	Ingresar al módulo de inventario desde reportes.	11/05/2026	Reportes	Acceso a inventario	Seleccionar módulo de inventario.	El sistema abre el módulo de inventario.	Sistema activo. Datos de módulos disponibles.	Seleccionar	ninguna	Se ingresó al módulo de inventario.
7.3	seleccionar tipo de reporte	Elegir el tipo de reporte a generar.	11/05/2026	Reportes	Tipo de reporte	Seleccionar tipo de reporte disponible.	El sistema registra el tipo de reporte.	Sistema activo. Datos de módulos disponibles.	Seleccionar	ninguna	El tipo de reporte fue seleccionado.
7.4	generar reporte	Procesar información para crear reporte.	11/05/2026	Reportes	Generación	Presionar 'Generar reporte'.	El sistema genera el reporte.	Sistema activo. Datos de módulos disponibles.	Ejecutar	ninguna	El reporte fue generado.
7.5	descargar reporte	Descargar archivo del reporte.	11/05/2026	Reportes	Descarga	Presionar 'Descargar reporte'.	El sistema descarga el archivo.	Sistema activo. Datos de módulos disponibles.	Ejecutar	ninguna	El reporte fue descargado.

Tabla 20. Prueba de Caja Negra Sprint Nº7 Módulo Reportes

Fuente: *Elaboración propia*

5.4.3. Métricas de Calidad del Software

La aplicación del estándar ISO/IEC 25010 permite evaluar de manera objetiva y cuantitativa los atributos de calidad del sistema desarrollado. A continuación, se presentan los cálculos y resultados obtenidos para cada métrica de calidad aplicada al Sistema de Inspección y Detección de Productos Dañados de INAL.

El estándar ISO/IEC 25010 define la calidad del software como un conjunto de seis características globales:

- **Funcionalidad.** Las funciones del software son aquellas que buscan satisfacer las necesidades del usuario/cliente.
- **Confiabilidad.** La capacidad del software de mantener su rendimiento bajo ciertas condiciones durante un período de tiempo específico.
- **Usabilidad.** Basada en la capacidad de entender el manejo y el funcionamiento del *software* por parte del usuario.
- **Eficiencia.** Basada en la relación entre el nivel de rendimiento del *software* y el volumen de recursos utilizado, bajo ciertas condiciones. “FIABILIDAD”
- **Capacidad de mantenimiento**. Basada en el esfuerzo necesario para realizar modificaciones específicas.
- **Portabilidad.** Basada en la capacidad del software para ser transferido de un entorno a otro.

5.4.3.1. Funcionalidad

Para medir la funcionalidad del sistema se utilizó el método de Puntos de Función, que cuantifica el tamaño funcional del software a partir del análisis de sus dominios de información.

Satisfacer las necesidades, los puntos de función (PF) se describen como medidas básicas desde donde calculan métricas de productividad.

Para estimaciones de PF, la descomposición funciona de la siguiente manera:

DOMINIO DE INFORMACIÓN	DESCRIPCIÓN
Número de entradas de usuario	Peticiones interactivas que generan respuesta inmediata del sistema: login, registro de usuarios, configuración de cámaras, filtros de inventario, registro de productos, alertas, trazabilidad, reportes y distribución.
Número de archivos	Archivos maestros lógicos en la base de datos PostgreSQL: Usuarios, Roles, Lotes, Productos, Detecciones, Alertas, Cámaras, LíneasProducción, InventarioProducto, Distribuciones, Destinos, ModelosDeteccion, TiposDefectos y ConfirmacionAlerta.
Número de interfaces externas	Interfaces legibles por máquina: módulo de visión artificial, servicio de detección y módulo de exportación de reportes.
Número de salidas de usuario	Salidas que proporcionan información orientada a la aplicación: dashboard de producción, alertas visuales/sonoras, historial de detecciones con evidencia fotográfica, inventario de conformes/no conformes, reportes estadísticos y exportación PDF/Excel.

Tabla 21. Dominio de información de punto de función

Fuente: *Elaboración propia*

En la siguiente tabla se presentan los valores de entrada utilizados para el cálculo.

Entradas para el cálculo de Puntos de Función

Dominio de Información	Cantidad
Número de entradas de usuario	35
Número de salidas de usuario	28
Número de peticiones de usuarios	22
Número de archivos	27
Número de interfaces externas	3

Tabla 21. Entradas para el cálculo de funcionalidad
Fuente: *Elaboración propia*

Aplicando los factores de ponderación correspondientes al nivel de complejidad medio de cada dominio de información, se obtiene el cálculo de puntos de cuenta total detallado en la siguiente tabla.

Cálculo de Puntos de Cuenta Total

Parámetro de Medición	Cantidad	Factor Simple	Factor Medio	Factor Complejo	Resultado
Entradas de usuario	35	3	4	6	140
Salidas de usuario	28	4	5	7	140
Peticiones de usuarios	22	3	4	6	88
Archivos	27	7	10	15	270
Interfaces externas	3	5	7	10	21
TOTAL					659

Tabla 22. Cálculo de puntos de cuenta total
Fuente: *Elaboración propia*

La sumatoria del factor de ajuste de complejidad técnica (Fi) se obtiene evaluando 14 características de sistema, resultando en un valor de Fi = 48. Aplicando la fórmula de Puntos de Función real:

PF (real) = 659 × (0.85 + 0.01 × 48) = 659 × 1.33 = 876.47

Para calcular el porcentaje de funcionalidad se obtiene el PF esperado con un nivel de confianza del 100%:

PF (esperado) = 659 × (1 + 0.01 × 48) = 659 × 1.48 = 975.32

El porcentaje de funcionalidad del sistema es:

%PF = 876.47 / 975.32 × 100% = 89.9% ≈ 90%

5.4.3.2. Confiabilidad

De acuerdo con la escala de punto de función, dado que PF ≥ 300, el nivel de funcionalidad del sistema se clasifica como ÓPTIMO, lo que indica que el sistema desarrollado satisface de manera sobresaliente los requerimientos funcionales establecidos.

La confiabilidad del sistema se calcula como la probabilidad de que opere sin fallos durante un período determinado. Utilizando la función de confiabilidad con los datos obtenidos del sistema y considerando una tasa de fallos λ basada en las pruebas realizadas:

F(t) = Fc × e^(-λ × 7 × 12)

Donde Fc = 0.90 (funcionalidad del sistema) y λ = 3 (tasa de fallos observada en 7 ejecuciones dentro de 1 mes). Reemplazando:

F(t) = 0.90 × e^(-3/7×12) = 0.0052

La probabilidad de no hallar una falla es: 1 − F(t) = 1 − 0.0052 = 0.9948 ≈ 0.99. El sistema presenta una confiabilidad del 99%, lo que indica que en el 99% de las ocasiones el sistema opera sin presentar fallos significativos.

5.4.3.3. Fiabilidad

La fiabilidad es fundamental para garantizar la reutilización confiable del software. Se calcula mediante la siguiente ecuación, considerando el número de errores detectados durante las pruebas y el total de líneas de código del sistema:

Fiabilidad = 1 - (Número de errores / Líneas de código × cuenta total)

Con base en los datos recopilados durante las fases de prueba del sistema (3 errores detectados en 3750 líneas de código):

Fiabilidad = 1 - (3 / 3750 × 0.0028) = 1 - 0.0028 = 0.9972

Se concluye que el sistema es un 99.72% fiable, lo que garantiza un comportamiento estable y predecible durante su operación en el entorno productivo de INAL.

5.4.3.4. Usabilidad

La usabilidad se evaluó mediante la participación de 8 usuarios representativos del sistema (operadores, supervisores, analistas y administradores), quienes calificaron la facilidad de comprensión y la facilidad de operación del sistema en una escala del 0% al 100%. Los resultados se presentan en la siguiente tabla.

Resultados de la Facilidad de Uso del Sistema

Usuario	Facilidad de Comprensión	Facilidad de Operación
Operador 1	100%	95%
Operador 2	95%	90%
Supervisor 1	100%	95%
Supervisor 2	90%	90%
Analista 1	95%	95%
Analista 2	90%	85%
Administrador 1	100%	95%
Distribuidor 1	85%	85%
PROMEDIO	94.4%	91.3%
TOTAL	92.8%	

Tabla 23. Resultados de facilidad de uso del sistema
Fuente: *Elaboración propia*

El resultado obtenido de un 92.8% de usabilidad indica que el sistema es comprendido y operado con facilidad por casi la totalidad de los usuarios evaluados, lo que confirma que la interfaz diseñada cumple con los criterios de usabilidad establecidos por el estándar ISO/IEC 25010.

5.4.3.5. Capacidad de movimiento

La capacidad de mantenimiento evalúa la facilidad con la que pueden realizarse modificaciones en el sistema, ya sea para corregir errores, incorporar mejoras o adaptarlo a nuevos requerimientos. Se mide mediante el Tiempo de Capacidad de Mantenimiento (TCM), calculado como:

TCM = TA + TD + TI + TP

Donde TA es el tiempo de análisis de la petición de cambio, TD es el tiempo de diseño de la modificación, TI es el tiempo de implementación y TP es el tiempo de prueba y distribución del cambio.

Mantenimiento Correctivo. Se calculó el tiempo de corrección en el mejor y el peor de los casos, expresado en horas:

- Mejor caso: TCM = 0.20 + 0.15 + 0.20 + 0.20 = 0.75 horas (1 hora y 15 minutos).
- Peor caso: TCM = 0.40 + 0.30 + 0.40 + 0.40 = 1.50 horas (1 hora y 30 minutos).

Mantenimiento Adaptativo. Para adaptaciones del sistema a nuevos entornos operativos, los tiempos estimados son:

- Mejor caso: TCM = 8 horas (un día laboral completo).
- Peor caso: TCM = 24 horas (tres días laborales).

Mantenimiento Preventivo. Dado que el sistema fue implementado recientemente, el mantenimiento preventivo no fue aplicado durante el período de evaluación, ya que este tipo de mantenimiento se recomienda para sistemas con cierto tiempo de operación continua.

5.4.3.6. Portabilidad

La portabilidad del sistema fue evaluada considerando sus tres subcaracterísticas principales: facilidad de instalación, facilidad de ajuste y facilidad de adaptación al cambio.

Facilidad de Instalación. El sistema fue desarrollado como una aplicación web basada en Laravel con base de datos PostgreSQL, lo que le permite ser desplegado en cualquier servidor que soporte PHP y PostgreSQL, independientemente del sistema operativo subyacente. El único requisito fundamental es que el servidor soporte la integración de PostgreSQL como gestor de base de datos.

Factor	Descripción	Peso
T1	Sistema distribuidor	4
T2	Objetivo de performance a tiempo de respuesta	5
T3	Eficiencia del usuario final	4
T4	Procesamiento interno complejo	5
T5	El código debe ser reutilizable	4
T6	Facilidad de instalación	3
T7	Facilidad de uso	4
T8	Portabilidad	4
T9	Facilidad de cambio	4
T10	Concurrencia	4
T11	Objetivos especiales de seguridad	4

T12	Acceso directo a terceras partes	5
T13	Facilidades especiales de entrenamiento a usuarios	3

Tabla 24. Factor Técnico
Fuente: *Elaboración propia*

Facilidad de Ajuste. Los ajustes al sistema se realizan mediante la creación de nuevos módulos o el rediseño de los existentes. Un ajuste completo del sistema se estima en un promedio de 25 días hábiles una vez superado el período de adecuación inicial.

Facilidad de Adaptación. Al tratarse de una aplicación web dinámica, el sistema puede adaptarse a cambios en los registros de la base de datos y modificaciones en sus funcionalidades. El 90% de las funcionalidades del sistema son configurables por los usuarios con permisos de administrador, lo que garantiza un alto grado de adaptabilidad.

En la siguiente tabla se detalla los valores obtenidos de las Métricas de Calidad del Software

Métrica de Calidad	Valor Obtenido
Funcionalidad	90%
Confiabilidad	99%
Fiabilidad	99%
Usabilidad	92%
Capacidad de Mantenimiento	98%
Portabilidad	87%

Tabla 25. Resumen de métricas de calidad del software - ISO/IEC 25010
Fuente: *Elaboración propia*

5.4.4. Presupuesto del Proyecto

La estimación del costo del proyecto se realizó mediante el Modelo COCOMO II en su variante Post-Arquitectónica, dado que el sistema se encuentra diseñado y en fase de desarrollo.

Este modelo proporciona estimaciones detalladas del esfuerzo, tiempo y costo requeridos para completar el proyecto.

5.4.4.1. Cálculo de Puntos de Función Ajustados

Utilizando el valor de Cuenta Total obtenido en el análisis de funcionalidad (PFSA = 659) y la sumatoria del factor de ajuste (Fi = 48), se calcula el Punto de Función Ajustado mediante la fórmula:

$$PFA = PFSA \times (0.65 + 0.01 \times Fi)$$

$$PFA = 659 \times (0.65 + 0.01 \times 48) = 659 \times 1.13 = 744.67 \approx 745$$

5.4.4.2. Cálculo de Kilo Líneas de Código (KDLC)

Para determinar el número de kilos de líneas de código se utiliza el factor de conversión correspondiente a los lenguajes de programación orientados a objetos (30 LDC/PF), de acuerdo con la tabla estándar de conversión de Punto de Función a Líneas de Código:

$$KDLC = (LDC \times PFA) / 1000 = (30 \times 745) / 1000 = 22.35$$

Línea de Código – Punto de Función

Lenguaje	LDC (Medio)
Ensamblador	320
C	128
COBOL	105
Pascal	90
Lenguajes Orientados a Objetos (PHP, Python)	30

Generadores de Código	15

Tabla 26. Cálculo de kilo de líneas de código
Fuente: *Elaboración propia*

5.4.4.3. Factores de Escala

Los factores de escala determinan el exponente B de la fórmula de esfuerzo, reflejando características del proyecto que tienen un impacto exponencial en el esfuerzo requerido. La selección de los niveles para cada factor se basó en las características del presente proyecto, tal como se muestra en la siguiente tabla.

Factores de Escala – COCOMO II

Factor de Escala	Nivel Seleccionado	Valor
Precedentes (PREC)	Alto	2.48
Flexibilidad de Desarrollo (FLEX)	Muy Alto	1.01
Resolución de Riesgos y Arquitectura (RESL)	Muy Alto	1.41
Cohesión del Equipo (TEAM)	Muy Alto	1.10
Maduración del Proceso (PMAT)	Muy Alto	1.56
TOTAL (ΣSFi)		7.56

Tabla 27. Factores de escala - COCOMO II
Fuente: *Elaboración propia*

Con la sumatoria de factores de escala (ΣSFi = 7.56), se calcula el exponente B:

$$B = 0.91 + 0.01 \times \Sigma SFi = 0.91 + 0.01 \times 7.56 = 0.9856 \approx 0.99$$

5.4.4.4. Multiplicadores de Esfuerzo

Los multiplicadores de esfuerzo (EMi) reflejan características del producto, de la plataforma, del personal y del proyecto que afectan el esfuerzo de desarrollo. Considerando las características del presente proyecto (un solo desarrollador con conocimientos en las tecnologías utilizadas, complejidad media del software y restricciones moderadas), el producto de los 17 multiplicadores de esfuerzo resulta en:

ΠEMi = 1.27

5.4.4.5. Cálculo de Esfuerzo

Aplicando la fórmula de esfuerzo del modelo Post-Arquitectónico de COCOMO II:

E = A × KDLC^B × ΠEMi

E = (2.94 × 22.35^0.99 × 1.27) / 10

E = (2.94 × 21.87 × 1.27) / 10 = 8.15 personas-mes

El esfuerzo calculado de 8.15 personas-mes indica que el proyecto requiere aproximadamente 8 meses de trabajo de una persona para completarse.

5.4.4.6. Cálculo del Tiempo de Desarrollo

Calculando primero el valor de d:

$$d = 0.28 + 0.002 \times (B - 0.91) = 0.28 + 0.002 \times (0.99 - 0.91) = 0.28 + 0.00016 \approx 0.28$$

Aplicando la fórmula de tiempo de desarrollo:

$$TDEV = C \times E^{(d - SCED\%/100)} = 3.67 \times 8.15^{(0.28 - 0.07)} = 3.67 \times 8.15^{0.21}$$

$$TDEV = 3.67 \times 1.72 = 6.31 \approx 6.5 \text{ meses}$$

El tiempo requerido para completar el proyecto es de aproximadamente 6 meses y medio.

5.4.4.7. Cálculo de Cantidad de Personas

$$P = E / T = 8.15 / 6.31 = 1.29 \approx 1 \text{ persona}$$

Los resultados indican que el proyecto puede ser desarrollado por 1 persona trabajando durante aproximadamente 7 meses, lo cual es coherente con la naturaleza del Proyecto Integrador Intermedio I, donde el sistema es desarrollado de forma individual.

5.4.4.8. Costo del Software

El costo del sistema se calcula mediante la fórmula:

$$Csis = P \times T \times Sp$$

Donde P es el número de personas, T es el tiempo de desarrollo en meses y Sp es el sueldo promedio de un desarrollador de software en Bolivia (Bs. 3,000/mes):

Csis = 1.29 × 6.31 × 3,000 = Bs. 24,451.17

Costo Total del Software

Descripción	Costo (Bs.)
Costo del Sistema (desarrollo)	24,451.17
Alquiler de Hosting (anual)	200.00
Costo Total del Software en Bolivianos	24,651.17
Costo Total del Software en Dólares (tipo de cambio: Bs. 7/USD)	3,521.60

Tabla 28. Costo total del software
Fuente: *Elaboración propia*

5.4.4.9. Costo del Hardware

Para el sistema desarrollado se requiere un computador con capacidad suficiente para ejecutar el servidor web y el modelo de detección YOLOv8. A continuación, se detalla el costo del hardware necesario:

Componente	Descripción	Costo (Bs.)
Computadora (servidor)	Intel Core i5, 16GB RAM, SSD 512GB	6,500.00
Cámara IP industrial	Resolución HD, resistente al polvo (x2)	1,400.00
Accesorios de red	Switch, cables, conectores	350.00
Costo Total Hardware (Bs.)		8,250.00

Costo Total Hardware (USD)		1,178.57

Tabla 29. Costo total del hardware
Fuente: *Elaboración propia*

5.4.4.10. Costo de la Investigación

Material	Costo (Bs.)
Papelería	100.00
Pasajes	120.00
Impresiones	200.00
Fotocopias	80.00
Teléfono/Comunicaciones	143.00
Internet	190.00
Costo Total Investigación (Bs.)	833.00
Costo Total Investigación (USD)	119.00

Tabla 30. Costo de la Investigación
Fuente: *Elaboración propia*

5.4.4.11. Costo Total del Proyecto

El costo total del proyecto se determina mediante la siguiente ecuación:

CTP = CSW + CHW + CINV

CTP = Bs. 24,651.17 + Bs. 8,250.00 + Bs. 833.00 = Bs. 33,734.17

Resumen del Presupuesto Total del Proyecto

Concepto	Monto (Bs.)	Monto (USD)
Costo del Software	24,651.17	3,521.60
Costo del Hardware	8,250.00	1,178.57

Costo de la Investigación	833.00	119.00
COSTO TOTAL DEL PROYECTO	33,734.17	4,819.17

Tabla 31. Resumen del presupuesto total del proyecto
Fuente: *Elaboración propia*

III. CONCLUSIONES.

El desarrollo de la red neuronal convolucional para la detección automática de productos dañados se llevará a cabo en el siguiente semestre, una vez se cuente con el conjunto de datos necesario y la infraestructura adecuada para su ejecución.

El módulo de inventario de productos conformes y no conformes fue desarrollado en un 80%, permitiendo al supervisor de producción llevar un registro organizado del estado de los productos. Las funcionalidades base de registro y control se encuentran operativas, quedando pendientes algunos ajustes relacionados con la integración final con los resultados de la red neuronal.

El módulo de reportes e indicadores de desempeño fue desarrollado en su totalidad, proporcionando a la gerencia de INAL Ltda. dashboards y paneles de visualización que permiten acceder a información confiable y oportuna para la toma de decisiones estratégicas dentro del proceso productivo.

La interfaz de monitoreo en tiempo real fue diseñada e implementada; sin embargo, dado que la red neuronal aún no se encuentra integrada al sistema, el módulo opera actualmente con videos de prueba, permitiendo validar el funcionamiento visual de la interfaz y su capacidad de visualización del estado de la línea de producción.

El módulo de alertas automáticas fue desarrollado de forma parcial, habilitando el sistema de notificaciones para eventos generales del sistema con su respectivo registro e historial. No obstante, las alertas específicas relacionadas con la detección de productos defectuosos quedan pendientes de implementación hasta que la red neuronal sea integrada en el siguiente semestre.

IV. RECOMENDACIONES.

Se recomienda priorizar en el siguiente semestre el desarrollo e integración de la red neuronal convolucional, iniciando con la recolección y etiquetado de imágenes reales de los productos de INAL Ltda. para construir un conjunto de datos representativo.

Es altamente aconsejable integrar el inventario de productos inspeccionados con el sistema ERP contable de la corporación. De esta manera, cualquier posible discrepancia entre el stock físico y el teórico se actualizará automáticamente en la plataforma principal de la empresa.

Se sugiere a la gerencia establecer reuniones de revisión semanales exclusivas para analizar los dashboards generados. Esta práctica preventiva fomentará una mejora continua, permitiendo ajustar parámetros mecánicos en las máquinas envasadoras con base en estas tendencias observadas.

Resulta vital capacitar continuamente a los operadores en la interpretación correcta de la interfaz gráfica y alertas visuales. Además, se deben realizar mantenimientos físicos y limpiezas rutinarias a las lentes de las cámaras para garantizar una entrada de vídeo permanentemente nítida.

Se recomienda definir diferentes niveles de escalabilidad para las alertas automáticas generadas por el sistema inteligente. Si un problema persiste y genera múltiples notificaciones en

pocos minutos, la alerta máxima debería enviarse directamente al equipo de mantenimiento mecánico.

V. BIBLIOGRAFÍA.

Aguirre, F. E. (2024). Detección de defectos mediante análisis estadístico no-paramétrico de imágenes digitales y redes neuronales convolucionales. MAESA-Madrid.

Alaimo, D. (2021). Proyectos Ágiles con Scrum. Flexibilidad, aprendizaje y trabajo en equipo.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). El Lenguaje Unificado de Modelado: Guía del Usuario. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Patrones de diseño: Elementos de software orientado a objetos reutilizables. Addison-Wesley.

Haro, A. (2025). Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales con optimizaciones específicas para hardware NVIDIA. Universitat Politecnica de Valencia.

Jalote, P. (2022). An Integrated Approach to Software Engineering. Springer.

Laudon, K. C., & Laudon, J. P. (2020). Sistemas de información gerencial: administración de la empresa digital. Pearson Educación.

López, J. (2025). Diseño e implementación de una red neuronal para la detección de anomalías en piezas sobrantes de un proceso industrial. Universitat Politecnica de Valencia.

Sánchez, J., & Moya de la Torre. (2023). Implementación de una red neuronal para la detección de anomalías en bandejas. Universidade da coruña.

VI. WEBGRAFÍA.

Laravel. (2026). *Laravel: The PHP framework for web artisans*. https://laravel.com/

MKLabs. (2026). *StarUML: A sophisticated software modeler*. https://staruml.io/

Oracle Corporation. (2026). *MySQL*. https://www.mysql.com/

Python Software Foundation. (2026). *Welcome to Python.org*. https://www.python.org/

The PHP Group. (2026). *PHP: Hypertext Preprocessor*. https://www.php.net/

Ultralytics. (2026). *Ultralytics YOLOv8*. https://ultralytics.com/yolov8

VII. ANEXOS.

- **Cronograma de Actividades**

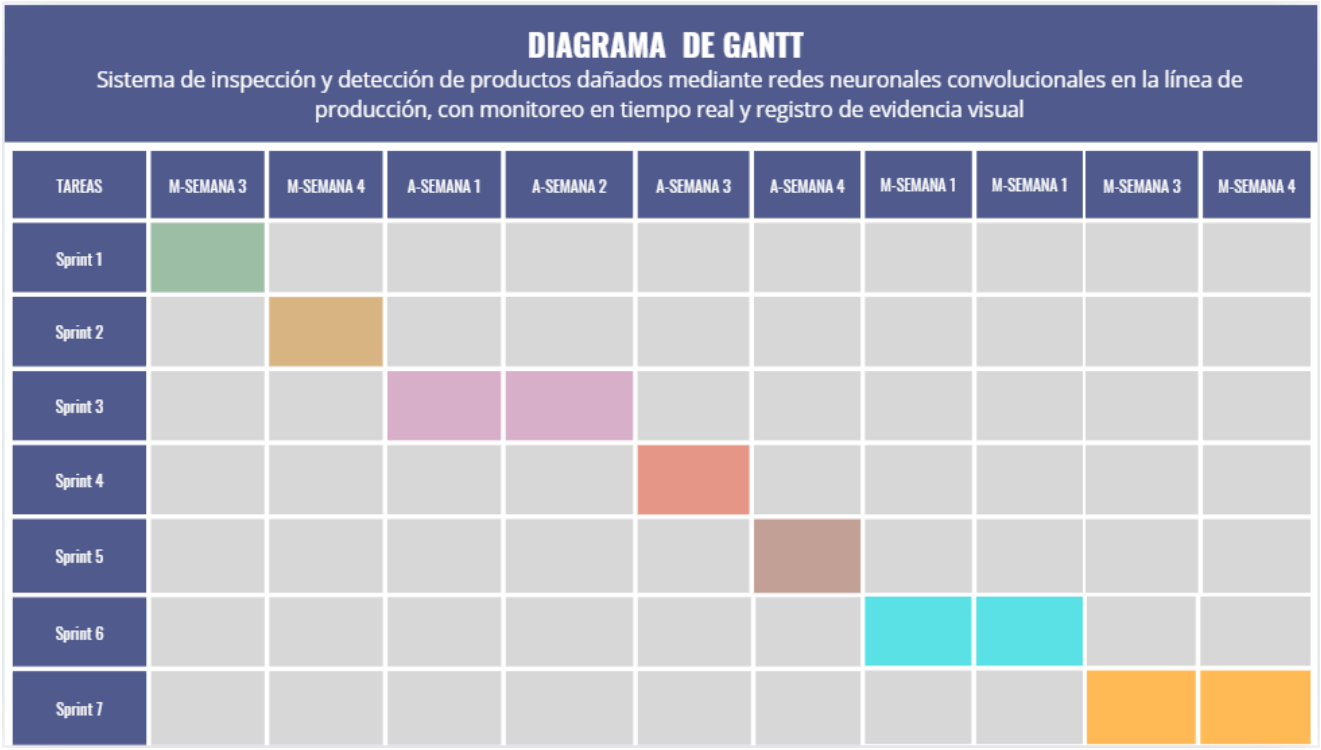


Figura 57. Cronograma de actividades

Fuente: *Elaboración propia*

- **Diccionario de Datos**

Tabla: camaras

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_camara	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(100)		Sí	NULL		
linea_id	bigint		Sí	NULL		FK → lineas_produccion
url_stream	varchar(300)		Sí	NULL		
estado	varchar(20)		Sí	NULL		

ubicacion	varchar(200)		Sí	NULL		
created_at	timestamp(0)	NOT NULL	No	CURRENT_TIMESTAMP		

Tabla 31. Tabla - Camaras

Fuente: *Elaboración propia*

Tabla: destinos

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_destino	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(150)		Sí	NULL		
ciudad	varchar(100)		Sí	NULL		
tipo	varchar(20)		Sí	NULL		
direccion	text		Sí	NULL		
activo	boolean	NOT NULL	No	true		

Tabla 32. Tabla - destinos

Fuente: *Elaboración propia*

Tabla: detecciones

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_deteccion	bigserial	NOT NULL	No		auto_increment	PK
camara_id	bigint		Sí	NULL		FK → camaras
modelo_id	bigint		Sí	NULL		FK → modelos_deteccion
tipo_defecto_id	bigint		Sí	NULL		FK → tipos_defecto
lote_id	bigint		Sí	NULL		FK → lotes

fecha_hora	timestamp(0)		Sí	NULL		
nivel_confianza	numeric(5,2)		Sí	NULL		
resultado	varchar(20)		Sí	NULL		
ruta_imagen	varchar(300)		Sí	NULL		

Tabla 34. Tabla - detecciones

Fuente: *Elaboración propia*

Tabla: distribuciones

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_distribucion	bigserial	NOT NULL	No		auto_increment	PK
lote_id	bigint		Sí	NULL		FK → lotes
destino_id	bigint		Sí	NULL		FK → destinos
responsable_id	bigint		Sí	NULL		FK → users
fecha_envio	timestamp(0)		Sí	NULL		
cantidad_productos	integer		Sí	NULL		
estado	varchar(20)		Sí	NULL		
observaciones	text		Sí	NULL		
salida_id	bigint		Sí	NULL		FK → salidas_revision
salida_detalle_id	bigint		Sí	NULL		FK → salida_detalles

Tabla 35. Tabla - distribuciones

Fuente: *Elaboración propia*

Tabla: ingreso_detalles

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_detalle_ingreso	bigserial	NOT NULL	No		auto_increment	PK
ingreso_id	bigint	NOT NULL	No			FK → ingresos_revision
producto_id	bigint	NOT NULL	No			FK → productos
lote_id	bigint		Sí	NULL		FK → lotes
cantidad	integer	NOT NULL	No			
cantidad_aprobada	integer	NOT NULL	No	0		
cantidad_rechazada	integer	NOT NULL	No	0		

Tabla 36. Tabla - ingreso _detalle

Fuente: *Elaboración propia*

Tabla: ingresos_revision

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_ingreso	bigserial	NOT NULL	No		auto_increment	PK
entregado_por_id	bigint	NOT NULL	No			FK → users
recibido_por_id	bigint	NOT NULL	No			FK → users
fecha_hora	timestamp(0)	NOT NULL	No	CURRENT_TIMESTAMP		
turno	varchar(20)		Sí	NULL		

observaciones	text		Sí	NULL		

Tabla 37. Tabla - ingreso_revisión

Fuente: *Elaboración propia*

Tabla: inventario_productos

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_inventario	bigserial	NOT NULL	No		auto_increment	PK
deteccion_id	bigint		Sí	NULL		FK → detecciones
producto_id	bigint		Sí	NULL		FK → productos
lote_id	bigint		Sí	NULL		FK → lotes
clasificacion	varchar(20)		Sí	NULL		
ruta_evidencia	varchar(300)		Sí	NULL		
fecha_registro	timestamp(0)		Sí	NULL		
cantidad	numeric(12,3)	NOT NULL	No	0		
ingreso_detalle_id	bigint		Sí	NULL		FK → ingreso_detalles
stock	numeric(12,3)	NOT NULL	No	0		
activo	boolean	NOT NULL	No	true		

Tabla 38. Tabla - inventario_productos

Fuente: *Elaboración propia*

Tabla: lineas_produccion

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_linea	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(100)		Sí	NULL		
descripcion	text		Sí	NULL		
estado	varchar(20)		Sí	NULL		
turno	varchar(20)		Sí	NULL		

Tabla 39. Tabla - lineas_produccion

Fuente: *Elaboración propia*

Tabla: lotes

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_lote	bigserial	NOT NULL	No		auto_increment	PK
codigo_lote	varchar(50)		Sí	NULL	UNIQUE	
linea_id	bigint		Sí	NULL		FK → lineas_produccion
usuario_id	bigint		Sí	NULL		FK → users
fecha_inicio	timestamp(0)		Sí	NULL		
fecha_fin	timestamp(0)		Sí	NULL		
turno	varchar(20)		Sí	NULL		
estado	varchar(20)		Sí	NULL		

producto_id	bigint		Sí	NULL		FK → productos
cantidad	numeric(12,3)	NOT NULL	No	0		
observaciones	text		Sí	NULL		

Tabla 40. Tabla - lotes

Fuente: *Elaboración propia*

Tabla: modelos_deteccion

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_modelo	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(100)		Sí	NULL		
version	varchar(20)		Sí	NULL		
ruta_archivo	varchar(300)		Sí	NULL		
umbral_confianza	numeric(5,2)		Sí	NULL		
activo	boolean		Sí	NULL		
descripcion	text		Sí	NULL		

Tabla 41. Tabla - modelos_deteccion

Fuente: *Elaboración propia*

Tabla: producto_recetas

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_receta	bigserial	NOT NULL	No		auto_increment	PK
producto_id	bigint	NOT NULL	No			FK → productos

materia_prima_id	bigint	NOT NULL	No			FK → productos
cantidad_por_unidad	numeric(12,3)	NOT NULL	No			
cantidad_total	numeric(12,3)	NOT NULL	No	0		
unidades_producidas	integer	NOT NULL	No	100		
unidad	varchar(50)		Sí	NULL		
created_at	timestamp(0)		Sí	NULL		
updated_at	timestamp(0)		Sí	NULL		

Tabla 42. Tabla - producto_resetas

Fuente: *Elaboración propia*

Tabla: productos

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_producto	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(150)		Sí	NULL		
codigo	varchar(50)		Sí	NULL	UNIQUE	
descripcion	text		Sí	NULL		
categoria	varchar(100)		Sí	NULL		
tipo_producto	varchar(100)		Sí	NULL		
presentacion	varchar(100)		Sí	NULL		
activo	boolean	NOT NULL	No	true		

Tabla 43. Tabla - productos

Fuente: *Elaboración propia*

Tabla: reportes

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_reporte	bigserial	NOT NULL	No		auto_increment	PK
generado_por	bigint		Sí	NULL		FK → users
tipo	varchar(100)		Sí	NULL		
parametros	jsonb		Sí	NULL		
fecha_generacion	timestamp(0)		Sí	NULL		
formato	varchar(20)		Sí	NULL		
ruta_archivo	varchar(300)		Sí	NULL		

Tabla 44. Tabla - reportes

Fuente: *Elaboración propia*

Tabla: roles

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_rol	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(50)		Sí	NULL		
descripcion	text		Sí	NULL		

Tabla 45. Tabla - roles

Fuente: *Elaboración propia*

Tabla: salida_detalles

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_detalle_salida	bigserial	NOT NULL	No		auto_increment	PK
salida_id	bigint	NOT NULL	No			FK → salidas_revision
producto_id	bigint	NOT NULL	No			FK → productos
lote_id	bigint		Sí	NULL		FK → lotes
cantidad	integer	NOT NULL	No			

Tabla 46. Tabla - salida_detalle
Fuente: *Elaboración propia*

Tabla: salidas_revision

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_salida	bigserial	NOT NULL	No		auto_increment	PK
nro_informe	varchar(50)	NOT NULL	No		UNIQUE	
despachado_por_id	bigint	NOT NULL	No			FK → users
distribuidor_id	bigint	NOT NULL	No			FK → users
destino_id	bigint		Sí	NULL		FK → destinos
fecha_hora	timestamp(0)	NOT NULL	No	CURRENT_TIMESTAMP		
turno	varchar(20)		Sí	NULL		
observaciones	text		Sí	NULL		

Tabla 47. Tabla - salida_revision

Fuente: *Elaboración propia*

Tabla: tipos_defecto

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id_tipo_defecto	bigserial	NOT NULL	No		auto_increment	PK
nombre	varchar(100)		Sí	NULL		
descripcion	text		Sí	NULL		
activo	boolean		Sí	NULL		

Tabla 48. Tabla - tipo_defecto

Fuente: *Elaboración propia*

Tabla: users

Columna	Tipo	Atributos	Nulo	Pred.	Extra	Enlaces
id	bigserial	NOT NULL	No		auto_increment	PK
name	varchar(255)	NOT NULL	No			
email	varchar(255)	NOT NULL	No		UNIQUE	
email_verified_at	timestamp(0)		Sí	NULL		
password	varchar(255)	NOT NULL	No			

remember_token	varchar(100)		Sí	NULL		
created_at	timestamp(0)		Sí	NULL		
updated_at	timestamp(0)		Sí	NULL		
otp_code	varchar(255)		Sí	NULL		
otp_expires_at	timestamp(0)		Sí	NULL		
phone	varchar(8)		Sí	NULL		
fecha_nacimiento	date		Sí	NULL		
nro_carnet	varchar(20)		Sí	NULL		
activo	boolean	NOT NULL	No	true		
face_photo_base64	text		Sí	NULL		
face_descriptor	json		Sí	NULL		
login_otp_code	varchar(6)		Sí	NULL		
login_otp_expires_at	timestamp(0)		Sí	NULL		
id_rol	bigint		Sí	NULL		FK → roles

Tabla 49. Tabla - users

Fuente: *Elaboración propia*

- **Árbol de Problemas**

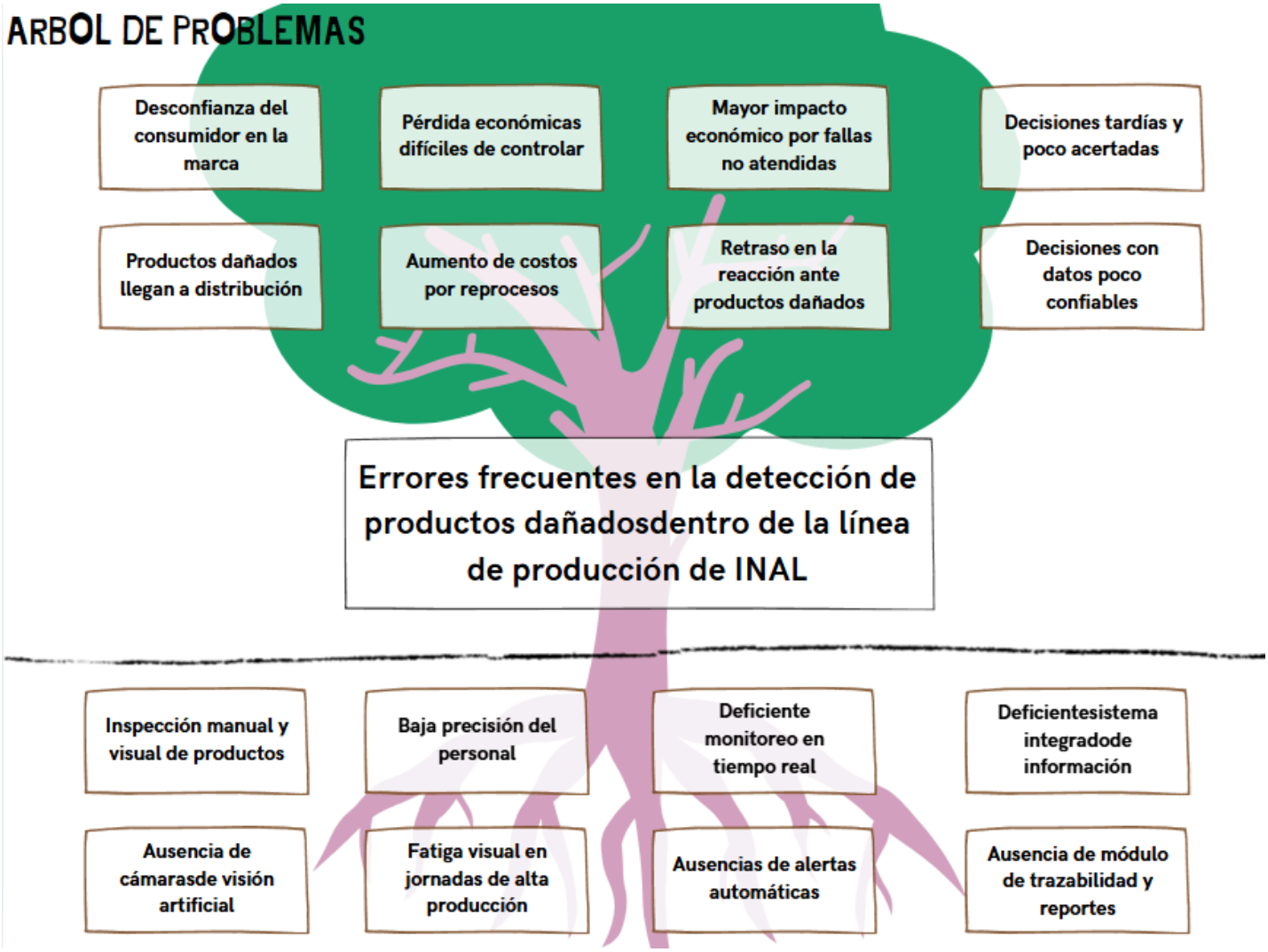


Figura 58. Árbol de problemas

Fuente: *Elaboración propia*

- **Árbol de Objetivos**

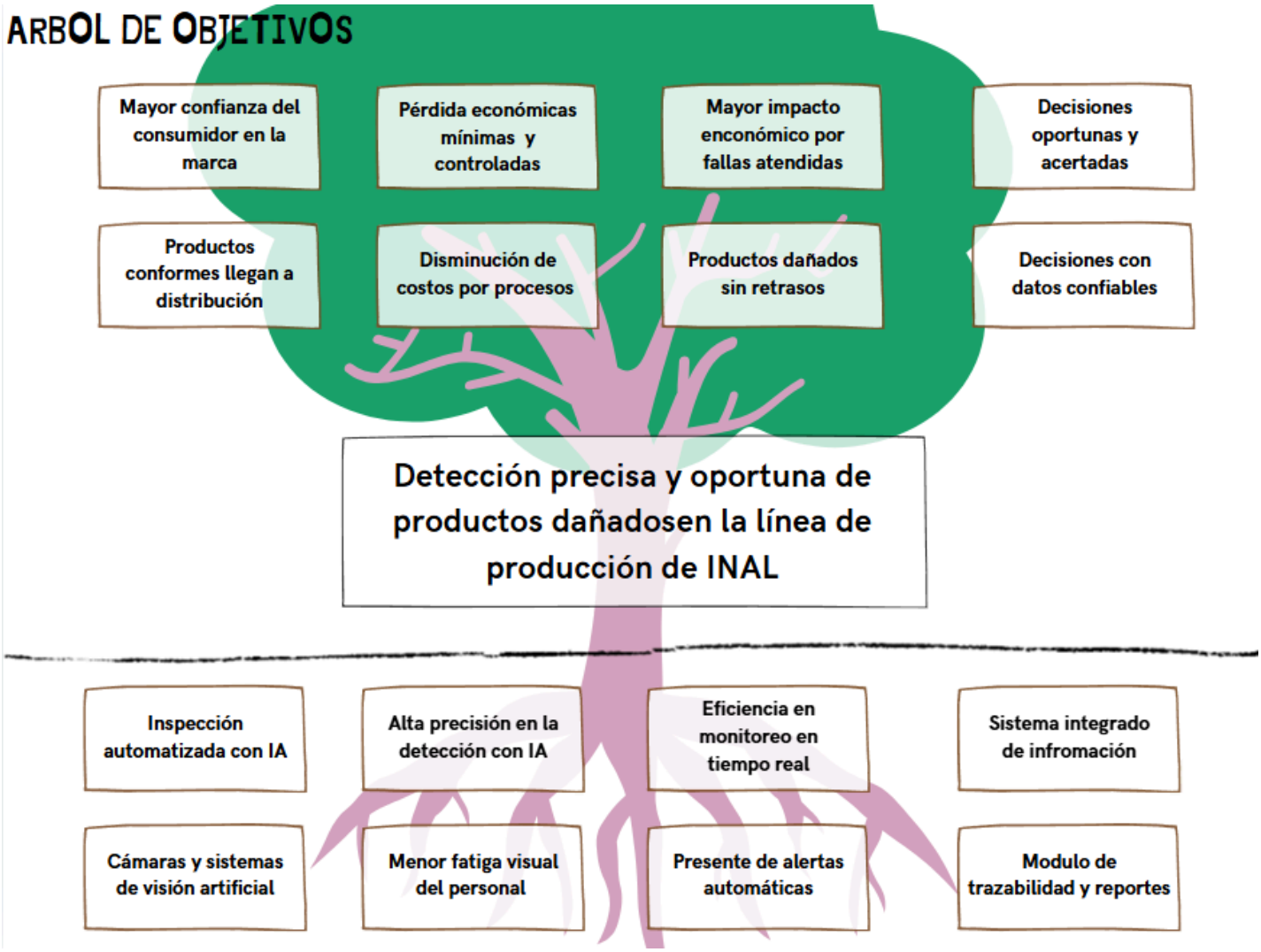


Figura 59. Árbol de Objetivos

Fuente: *Elaboración propia*